

10

Formulation and solution of the equations

This chapter describes the formulation and solution of the equations used by Eclipse:

- [Recovery mechanisms](#)
- [Formulation of the equations](#)
- [Solution of the linear equations](#)
- [Tracer tracking](#)
- [Diffusion](#)
- [Non-Darcy flow](#)
- [Non-Newtonian fluid flows](#)
- [Nine-point schemes](#)
- [Velocity calculations](#)

Recovery mechanisms

x	Eclipse 100
	Eclipse 300

The oil production from a reservoir will typically be due to a combination of different recovery mechanisms, for example rock compaction and water influx. Eclipse 100 provides a facility to quantify the proportion of oil produced by each physical process, accumulated during the simulation. This information is important both to understand the behavior of the reservoir and to allow greater investigation of the data uncertainty associated with each production mechanism.

The oil in place within a grid block is:

$$\text{Total oil in place} = PV \cdot S_o \cdot \frac{1}{B_o} \quad \text{Eq. 10.1}$$

$$S_o = 1 - S_w - S_g$$

where

PV is the pore volume

S_o is the oil saturation

B_o is the oil formation volume factor

S_w is the water saturation

S_g is the gas saturation

The change in oil in place can be attributed as follows:

$$\begin{aligned} DC &= -d(PV) \cdot S_o \frac{1}{B_o} \\ DW &= PV \cdot d(S_w) \frac{1}{B_o} \\ DG &= PV \cdot d(S_g) \frac{1}{B_o} \\ DE &= PV \cdot S_o \cdot d\left(\frac{1}{B_o}\right) \end{aligned} \quad \text{Eq. 10.2}$$

where

DC is the production due to rock compaction

DW is the production due to water influx

DG is the production due to gas influx

DE is the production due to oil expansion

$d(PV)$ is the change in pore volume over the timestep

$d(S_w)$ is the change in water saturation over the timestep

$d(S_g)$ is the change in gas saturation over the timestep

$d\frac{1}{B_o}$ is the change in formation volume factor over the timestep.

In the case of a saturated cell reducing in pressure, the oil expansion term, DE, will be negative as $1/B_o$ will increase. Hence if a cell is saturated then the (negative) production is accumulated with the gas influx:

If a cell is saturated then:

$$\begin{aligned} DG &= DG - DE \\ DE &= 0 \end{aligned} \quad \text{Eq. 10.3}$$

The oil production from each mechanism (DC, DW, DG, DE) is summed in each fluid-in-place region and also summed over timesteps. A table of production by each recovery mechanism can be output by using the RECOV mnemonic in the `RPTSCHEG` keyword, and appears under the heading of RECOVERY.

Limitations

At present the facility cannot be used in runs containing vaporized oil, (that is, with `VAPORIL` present in the RUNSPEC section).

Solution gas drive

Dividing the oil production between the above set of recovery mechanisms will give the total oil production due to gas influx, but this does not differentiate between the oil production associated with free gas influx and that associated with solution gas.

The oil production due to gas influx can be subdivided into oil produced by solution gas drive and oil produced by free gas, by using the tracer tracking facility. If a gas tracer is defined and initialized to be 1.0 for gas dissolved in the oil, and 0.0 for free gas, then the recovery mechanism option will use the tracer information to calculate the split between solution gas and free gas.

To use this facility the following steps must be taken:

1. Increase the number of gas tracers by one in the `TRACERS` keyword in the RUNSPEC section.
2. Add a tracer called 'DGS' in the `PROPS` section. The special name 'DGS' **must** be used to enable the recovery mechanism to recognize the tracer.

```
TRACER
'DGS' 'GAS' /
/
```

3. Initialize the tracer in the `SOLUTION` section, giving the solution gas a value of 1.0 and the free gas a value of 0.0, for example:

```
TBLKFDGS
300*0 /
TBLKSDGS
300*1 /
```

Two new oil production values associated with solution gas and free gas will be calculated as follows:

$$\begin{aligned} DS &= PV \cdot F_s \cdot d(S_g) \cdot \frac{1}{B_o} \\ DF &= PV \cdot (1 - F_s) \cdot d(S_g) \cdot \frac{1}{B_o} \end{aligned} \quad \text{Eq. 10.4}$$

where

DS is the production due to solution gas

DF is the production due to free gas

F_s is the fraction of S_g that was originally solution gas.

Water drive

A similar procedure is available to distinguish production from two different water sources, for example injected water and aquifer water. Again, a tracer is used to track (say) the aquifer water, and the production is subdivided into production associated with ‘traced’ water and production associated with the remaining water.

To use this facility the following steps must be taken:

1. Increase the number of water tracers by one in the [TRACERS](#) keyword in the RUNSPEC section.
2. Add a tracer called ‘WTR’ in the PROPS section. The special name ‘WTR’ **must** be used to enable the recovery mechanism to recognize the tracer.

```
TRACER  
  'WTR' 'WAT' /  
  /
```

3. Initialize the tracer in the SOLUTION section in the usual way using the [TBLK](#) or [TVDP](#) keywords, and set the tracer concentration for any analytic aquifers using the [AQANTRC](#) keyword.

The initial tracer concentration should be either 1.0 for ‘traced’ water, or 0.0 for the rest.

Two new oil production values associated with the traced water and the remaining water will be calculated as follows:

$$\begin{aligned} DWT &= PV \cdot F_t \cdot d(S_w) \cdot \frac{1}{B_o} \\ DWR &= PV \cdot (1 - F_t) \cdot d(S_w) \cdot \frac{1}{B_o} \end{aligned} \quad \text{Eq. 10.5}$$

where

DWT is the production due to traced water

DWR is the production due to remaining water

F_t is the fraction of S_w that is ‘traced’ water.

Summary output

The oil production attributable to each mechanism, and the corresponding fraction of the total oil production, can also be output to the SUMMARY file.

Refer to [Oil recovery mechanism](#) in the *Eclipse Reference Manual*.

Details of the recovery calculation

Eclipse solves the oil conservation equation:

$$d(\text{OIP}) = \text{PV}^{t+dt} \cdot S_o^{t+dt} \cdot \left[\frac{1}{B_o} \right]^{t+dt} - \text{PV}^t \cdot S_o^t \cdot \left[\frac{1}{B_o} \right]^t \quad \text{Eq. 10.6}$$

Some care is needed to associate the production mechanisms such that the total oil produced equals the change in oil-in-place, $d(\text{OIP})$. Below we show how the production associated with each recovery mechanism is calculated.

Let:

$$\begin{aligned} \text{PV}^{t+dt} &= \text{PV}^t + d(\text{PV}) \\ S_o^{t+dt} &= S_o^t + d(S_o) \\ \left[\frac{1}{B_o} \right]^{t+dt} &= \left[\frac{1}{B_o} \right]^t + d \left[\frac{1}{B_o} \right] \end{aligned} \quad \text{Eq. 10.7}$$

$$d(\text{OIP}) = [\text{PV} + d(\text{PV})][S_o + d(S_o)] \left[\frac{1}{B_o} + d \left(\frac{1}{B_o} \right) \right] - \left[\text{PV} \cdot S_o \cdot \frac{1}{B_o} \right]$$

Expanding the conservation equation,

$$\begin{aligned} d(\text{OIP}) &= d(\text{PV}) \cdot S_o^t \cdot \left[\frac{1}{B_o} \right]^t + d(\text{PV}) \cdot S_o^t \cdot d \left[\frac{1}{B_o} \right] \\ &+ \text{PV}^t \cdot d(S_o) \cdot \left[\frac{1}{B_o} \right]^t + d(\text{PV}) \cdot d(S_o) \cdot \left[\frac{1}{B_o} \right]^t + \text{PV}^t \cdot S_o^t \cdot d \left[\frac{1}{B_o} \right] \\ &+ \text{PV}^t \cdot d(S_o) \cdot d \left[\frac{1}{B_o} \right] + d(\text{PV}) \cdot d(S_o) \cdot d \left[\frac{1}{B_o} \right] \end{aligned} \quad \text{Eq. 10.8}$$

The terms in this equation that are associated with the various production mechanisms are:

$$\begin{aligned} \text{Rock Compaction} = \text{DC} &= -d(\text{PV}) \cdot S_o^t \cdot \left[\frac{1}{B_o} \right]^{t+dt} \\ \text{Saturation Change} = \text{DS} &= -\text{PV}^{t+dt} \cdot d(S_o) \cdot \left[\frac{1}{B_o} \right]^t \\ \text{Oil Expansion} = \text{DE} &= -\text{PV}^t \cdot S_o^{t+dt} \cdot d \left[\frac{1}{B_o} \right] - d(\text{PV}) \cdot d(S_o) \cdot d \left[\frac{1}{B_o} \right] \end{aligned} \quad \text{Eq. 10.9}$$

Now, $d(S_o) = -d(S_w) - d(S_g)$, giving the final breakdown of the production:

$$\begin{aligned} \text{Rock Compaction} = \text{DC} &= -d(\text{PV}) \cdot S_o^t \cdot \left[\frac{1}{B_o} \right]^{t+dt} \\ \text{Water Influx} = \text{DW} &= \text{PV}^{t+dt} \cdot d(S_w) \cdot \left[\frac{1}{B_o} \right]^t \\ \text{Gas Influx} = \text{DG} &= \text{PV}^{t+dt} \cdot d(S_g) \cdot \left[\frac{1}{B_o} \right]^t \\ \text{Oil Expansion} = \text{DE} &= -\text{PV}^t \cdot S_o^{t+dt} \cdot d \left[\frac{1}{B_o} \right] - d(\text{PV}) \cdot d(S_o) \cdot d \left[\frac{1}{B_o} \right] \end{aligned} \quad \text{Eq. 10.10}$$

When the pressure falls below the bubble point the oil expansion term will usually be negative, as the oil production will then be by solution gas drive. In this case the oil expansion term (DE) is added to the gas influx term (DG):

If gas saturation is greater than zero then:

$$DG = DG + DE$$

$$DE = 0.0$$

Eq. 10.11

Formulation of the equations

x	Eclipse 100
x	Eclipse 300

Eclipse contains options for the fully implicit, IMPES (IMplicit Pressure EXplicit Saturation) and AIM (Adaptive IMplicit Method) solution procedures. The fully implicit method is totally stable and may be used to solve ‘difficult’ problems such as coning studies. Although IMPES is potentially unstable, it is less dispersive and sometimes faster than the fully implicit method, and may be used on ‘easy’ problems such as history matching applications where the timesteps are usually small. The AIM method for compositional studies is a compromise between fully implicit and IMPES methods, allowing cells in ‘difficult’ regions to remain fully implicit while employing the advantage of an IMPES description in ‘easy’ regions.

Eclipse 100

The default solution procedure is fully implicit for all black oil runs. This is robust and efficient for black oil problems, generally allowing for large timesteps in the simulation. Both fully implicit and IMPES may be used in the same run by using keywords `DIMPES` or `DIMPLICIT` at appropriate places in the `SCHEDULE` section.

Eclipse 300

The default solution procedure is `AIM` for all compositional runs and `FULLIMP` for all black oil or thermal runs. The default solution procedure may be changed by using one of the keywords `AIM`, `FULLIMP` or `IMPES` in the `RUNSPEC` section.

Description of equations

The non-linear residual, R_{fl} , for each fluid component in each grid block at each timestep is:

$$R_{fl} = \frac{\delta M}{\delta t} + F + Q \quad \text{Eq. 10.12}$$

where

dM is the mass, per unit surface density, accumulated during the current timestep, $d t$

F is the net flow rate into neighboring grid blocks

Q is the net flow rate into wells during the timestep

R_{fl} is defined for each cell and each fluid in the study.

In the black oil case the fluids are oil, water and gas; in the compositional case they are the hydrocarbon components and water.

Eclipse 300

In the compositional case, for two hydrocarbon phase cells, there is in addition the requirement that the Gibbs energy of the system is at a minimum. The first order conditions for this yield a component equal fugacity condition:

$$R_{fu} = f_l - f_v = 0 \quad \text{Eq. 10.13}$$

where f_l and f_v are the component fugacities in the liquid and vapor phases respectively.

To solve the entire system, the simulator requires that $R_{fl} \rightarrow 0$ and $R_{fu} \rightarrow 0$ to within a required tolerance.

Variable sets

Solving the residual equations requires a set of solution variables. The number of independent variables must be equal to the number of residual conditions; the residual equations may then be solved using Newton's method. The actual variable set chosen differs between Eclipse 100 and Eclipse 300, so each is described separately.

Eclipse 100

The primary solution variables X are pressure P and two saturations for a three phase black oil study. The water saturation S_w and either S_g , R_s or R_v are chosen to complete the set. For a 3 component black oil system (oil, water, gas), the residual R and the solution, X , are 3 component vectors in each grid block. By default, the solution procedure is fully implicit;

$$R = \begin{bmatrix} R_o \\ R_w \\ R_g \end{bmatrix} \quad X = \begin{bmatrix} P_o \\ S_w \\ S_g \text{ or } R_s \text{ or } R_v \end{bmatrix} \quad \text{Eq. 10.14}$$

and the Jacobian, $J = \frac{dR}{dX}$, takes the form

$$\frac{dR_i}{dX_j} = \begin{bmatrix} \frac{dR_o}{dP_o} & \frac{dR_o}{dS_w} & \frac{dR_o}{dS_g} \\ \frac{dR_w}{dP_o} & \frac{dR_w}{dS_w} & \frac{dR_w}{dS_g} \\ \frac{dR_g}{dP_o} & \frac{dR_g}{dS_w} & \frac{dR_g}{dS_g} \end{bmatrix}_{ij} \quad \text{Eq. 10.15}$$

The mass change during the timestep, dt , is then proportional to

$$dM = M_{t+dt} - M_t \quad \text{Eq. 10.16}$$

with

$$M = PV \begin{bmatrix} \frac{S_o}{B_o} + \frac{R_v S_g}{B_g} \\ \frac{S_w}{B_w} \\ \frac{S_g}{B_g} + \frac{R_s S_o}{B_o} \end{bmatrix} \quad \text{Eq. 10.17}$$

where

PV is the pore volume

B_o is the oil formation volume factor

B_w is the water formation volume factor

B_g is the gas formation volume factor

R_s is the solution gas/oil ratio

R_v is the vapor oil/gas ratio.

When S_g is zero the solution variable becomes R_s (undersaturated oil) and when S_o is zero the solution variable becomes R_v (undersaturated gas). Terms in the Jacobian are adjusted in accordance with the change of variable. No approximations are made in evaluating the Jacobian in Eclipse 100. Great care is taken to compute all derivatives to ensure quadratic convergence of Newton's method.

Hydrocarbon states are:

1. State 1: Gas only $R_s = 0$, $S_g = 1 - S_w$,
variables are P_o , S_w , R_v .
2. State 2: Gas & oil, $R_v = R_{v \text{ sat}}$, $R_s = R_{s \text{ sat}}$,
variables are P_o , S_w , S_g
3. State 3: Oil only, $R_v = 0$, $S_g = 0$,
variables are P_o , S_w , R_s .

The number of cells in each state, together with the number of cells changing state during a Newton iteration (NTRAN), are printed in the summary of each Newton iteration.

Note: Cells almost entirely full of water are in state 2 by default. Therefore the solution variable R_s is set to $R_{s \text{ sat}}$. A transition from state 2 to state 3 may occur in such blocks, for example due to the influx of undersaturated oil. The cell's R_s value is then limited to not exceed the average GOR of inflows from neighboring cells as this will probably be a better estimate than $R_{s \text{ sat}}$.

Eclipse 300

The primary solution variables, used for cells in all phase states, are $X = (P, m_1, \dots, m_{N_c}, m_w)$

where

$m_1, \dots, m_{N_c}$ are molar densities of each component, and

m_w is the molar density of water.

In **IMPSTAT** mode there are three extra primary solution variables: X_o, X_g, X_w , which represent the phase saturations.

To normalize the residual equations, an additional residual condition is needed, which is usually referred to as the volume balance constraint. This condition requires that the saturations should all add up to unity and is written as

$$R_m = PV - FV = 0 \quad \text{Eq. 10.18}$$

where PV is the cell pore volume, and FV is the fluid volume, given by

$$FV = PV \left(\frac{m_t L}{b_o^m} + \frac{m_t V}{b_g^m} + \frac{m_w}{b_w^m} \right) \quad \text{Eq. 10.19}$$

with the total hydrocarbon molar density given by

$$m_t = \sum_c m_c \quad \text{Eq. 10.20}$$

where

L and V are the liquid and vapor mole fractions, and

b_o^m , b_g^m and b_w^m are the molar densities of oil, gas and water.

The form of the Jacobian matrix condition in the compositional case is then

$$J = \frac{dR_{fl}}{dX}$$

which can be depicted in terms of the primary solution variables (P , m_c for each hydrocarbon component c), m_w and residual equations (R_c , R_w and R_m) for the fully implicit method as

$$J = \begin{matrix} & P & m_1 & \dots & m_{N_c} & m_w \\ \begin{matrix} Comp1 \\ \dots \\ CompN_c \\ Water \\ Volumebal \end{matrix} & \begin{bmatrix} x & x & \dots & x & x \\ x & x & \dots & x & x \\ x & x & \dots & x & x \\ x & x & \dots & x & x \\ x & x & \dots & x & x \end{bmatrix} \end{matrix} \quad \text{Eq. 10.21}$$

The residual equation $R(X) = 0$ is solved for X using Newton's method.

In two phase blocks further secondary solution variables are required. These are the $J_i = \log(K_i)$ values for each component, and the vapor mole fraction V . In terms of these variables the equal fugacity residual condition may be expressed as:

$$R_{fu} = J - H_l + H_v \quad \text{Eq. 10.22}$$

where

$$H_l = \ln\left(\frac{f_l}{Px}\right) \quad \text{Eq. 10.23}$$

$$H_v = \ln\left(\frac{f_v}{Py}\right) \quad \text{Eq. 10.24}$$

An additional normalization residual (the Rachford-Rice equation) exists in the compositional case,

$$R_n = \sum_i \left(z_i \frac{(K_i - 1)}{1 + V(K_i - 1)} \right) = 0 \quad \text{Eq. 10.25}$$

where z_i and K_i are component mole fractions and K-values respectively. The solution variables for the flash calculation are $X_f = (J_1, \dots, J_N, V)$.

The flash solution X_f is found at each iteration by solving the equilibrium condition that

$$R_{fu}(X_f) = 0 \quad \text{Eq. 10.26}$$

In **IMPSAT** mode there are three additional residuals to define the saturation variables X_o , X_g , X_w :

$$R_p = PV(X - S_p(P, m)) = 0 \quad \text{Eq. 10.27}$$

where $S_p(P, m)$ is the fractional volume occupied by phase p .

The fully implicit method

The fully implicit method is the default for black oil runs in both Eclipse 100 and Eclipse 300. It is not recommended for compositional runs where there are often too many components to be able to use the fully implicit method efficiently.

Newton's method is used to solve the non-linear residual equations $R(X) = 0$. Several iterations may be required to reduce the residuals to a 'sufficiently small' value. In Eclipse 100, two measures are used to define the meaning of 'sufficiently small'. These are the material balance error and the maximum saturation normalized residual.

The IMPES and AIM methods

The IMPES (Implicit Pressures Explicit Saturations) residual is similar to the fully implicit residual except that all flow and well terms are computed using saturations (or R_s , R_v) in a black oil run; or molar densities in a compositional run at the beginning of each timestep.

$$R = \frac{M_{t+dt} - M_t}{dt} + F(P_{t+dt}, S_t) + Q(P_{t+dt}, S_t) \quad \text{Eq. 10.28}$$

The mass terms M_{t+dt} are evaluated using both pressures and saturations at the end of the timestep. This makes the non-linear residual equation, $R = 0$ much easier to solve because there are now no non-linearities arising from relative permeabilities that remain fixed throughout the timestep. However, to solve the IMPES equations correctly it is still necessary to iterate until all residuals have been reduced to a sufficiently small value.

The linear equations arising from Newton's method are also much easier to solve in the IMPES case because derivatives of flows with respect to saturations are zero. The linear equations are solved sequentially, first for pressure and subsequently for saturation changes. This contrasts with the fully implicit method where the linear equations must be solved simultaneously.

Eclipse 100

Timesteps are chosen automatically to limit saturation changes to 5%. However a converged timestep is accepted if the largest saturation change is less than 10% and if the largest pressure change is less than 200 psi. These values may be altered using keyword [DIMPES](#).

Eclipse 300

Timesteps are chosen automatically to limit saturation changes to 5%. The default timestep selection may be modified using the [TSCRIT](#) keyword.

Eclipse 300

In Eclipse 300, the IMPES formulation is strictly an IMPEM (IMplicit Pressure Explicit Mobility) method. The mass terms M_{t+dt} are evaluated using both pressures and molar densities at the end of the timestep. The flow terms between cells are evaluated assuming the saturations, generalized mobilities and reservoir density terms are all fixed at the previous timestep.

AIM

Eclipse 300

The Adaptive IMplicit method ([AIM](#)) is a compromise between the fully implicit and IMPES procedures. Cells with a high throughput ratio are chosen to be implicit for stability and to obtain large timesteps, while the majority of cells can still be treated as IMPES where the solution may be changing little. All completions are treated implicitly. The target fraction of implicit cells in a compositional run is 1%, but this can be altered by the [AIMFRAC](#) keyword.

The CFLLIMIT option for AIM and IMPES runs

This option is based on the paper “IMPES Stability: The Stable Step” [Coats \(2001\)](#). The reader is referred there for all technical points. By default, AIM and IMPES runs choose the implicit cells/timestep by looking at throughput and/or solution change. Whilst this usually works it is not a rigorous procedure, and in some situations, such as when there are large capillary forces, it can make the wrong choice.

The [CFLLIMIT](#) option places the timestep selection and implicit cell choice on a much more rigorous footing based on the maximum **stable** timestep allowed in the **explicit** cells. This timestep can be calculated from conditions given in the above paper. Due to the robustness of this approach when it is used there is no limit placed on the maximum rate of change of capillary pressure. See keyword [DPCDT](#) and compare with the usual default for AIM or IMPES runs.

For those runs where this option does make a difference: aside from differences caused by strictly honoring the rate of change of capillary pressure, you may notice the following:

IMPES runs

The run takes longer due to smaller, in some cases substantially smaller, timesteps being taken. The advantage is that oscillations in the results virtually disappear.

AIM runs

Times can actually improve as now the ‘right’ cells have been chosen to be implicit, which can lead to fewer convergence problems. In some cases, because the correct cells are being treated implicitly and the timestep has been chosen to be stable for the remaining explicit cells, a significant improvement in performance and results are experienced.

There are also cases where performance is severely degraded, although results are not. This occurs when the case can run, with no obvious ill-effects, with timesteps very much larger than those required by stability considerations. There is usually some benefit to be seen in a close examination of the results, but if these benefits do not justify a large increase in CPU time, or indeed if the increase makes running the problem uneconomic, then there are two courses of action.

- items 2 to 4 of the [CFLLIMIT](#) keyword can be altered to allow timesteps that are theoretically unstable
- not to use this keyword, but to use the default [AIM](#) settings, and perhaps use [TUNING](#) to prevent the default AIM settings from selecting some of the larger timesteps.

Instability in IMPES

The most common instability using IMPES arises when a phase crosses the critical saturation and becomes mobile. In figure [10.1](#) water is flowing from cell A to cell B. Water levels at time t are shown by the dark shading. Because the saturation in cell B is below S_{wcr} at time t , water cannot flow out of cell B. During the timestep, dt , the water saturation in cell B can rise significantly above S_{wcr} to a value greater than that in cell A. At the next timestep, water is allowed to flow out of cell B at a high rate especially if k_{rw} increases sharply above S_{wcr} , and the saturation may fall below S_{wcr} making water in cell B immobile again at the following timestep. Saturations may therefore oscillate about their critical values using IMPES. These difficulties do not arise with the fully-implicit method because as soon as S_w is greater than S_{wcr} it is permitted to flow out of cell B.

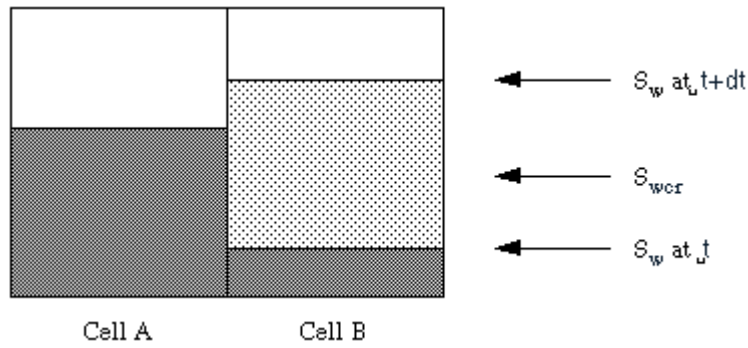


Figure 10.1. The most common instability using IMPES

This type of instability may limit the efficiency of the IMPES method. The problem is reduced in some simulators (but not in Eclipse) by automatically rescaling the input relative permeability curves to equally spaced saturation intervals. This is not done in Eclipse because:

- you may wish the data to be honored precisely
- sharp relative permeability contrasts are handled efficiently by the fully implicit method
- table look-up is equally efficient either way (contrary to popular belief)
- you have the option of inputting 'smooth' k_r curves

Convergence criteria for residual equations

The Newton solver aims to reduce the residuals $R(X)$ to zero. In Eclipse 100 the convergence criteria are primarily based on material balance and maximum residual checks on R . In Eclipse 300, the convergence criteria are primarily based on the solution change Δx being 'sufficiently small' in the Newton iteration.

Eclipse 100

Material balance

If the residuals are summed over all cells in the reservoir, the flow terms F cancel, because the flow out of one cell is always equal and opposite in sign to the corresponding flow into its neighboring cell. Thus the sum of the residuals for each phase or component corresponds to the net mass accumulation within the reservoir less the net influx through wells. This is the material balance error. For a three-component system:

$$\begin{aligned}\sum_i (R_o)_i &= \sum_i \left(\frac{dM_o}{dt} \right)_i + \sum_i (Q_o)_i \\ \sum_i (R_w)_i &= \sum_i \left(\frac{dM_w}{dt} \right)_i + \sum_i (Q_w)_i \\ \sum_i (R_g)_i &= \sum_i \left(\frac{dM_g}{dt} \right)_i + \sum_i (Q_g)_i\end{aligned}\tag{Eq. 10.29}$$

where

$\sum_i$ refers to the sum over all reservoir cells and

$(R_o)_i$ is the oil residual in cell i

In Eclipse 100 the material balance errors are converted to meaningful, problem independent, numbers by scaling to equivalent field saturation values:

$$\begin{aligned} MB_o &= \overline{B_o} dt \left\{ \left(\sum_i (R_o)_i \right) / \left(\sum_i (PV)_i \right) \right\} \\ MB_w &= \overline{B_w} dt \left\{ \left(\sum_i (R_w)_i \right) / \left(\sum_i (PV)_i \right) \right\} \\ MB_g &= \overline{B_g} dt \left\{ \left(\sum_i (R_g)_i \right) / \left(\sum_i (PV)_i \right) \right\} \end{aligned} \quad \text{Eq. 10.30}$$

where $\overline{B_o}$ is the average oil formation factor etc.

The numerical values of MB_o , MB_w and MB_g are computed after each Newton iteration and the material balance errors are considered to be sufficiently small if they are all less than $1.0E-7$. MB values are printed out in the summary of each Newton iteration. Conventional material balance accounts in conventional units can also be printed at each report time.

Normalized residuals

Although great care is taken to ensure that material balance errors are unusually small in Eclipse 100, this is not considered to be a sufficiently rigorous test of convergence. The second test is obtained by computing the maximum saturation normalized residuals

$$\begin{aligned} CNV_o &= \overline{B_o} dt \cdot \text{MAX}_i \left| \frac{(R_o)_i}{(PV)_i} \right| \\ CNV_w &= \overline{B_w} dt \cdot \text{MAX}_i \left| \frac{(R_w)_i}{(PV)_i} \right| \\ CNV_g &= \overline{B_g} dt \cdot \text{MAX}_i \left| \frac{(R_g)_i}{(PV)_i} \right| \end{aligned} \quad \text{Eq. 10.31}$$

where MAX_i is the maximum value over all cells in the reservoir.

Converting each convergence error to an equivalent saturation value attaches sensible limits to the CNV numbers, which are considered to have converged if they are all less than 0.001.

The convergence criteria can be altered by the [TUNING](#) keyword. However, the default values are most often sufficient and it is **not** recommended that they should be changed. Loosening the convergence tolerances in a difficult run often results in increased run times.

Eclipse 300

The convergence criteria in Eclipse 300 are based on solution change targets. Since the primary solution variables X are molar densities, it is natural to use these to control the convergence. However, large saturation changes may occur for very small changes in composition, so the use of an effective saturation is preferred. Consider the expression:

$$\Delta S' = \frac{du^T}{dm_c} \cdot \Delta m_c \quad \text{Eq. 10.32}$$

where

$$u^T = \frac{m_t L}{b_o^m} + \frac{m_t V}{b_g^m} + \frac{m_w}{b_w^m} \quad \text{Eq. 10.33}$$

u^T is the fluid volume per unit pore volume

$\Delta S'$ is thus the change in fluid volume per unit pore volume for a given molar density change, that is the effective saturation change for the molar density change.

In all cases the maximum solution change over a timestep is monitored and compared with the specified target.

Convergence is based on the pressure and saturation changes Δp and $\Delta S'$ being 'sufficiently small'. These targets `SCONVP` and `SCONVS` in the `CVCRIT` keyword can be used to tighten down the convergence.

Flows

The flow rates between neighboring cells are computed for each simulator as described below.

Eclipse 100

The flow rate into cell i from a neighboring cell n , F_{ni} , is

$$F_{ni} = T_{ni} \begin{bmatrix} \frac{k_{ro}}{B_o \mu_o} & 0 & \frac{R_v k_{rg}}{B_g \mu_g} \\ 0 & \frac{k_{rw}}{B_w \mu_w} & 0 \\ \frac{R_s k_{ro}}{B_o \mu_o} & 0 & \frac{k_{rg}}{B_g \mu_g} \end{bmatrix} \times \begin{bmatrix} dP_{oni} \\ dP_{wni} \\ dP_{gni} \end{bmatrix} \quad \text{Eq. 10.34}$$

where

$$dP_{oni} = P_{on} - P_{oi} - \rho_{oni} g(D_n - D_i)$$

$$\begin{aligned} dP_{wni} &= P_{wn} - P_{wi} - \rho_{wni} g(D_n - D_i) \\ &= P_{on} - P_{oi} - \rho_{wni} g(D_n - D_i) - P_{cown} + P_{cowi} \end{aligned}$$

$$\begin{aligned} dP_{gni} &= P_{gn} - P_{gi} - \rho_{gni} g(D_n - D_i) \\ &= P_{on} - P_{oi} - \rho_{gni} g(D_n - D_i) + P_{cogn} - P_{cogi} \end{aligned}$$

T_{ni} is the transmissibility between cells n and i ,

k_r is the relative permeability (k_{ro} is the relative permeability of oil etc.),

μ is the viscosity (μ_w is the viscosity of water etc.),

dP is the potential difference (dP_{gni} is the gas potential difference between cells n and i),

ρ is the fluid density (ρ_{oni} is the density of oil at the interface between cells n and i),

g is the acceleration due to gravity
(0.0000981 in metric units, 0.00694 in field units and 0.000968 in lab units),

D is the cell center depth.

The subscript u indicates that the fluid mobilities are to be evaluated in the upstream cell (cell n if dP_{ni} is positive, cell i if dP_{ni} is negative). The upstream calculation applies separately for each equation (oil, water, gas) so that, for example, oil may flow from cell i to cell n while water flows from cell n to cell i .

The net flow rate from cell i into neighboring cells is obtained by summing over the neighboring cells,
 $F_i = \sum_n F_{ni}$.

The rate of flow into a production well from cell i is

$$Q_i = -T_{wi}(P_{oi} - H_{iw} - P_{bh}) \left[\begin{array}{c} \frac{k_{ro}}{B_o \mu_o} + \frac{R_v k_{rg}}{B_g \mu_g} \\ \frac{k_{rw}}{B_w \mu_w} \\ \frac{k_{rg}}{B_g \mu_g} + \frac{R_s k_{ro}}{B_o \mu_o} \end{array} \right]_i \quad \text{Eq. 10.35}$$

where

T_{wi} is the well connection transmissibility factor

H is the hydrostatic head correction

P_{bh} is the bottom hole pressure

Well terms are discussed in detail in a separate chapter: see "[Well inflow performance](#)".

Eclipse 300

The flow rate of component c embedded in a phase p ($p=o,w,g$) into cell i from a neighboring cell n , F_{pni}^c , is

$$F_{pni}^c = T_{ni} M_p^c dP_{pni} \quad \text{Eq. 10.36}$$

where

M_p^c is the generalized mobility of component c in phase p and is given by

$$M_p^c = x_p^c k_{rp} (S_p) \frac{b_p^m}{\mu_p} \quad \text{Eq. 10.37}$$

where

x_p^c is the mole fraction of component c in phase p

k_{rp} is the relative permeability of phase p

S_p is the saturation of phase p

b_p^m is the molar density of phase p

μ_p is the viscosity of phase p

dP_{pni} is the potential difference of phase p between cells n and i , given by

$$dP_{pni} = P_{pn} - P_{pi} - \rho_{pni}g(D_n - D_i) \quad \text{Eq. 10.38}$$

or

$$dP_{pni} = P_n - P_i + P_{cpn} - P_{cpi} - \rho_{pni}g(D_n - D_i) \quad \text{Eq. 10.39}$$

where

P_{cp} is the capillary pressure for phase p

ρ_p is the mass density of phase p

g is the acceleration due to gravity

(0.0000981 in metric units, 0.00694 in field units and 0.000968 in lab units)

D is the cell center depth and

T_{ni} is the transmissibility between cells n and i .

The fluid mobilities M_p^c are evaluated in the upstream cell for each phase p separately (cell n if dP_{pni} is positive, cell i if dP_{pni} is negative). The upstream calculation applies separately for each phase (oil, water, gas) so that, for example, oil may flow from cell i to cell n while water flows from cell n to cell i .

The net flow rate of component c from cell i into neighboring cells is obtained by summing over all phases over the neighboring cells, $F_i^c = \sum_n F_{pni}^c$.

For a component c existing in both oil and gas phases with liquid and vapor mole fractions x_c and y_c , the component mobilities in oil and gas can then be written as

$$M_o^c = x_c k_{ro}(S_w, S_g) \frac{b_o^m}{\mu_o} \quad \text{Eq. 10.40}$$

and

$$M_g^c = y_c k_{rg}(S_g) \frac{b_g^m}{\mu_g} \quad \text{Eq. 10.41}$$

The rate of flow of component c embedded in phase p into a production well from cell i is

$$Q_{pi}^c = -T_{wi}(P_{pi} - H_{iw} - P_{bh})M_{pi}^c \quad \text{Eq. 10.42}$$

where

T_{wi} is the well connection transmissibility factor

H is the hydrostatic head correction

P_{bh} is the bottom hole pressure

M_{pi}^c is the generalized mobility of component c in phase p

See "Well inflow performance" for further information.

Transport coefficients

The component mobilities can be modified in Eclipse 300 by including transport coefficients (*Barker and Fayers, 1994*). The transport coefficients α_p^c act as multipliers to the mobility. Thus equation 10.37 becomes:

$$M_p^c = \alpha_p^c x_p^c k_{rp} \frac{b_p^m}{\mu_p} \quad \text{Eq. 10.43}$$

where

$\alpha_p^c = \alpha_p^c(z^t)$ is the transport coefficient given by the **ALPHA** keyword

z^t is the mole fraction of component t defined by the **TRCOEF** keyword.

Transport coefficients for drainage and imbibition

When hysteresis is present it is possible in Eclipse 300 to modify the component mobilities depending on whether drainage (decreasing wetting phase saturation) or imbibition (increasing wetting phase saturation) is occurring. The component mobilities are modified by introducing transport coefficients which act as multipliers to the mobility. The transport coefficients for the drainage and imbibition process are specified separately and denoted by $\alpha_p^{c,d}$ and $\alpha_p^{c,i}$ respectively.

Thus equation 10.37 becomes:

$$M_p^c = \begin{cases} \alpha_p^{c,d} x_p^c k_{rp}^d \frac{b_p^m}{\mu_p} & \text{if drainage process} \\ \alpha_p^{c,i} x_p^c k_{rp}^i \frac{b_p^m}{\mu_p} & \text{if imbibition process} \end{cases}$$

where

$\alpha_p^{c,d} = \alpha_p^{c,d}(z^t)$ is the transport coefficient given by the **ALPHAD** keyword

$\alpha_p^{c,i} = \alpha_p^{c,i}(z^t)$ is the transport coefficient given by the **ALPHAI** keyword

z^t is the mole fraction of component t defined by the **TRCOEF** keyword.

Reservoir densities

Eclipse 100

In the black oil case, reservoir densities (ρ) of oil and gas are computed from surface densities (ρ_s) using

$$\begin{aligned}\rho_o &= (\rho_{so} + CR_s\rho_{sg})/B_o \\ \rho_g &= (C\rho_{sg} + R_v\rho_{so})/B_g\end{aligned}\quad \text{Eq. 10.44}$$

with $C = 1$ for metric and lab units and $C = 178.1076$ in field units.

The reservoir density at the interface between cells n and i is computed as the average of the reservoir densities in cells n and i

$$\begin{aligned}\rho_{oni} &= (\rho_{on} + \rho_{oi})/2 \\ \rho_{gni} &= (\rho_{gn} + \rho_{gi})/2\end{aligned}\quad \text{Eq. 10.45}$$

Eclipse 300

In the compositional case, the reservoir densities are obtained from

$$\rho_p = MW_p b_p^m \quad \text{Eq. 10.46}$$

where

MW_p is the average molecular weight of the phase p .

The reservoir density at the interface between cells n and i is computed as the saturation weighted average of the reservoir densities in cells n and i

$$\rho_{pni} = \frac{(\rho_{pn}S_{pn} + \rho_{pi}S_{pi})}{(S_{pn} + S_{pi})} \quad \text{Eq. 10.47}$$

Newton iteration of the non-linear residual

Given the non-linear residual $R = R_{fl}$ and the solution variables X , the requirement is to solve the non-linear equation $R(X) = 0$. In each iteration of the non-linear equations the sequence of operations is:

1. Given current solution X , obtain $R(X)$, then find a correction Δx such that

$$R(X + \Delta x) \sim R(X)J \cdot \Delta x = 0, \text{ where } J \text{ is the Jacobian } \frac{d}{dX}R(X).$$

2. To do this, solve the linear equations

$$J \cdot \Delta x = R(X) \quad \text{Eq. 10.48}$$

or

$$\Delta x = J^{-1}R(X) \quad \text{Eq. 10.49}$$

3. Update the solution vector, $X \rightarrow X + \Delta x$. If some measure of convergence has been achieved, exit; if not, repeat.

In the fully implicit case, these equations may be solved for X , using the linear solver to give

$$\Delta x = J^{-1} \cdot R'_{fl} \quad \text{Eq. 10.50}$$

In the [IMPES](#) case, the matrix J^{-1} has a special structure, in that band terms only appear for pressure derivatives.

Eclipse 300

For the compositional case in Eclipse 300, with the solution variables described above, the forms of the diagonal and band terms of J^{-1} are

$$J'_D = \begin{matrix} & P & S_w & m_1 & \dots & m_N \\ \text{Water} & \begin{bmatrix} x & x & 0 & \dots & 0 \end{bmatrix} \\ \text{Compl} & \begin{bmatrix} x & 0 & x & \dots & 0 \end{bmatrix} \\ \dots & \dots \\ \text{Comp}N_c & \begin{bmatrix} x & 0 & 0 & \dots & x \end{bmatrix} \\ \text{Volumebal} & \begin{bmatrix} x & x & x & \dots & x \end{bmatrix} \end{matrix} \quad \text{Eq. 10.51}$$

$$J'_B = \begin{matrix} & P & S_w & m_1 & \dots & m_N \\ \text{Water} & \begin{bmatrix} x & 0 & 0 & \dots & 0 \end{bmatrix} \\ \text{Compl} & \begin{bmatrix} x & 0 & 0 & \dots & 0 \end{bmatrix} \\ \dots & \dots \\ \text{Comp}N_c & \begin{bmatrix} x & 0 & 0 & \dots & 0 \end{bmatrix} \\ \text{Volumebal} & \begin{bmatrix} 0 & 0 & 0 & \dots & 0 \end{bmatrix} \end{matrix} \quad \text{Eq. 10.52}$$

Thus there is one column, the diagonal, and the bottom row in the diagonal case, and one column only in the band case.

By subtracting a multiple of each line from the last, all terms in the molar balance equations $R_c = 0$ may be zeroed except the first, $\frac{dR'_m}{dP}$. This is a banded pressure equation, which may be solved using the nested factorization linear solver (see ["Solution of the linear equations"](#)).

The remaining elements of X may then be found by back-substitution.

Eclipse 100

In the Adaptive IMplicit ([AIM](#)) method, the molar densities can be eliminated from the explicit cells as in the IMPES case. However there are also equations coupling pressure and molar density changes for all implicit cells. The resulting Jacobian can still be solved by the nested factorization linear solver. Remaining elements of X are obtained by back substitution only in explicit cells.

Eclipse 100

Similarly in the black oil case in Eclipse 100, the diagonal and band terms of J^{-1} for a three phase oil-water-gas system are

$$J'_D = \begin{matrix} & P & S_w & X_3 \\ \text{Water} & \begin{bmatrix} x & x & 0 \end{bmatrix} \\ \text{Oil} & \begin{bmatrix} x & 0 & x \end{bmatrix} \\ \text{Gas} & \begin{bmatrix} x & 0 & x \end{bmatrix} \end{matrix} \quad \text{Eq. 10.53}$$

$$J'_B = \begin{matrix} & P & S_w & X_3 \\ \text{Water} & \begin{bmatrix} x & 0 & 0 \end{bmatrix} \\ \text{Oil} & \begin{bmatrix} x & 0 & 0 \end{bmatrix} \\ \text{Gas} & \begin{bmatrix} x & 0 & 0 \end{bmatrix} \end{matrix} \quad \text{Eq. 10.54}$$

Solution of the linear equations

x	Eclipse 100
x	Eclipse 300

The linear equations in Eclipse 100 are solved iteratively using Orthomin ([Vinsome, 1976](#)). The default preconditioner for the ORTHOMIN iterations is Nested Factorization. Alternatively, if the [CPR](#) keyword is entered, the Constrained Pressure Residual (CPR) preconditioner can be used instead. At each iteration, Orthomin minimizes the sum of the squares of the residuals by orthogonalizing each new search direction to the existing search directions. To carry out these orthogonalizations, each search direction must be stored. Usually, only the last few search directions are significant and memory is saved if the number of stored search directions is restricted. This is done in Eclipse through the [NSTACK](#) keyword of the RUNSPEC data section. The method for solving the linear equations in Eclipse 300 is similar to Eclipse 100, except that the GMRES ([Saad, 1996](#)) algorithm is employed instead of ORTHOMIN. In Eclipse 300 the GMRES iterations can be preconditioned by nested factorization, which is the default, or by the CPR preconditioner.

The default [NSTACK](#) is 10 in Eclipse 100 and 40 in Eclipse 300. [NSTACK](#) should always be less than or equal to the maximum number of linear iterations ([LITMAX](#)), which may be altered using keyword [TUNING](#) (Eclipse 100) or [CVCRT](#) (Eclipse 300) in the [SCHEDULE](#) section.

Each search direction consists of pressures, saturations and residuals for each cell in an Eclipse 100 black oil run, which is six double precision numbers per cell in a three-phase black oil run. In Eclipse 300, each search direction consists of pressures for each cell and also molar densities for any implicit cells.

It should rarely be necessary to alter the default settings of [NSTACK](#) and [LITMAX](#). However, for very difficult problems, they may have to be increased to ensure that the linear equations are solved correctly. Warning messages are printed by Eclipse when the linear equations are not fully converged. The total memory required for each run is printed out by Eclipse as soon as the required data has been read in and analyzed.

The progress of the linear solver may be monitored by setting the seventh argument of the [DEBUG](#) keyword to 1 in Eclipse 100 and by setting the ninth argument of the [DEBUG3](#) keyword to 1 in Eclipse 300.

Material balance

At each Newton iteration we solve the linear equation

$$Ax = b$$

where

A is the Jacobian matrix (dR/dX), and

b is the non-linear residual of the prior Newton iteration, $b = R = (dM/dt) + F + Q$.

If the elements of b are summed over all cells in the reservoir then the flow terms, F , will cancel and the sum corresponds to the rate of mass accumulation in the reservoir. Eclipse computes an initial approximate solution, y , by solving an approximate equation

$$By = b$$

The material balance error (that is the error in the mass accumulation rate) corresponding to y is obtained by summing the elements of the residual, r

$$r = b - Ay = (B - A)y \quad \text{sum} = \sum_{i=1}^{n \text{ cells}} r_i \quad \text{Eq. 10.55}$$

This sum is zero if B is chosen such that

$$\text{colsum}(B) = \text{colsum}(A)$$

where $(\text{colsum}A)$ is the diagonal matrix formed by summing A in columns. The above colsum constraint is obeyed independently for each component by Eclipse. The **colsum** constraint is appropriate to obtain fast convergence for fully implicit black oil cases in Eclipse 100 because of the material balance constraint.

In adaptive implicit cases in Eclipse 300, it is more efficient to use a **rowsum** constraint where the preconditioner may be constructed such that

$$\text{rowsum}(B) = \text{rowsum}(A)$$

where $(\text{rowsum}A)$ is the diagonal matrix formed by summing the pressure terms in A by rows. This technique is most effective for solving the pressure equation in an Eclipse 300 AIM run. This technique is described in [Wallis \(1983\)](#).

Nested factorization preconditioning

A typical matrix for a simple three-dimensional system is shown in equation 10.56 which shows the structure of the Jacobian matrix for a simple 3*2*3 problem.

$$A = \begin{bmatrix} D & U1 & . & U2 & . & . & U3 & . & . & . & . & . & . & . & . & . & . & . \\ L1 & D & U1 & . & U2 & . & . & U3 & . & . & . & . & . & . & . & . & . & . \\ . & L1 & D & . & . & U2 & . & . & U3 & . & . & . & . & . & . & . & . & . \\ L2 & . & . & D & U1 & . & . & . & . & U3 & . & . & . & . & . & . & . & . \\ . & L2 & . & L1 & D & U1 & . & . & . & . & U3 & . & . & . & . & . & . & . \\ . & . & L2 & . & L1 & D & . & . & . & . & . & U3 & . & . & . & . & . & . \\ L3 & . & . & . & . & . & D & U1 & . & U2 & . & . & U3 & . & . & . & . & . \\ . & L3 & . & . & . & . & L1 & D & U1 & . & U2 & . & . & U3 & . & . & . & . \\ . & . & L3 & . & . & . & . & L1 & D & U1 & . & U2 & . & . & U3 & . & . & . \\ . & . & . & L3 & . & . & L2 & . & L1 & D & U1 & . & . & . & U3 & . & . & . \\ . & . & . & . & L3 & . & . & L2 & . & L1 & D & . & . & . & . & U3 & . & . \\ . & . & . & . & . & L3 & . & . & . & . & D & U1 & . & U2 & . & . & . & . \\ . & . & . & . & . & . & L3 & . & . & . & L1 & D & U1 & . & U2 & . & . & . \\ . & . & . & . & . & . & . & L3 & . & . & . & L1 & D & . & . & U2 & . & . \\ . & . & . & . & . & . & . & . & L3 & . & . & L2 & . & . & D & U1 & . & . \\ . & . & . & . & . & . & . & . & . & L3 & . & . & L2 & . & L1 & D & U1 & . \\ . & . & . & . & . & . & . & . & . & . & L3 & . & . & L2 & . & L1 & D & . \end{bmatrix} \quad \text{Eq. 10.56}$$

Eclipse chooses direction 1 automatically to correspond to the direction of highest transmissibility (usually the Z direction). Thus L1 and U1 usually correspond to derivatives of flows between cells in the Z direction. The inner solver direction may be overridden using the [SOLVDIRS](#) keyword. This may be useful, for example, if it is known in advance that the principal flow is in the X direction.

Three-dimensional finite difference systems give rise to sparse linear equations with banded matrices equation (10.56) such as

$$A = D + L1 + U1 + L2 + U2 + L3 + U3 \quad \text{Eq. 10.57}$$

where

D is a diagonal matrix

$L1$, $L2$ and $L3$ are lower bands

$U1$, $U2$ and $U3$ are upper bands

The elements of D , $L1$, $L2$, $L3$, $U1$, $U2$, $U3$ are 2×2 matrices for implicit two-phase systems (for example oil/water, oil/gas, gas/water) and 3×3 matrices for implicit three-phase systems. In the [IMPES](#) case these elements are scalars.

$L2$ and $U2$ connect cells in direction 2; $L3$ and $U3$ connect cells in direction 3. In Nested Factorization, an approximation, B , is constructed by the following nested sequence of factorizations

$$\begin{aligned} B &= (P + L3)P^{-1}(P + U3) = P + L3 + U3 + L3 \cdot P^{-1} \cdot U3 \\ P &= (T + L2)T^{-1}(T + U2) = T + L2 + U2 + L2 \cdot T^{-1} \cdot U2 \\ T &= (G + L1)G^{-1}(G + U1) = G + L1 + U1 + L1 \cdot G^{-1} \cdot U1 \end{aligned} \quad \text{Eq. 10.58}$$

where

G is a diagonal matrix and

P^{-1} , T^{-1} and G^{-1} are the inverse of PT and G respectively.

It follows that

$$\begin{aligned} B &= G + L1 + U1 + L2 + U2 + L3 + U3 + L1G^{-1}U1 + L2T^{-1}U2 + L3P^{-1}U3 \\ &= A + G - D + L1G^{-1}U1 + L2T^{-1}U2 + L3P^{-1}U3 \end{aligned} \quad \text{Eq. 10.59}$$

The diagonal matrix G is computed using

$$0 = G - D + L1G^{-1}U1 + \text{Colsum}(L2T^{-1}U2 + L3P^{-1}U3) \quad \text{Eq. 10.60}$$

This choice of G ensures that **colsum** (B) = **colsum** (A) which, in turn, ensures that there are no material balance errors in the solution of the linear equations.

Alternatively, the preconditioner may be constructed such that **rowsum** (B) = **rowsum** (A), thus summing the pressure terms in A by rows to ensure accurate solution of the pressure equation in [IMPES](#) or [AIM](#) cases. This technique is described in [Wallis \(1983\)](#).

Off band elements, corresponding to non-neighbor connections, arising in the treatment of, for example, faults, local grid refinement and completing the circle in 3D radial studies, destroy the simple structure of the bands $L1$, $U1$..., but are easily incorporated in the Nested Factorization procedure by simply generalizing the definition of the bands.

Initialization for nested factorization preconditioner

Before iteration begins we must compute the diagonal matrix G . Eclipse computes and stores G^{-1} rather than G as this is more efficient in subsequent calculations. To conserve material, using the **colsum** variant on the preconditioner we have

$$G = D - L1G^{-1}U1 - \text{colsum}(L2T^{-1}U2) - \text{colsum}(L3P^{-1}U3)$$

The calculation proceeds one cell at a time; thus when G^{-1} is known for a cell, we can calculate the contribution $L1G^{-1}U1$ to the value of G in the next cell. When G^{-1} is known on a line, we can calculate the contribution of $\text{colsum}(L2T^{-1}U2)$ to G on the next line. When G is known on a plane, we can calculate

the contribution of $\text{colsum}(L3P^{-1}U3)$ to G on the next plane. The cost of computing G (and G^{-1}) is reported under the mnemonic **GINV** in the Eclipse timing reports.

Alternatively, the **rowsum** variant on the preconditioner leads to a similar expression for G , that is:

$$G = D - L1G^{-1}U1 - \text{rowsum}(L2T^{-1}U2) - \text{rowsum}(L3P^{-1}U3)$$

Solution procedure for nested factorization

To calculate a new search direction Eclipse solves the equation $By = r$ by the following hierarchical procedure. At the outermost level we solve

$$(P + L3)(I + P^{-1}U3)y = r \quad \text{Eq. 10.61}$$

using

$$y^* = P^{-1}(r - L3y^*)$$

and

$$y^* = y - P^{-1}U3y \quad \text{Eq. 10.62}$$

Equation 10.61 is solved one plane at a time, starting with the first plane and progressing forward until y is known on each plane. The equation is explicit because $L3y$ involves the ‘known’ solution, y , on the previous plane. Similar considerations apply to the solution of equation 10.62 that is performed in reverse order starting with the last plane and sweeping backwards one plane at a time.

During the solution of equations 10.61 and 10.62 we must compute vectors of the form $z = P^{-1}q$ on each plane. This involves solving equations like

$$(T + L2)(I + T^{-1}U2)z = q \quad \text{Eq. 10.63}$$

using

$$z^* = T^{-1}(q - L2z^*)$$

and

$$z^* = z - T^{-1}U2z \quad \text{Eq. 10.64}$$

Equation 10.63 is solved in a forward sweep through the plane, one line at a time. Similarly equation 10.64 is solved by sweeping backwards one line at a time.

Finally, we note that during the solution of equations 10.63 and 10.64 we must compute vectors of the form $w = T^{-1}v$ on each plane. This involves solving the tri-diagonal equation

$$(G + L1)(I + G^{-1}U1)w = v \quad \text{Eq. 10.65}$$

using

$$w^* = G^{-1}(v - L1w^*) \quad \text{Eq. 10.66}$$

and

$$w^* = w - G^{-1}U1w \quad \text{Eq. 10.67}$$

Equations 10.66 and 10.67 are solved by sweeping first forwards, then backwards through the cells in a line. This may be solved as a block tri-diagonal in the intermediate nesting if a method exists of inverting the line tri-diagonal equations. This may be done simply, using the Thomas algorithm, and so the entire system may be solved. Computing time required to calculate a new search direction is reported under the mnemonic SRCH in the Eclipse timing reports.

Orthomin

The Orthomin procedure used by Eclipse can be summarized in the steps below:

1. Calculate the initial solution

$$y = B^{-1}b, x \rightarrow y \quad \text{Eq. 10.68}$$

2. Calculate the initial residual

$$r = b - Ay \quad \text{Eq. 10.69}$$

- If the colsum constraint is used, the sum of the elements of r is now zero
 - If the rowsum constraint is used r is identically zero for the special case $b = Ap$ where p is the vector consisting of unity in each pressure location and zero elsewhere.
3. Test for convergence and escape if converged
 4. Find a new search direction

$$y = B^{-1}b \quad \text{Eq. 10.70}$$

5. Orthogonalize y to the stack of previous search directions (z)

$$y = y - \frac{(y \cdot z)z}{(z \cdot z)} \quad \text{Eq. 10.71}$$

6. Find the optimum step length, s , which minimizes the saturation normalized sum of the squares of the residuals ($r \cdot r$):

$$s = \frac{r \cdot q}{q \cdot q} \quad \text{Eq. 10.72}$$

$$q = A \cdot y \quad \text{Eq. 10.73}$$

7. Update the solution ($x \rightarrow x + sy$), the residual $r \rightarrow r - sq$ and the stack $z = (z, y)$. If not converged, return to step 3.

Eclipse 100

The progress of the linear solver may be monitored by setting the seventh argument of the [DEBUG](#) keyword to 1. The convergence criterion for the linear solver may be modified using the [TUNING](#) keyword.

Eclipse 300

The progress of the linear solver may be monitored by setting the ninth argument of the [DEBUG3](#) keyword to 1. The convergence criterion for the linear solver may be modified using the [CVCRT](#) or [LSCRT](#) keywords.

Convergence criteria

The convergence criterion in Eclipse 100 is based on the maximum residuals for each phase. These are normalized to represent effective saturation normalized residuals so that the convergence criteria are physically meaningful in terms of saturation changes.

In the Eclipse 300 formulation, because the equations are formulated in terms of pressure and molar densities, it is more appropriate to base the convergence criterion on a reduction in root mean square residual.

The convergence criteria for the linear equations in both Eclipse 100 and Eclipse 300 are described in more detail below.

Eclipse 100 convergence criteria

Convergence of the linear solver can be monitored by setting the 7th parameter of keyword `DEBUG` greater than zero. If set to 1, a debug report is generated showing, at each iteration for each phase:

1. Saturation normalized root mean square residuals,
2. Residual sums (material balance errors) and
3. Absolute maximum saturation normalized residuals.

The Orthomin procedure is set to minimize item 1 above. Item 2 should be zero to within rounding error, while item 3 is used to test for convergence. The standard convergence criterion is that the maximum saturation normalized residual, $|r|_{\max}$, be less than TRGLCV ($= 0.0001$). The value of TRGLCV may be reset using keyword `TUNING`, but this is seldom necessary and should not be done without good reason. Convergence is also assumed if $|r|_{\max}$ is less than $\text{MIN}(0.1|b|_{\max}, 0.001r_{\text{top}}, \text{XXXLCV})$. The default setting of XXXLCV is 0.001 and this may also be reset in `TUNING`, $|b|_{\max}$ is the largest saturation normalized input residual and r_{top} is the greatest saturation normalized residual to occur at any stage of the procedure. r_{top} may be much larger than $|b|_{\max}$ because the colsum constraint makes the initial residuals quite large. Values of the step length, s , and the orthogonalizations coefficients (beta) are also printed in the debug output. The perfect step is $s = 1.0$, but this is seldom achieved. If the seventh `DEBUG` parameter is set to 2 then the sum of the absolute un-normalized residuals is printed corresponding to the input residual, b , and the final residual, $r = b - Ax$. This is the ‘residual from solution check’. If the seventh `DEBUG` parameter is set to 3 then residual maps are printed for each phase at each iteration.

Eclipse 300 convergence criteria

The convergence criterion on the linear solver is a reduction in RMS of the residual, so that convergence is achieved at the n th iteration if

$$\frac{\|R\|_2}{\|R^0\|_2} < TOL \quad \text{Eq. 10.74}$$

where

$$\|R\|_2 = \sqrt{\frac{\sum_{i=1}^N \sum_{c=1}^{N_{ci}} \left(\frac{R_{i,c}}{PV_i} \right)^2}{N}} \quad \text{Eq. 10.75}$$

with

R = linear residual

R^0 = initial linear residual

$R_{i,c}$ = residual for equation c in grid cell i

PV_i = pore volume for cell i

N = total number of active cells

N_{ci} = number of equations for cell i

TOL = RMS convergence tolerance specified in [CVCRIT](#) (default 1.0E-5).

Treatment of wells

The strongly coupled fully-implicit treatment of wells in Eclipse 100 extends the linear equations to include the well variables, w , corresponding to the bottom hole flowing pressure, and well bore flowing fractions. Thus each well, like each cell, has three variables in the three phase case. Including the well terms, the linear equations now become

$$\begin{aligned} Ax + Cw &= r \\ Rx + Dw &= u \end{aligned} \tag{Eq. 10.76}$$

where

u is the well residuals,

and if there are N grid blocks and L wells:

A is a banded $N \times N$ matrix (equation [10.56](#)),

C is a sparse $N \times L$ matrix,

R is a sparse $L \times N$ matrix,

D is a sparse $L \times L$ matrix.

Usually D is lower triangular and easily inverted, allowing the well terms to be eliminated. Thus the equation for x becomes

$$(A - CD^{-1}R)x = r - CD^{-1}u \tag{Eq. 10.77}$$

which is easily solved by preconditioned Orthomin.

Eclipse 300

In Eclipse 300, there is a single well variable w , corresponding to well bottom-hole pressure. When an implicit treatment of the gravity head for each completion is used, the well constraint equations cannot be solved as in equation [10.76](#). In the case of wells with multi-layer completions, extra coupling terms are introduced between all the cells on a given well. This is because the required well bottom hole pressure is a function of the solution values in each completion and each completion flow is a function of this well bottom-hole pressure. An additional contribution to the Jacobian is then defined:

$$J_w = \frac{dQ_w}{dX} \tag{Eq. 10.78}$$

which we include in the matrix A above. The column sum (or row sum) of J_w is included on the diagonal of the pre-conditioning matrix B . When using explicit head calculations the well equations do, however, reduce to the form of equation [10.77](#).

See [Holmes \(1983\)](#) and [Cheshire and Pollard \(1987\)](#) for further information.

Nested factorization for LGRs

The linear equations to be solved take the form

$$Ax = b \quad \text{Eq. 10.79}$$

where A is a banded matrix. The banded nature can be expressed as

$$A = D + L1 + U1 + L2 + U2 + L3 + U3 + L4 + U4 \quad \text{Eq. 10.80}$$

where

D is a diagonal matrix,

$L1, L2, L3, L4$ are lower bands, and

$U1, U2, U3, U4$ are upper bands.

The bands $L4$ and $U4$ represent couplings between sub-grids and are usually irregular.

In nested factorization, an approximate preconditioner M is constructed by the following nested sequence of factorizations:

$$M = (B + L4)B^{-1}(B + U4) = B + L4 + U4 + L4 \cdot B^{-1} \cdot U4 \quad \text{Eq. 10.81}$$

$$B = (P + L3)P^{-1}(P + U3) = P + L3 + U3 + L3 \cdot P^{-1} \cdot U3 \quad \text{Eq. 10.82}$$

$$P = (T + L2)T^{-1}(T + U2) = T + L2 + U2 + L2 \cdot T^{-1} \cdot U2 \quad \text{Eq. 10.83}$$

$$T = (G + L1)G^{-1}(G + U1) = G + L1 + U1 + L1 \cdot G^{-1} \cdot U1 \quad \text{Eq. 10.84}$$

It follows that

$$\begin{aligned} M = & G + L1 + U1 + L2 + U2 + L3 + U3 + L4 + U4 \\ & + L1G^{-1}U1 + L2T^{-1}U2 + L3P^{-1}U3 + L4M^{-1}U4 \end{aligned} \quad \text{Eq. 10.85}$$

that results in

$$M = A + G - D + L1G^{-1}U1 + L2T^{-1}U2 + L3P^{-1}U3 + L4M^{-1}U4 \quad \text{Eq. 10.86}$$

The diagonal matrix G is defined as

$$G = D - L1G^{-1}U1 - \text{rowsum}(L2T^{-1}U2 + L3P^{-1}U3 + L4P^{-1}U4) \quad \text{Eq. 10.87}$$

This choice ensures that

$$\text{rowsum}(A) = \text{rowsum}(M) \quad \text{Eq. 10.88}$$

where $\text{rowsum}(A)$ is the diagonal matrix formed by summing the pressure coefficients of A in rows.

This ensures that M^{-1} is an exact inverse on the sub-space spanned by constant pressure within each sub-grid. This technique is described in [Wallis et al \(1985\)](#).

Eclipse 100

In-place LGRs (specified using keyword `LGRLOCK`) are solved using the above technique but with the rowsum constraint for black oil models.

Eclipse 300

For local grid refinements, the factorization can be enhanced by inner iterations on specific sub-grids. This technique is particularly effective when the inner iterations involve relatively small LGRs such as radial

refinements. The sub-grid iterations are controlled by the `LSCRITS` keywords when using the WARP linear solver (see keyword `LSCRIT`).

The Eclipse 300 WARP linear solver

Eclipse 300

The WARP linear solver is the default solver for Eclipse 300. It is at present based on nested factorization only. Prior to 2002A the default solver was ECLSOL, which can be activated instead of WARP by using [item 66](#) of the `OPTIONS3` keyword. The WARP solver allows more flexibility for solutions of LGR problems than the ECLSOL solver.

Restrictions of the WARP solver

The 2003A WARP linear solver is incompatible with the following keywords:

- `MPFA`
- `NINEPOIN`
- `NONNC`

The 2002A WARP linear solver is also incompatible with

- `IMPSAT`

WARP sub-grids

The base entities that the WARP solver manipulates are sub-grids. These represent a portion of the total cells that are ordered to be a contiguous block of matrix.

- Every active cell in the grid is assigned to a sub-grid. In principle this association could be arbitrary; however, there are severe efficiency implications if there are very many sub-grids, or the sub-grids do not mainly contain cells which are only connected to other cells within the sub-grid.
- Sub-grid-to-sub-grid connections are implemented as NNC connections; the total generated is reported in the `PRT` file. Connections internal to sub-grids are, in general, stored as a mixture of band and NNC connections.
- Sub-grids may or may not be related to LGR grids. It is possible to have separate sub-grids for LGRs and the base grid. However, it is also possible to have sub-grids:
 - representing an amalgamation of portions of the base grid and portions of LGR grids (in this case the LGR planes are ordered first within any sub-grid to enhance the efficiency of this approach);
 - representing an amalgamation of portions of LGRs in a particular level of refinement.
- The form of the sub-grids is best observed by visualizing the output generated from the ‘`SUBG`’ mnemonic in the `RPTRST` keyword.
- When visualizing output from a run which has sub-grids, note that at sub-grid boundaries certain properties such as transmissibility will appear to be zero as the value is stored in an NNC connection rather than a neighbor connection.
- The preconditioner applied to each sub-grid is nested factorization (using `ROWSUM` on pressure only).
- The WARP linear solver performs nested factorization on the block matrix, where each block is a sub-grid. As with the sub-grid pre-conditioner, this block nesting preserves `ROWSUM` (on pressure only).

- When using the parallel option, sub-grids do not span parallel partitions; each sub-grid is completely contained within a processor.

WARP coloring

Each sub-grid is given a color. The WARP solver introduces extra nesting between colors. Sub-grids of the same color can be executed in parallel.

- NNCs between colors are preconditioned by extra nesting levels. If sub-grids only contain cells of a particular refinement level, then this is used by WARP to force a nesting between refinement levels. That is all colors on different levels of refinement are treated independently. Hence if two levels of refinement exist (because of the presence of LGRs), and each level has sub-grids containing two colors, then effectively four colors exist. The level of refinement is just used as an ordering tool for WARP to ensure that the highest level of refinement is the most heavily nested.
- Ideally there should be no NNC connections between sub-grids of the same color. Any NNC connections that do exist are not fully pre-conditioned, they are just added to the diagonal nesting matrix G to preserve rowsum.
- The coloring can be observed by visualizing the output from the COLR mnemonic in the `RPTRST` keyword.
- There is a limit of 4 levels of nesting between sub-grids, which correspond to a maximum of 16 colors.

WARP parallel example

The WARP solver can be used with the Parallel option. Each parallel partition is split into a number of sub-grids, which are connected through NNCs. Each sub-grid is colored (using the keyword `WCOL`), with an aim of having no NNCs connecting sub-grids of the same color. The WARP linear solver performs nested factorization on a sub-grid level. Sub-grids of the same color can be computed in parallel. The preconditioner performed on a sub-grid is nested factorization. Note that:

- The generation of NNCs between sub-grids will cause a memory overhead.
- Increasing the number of colors will increase the expense of each linear iteration, as extra levels of nesting are required to solve the block sub-grid matrix.

The creation of sub-grids and coloring is controlled by the `PARALLEL` and `WCOL` keywords. The `PARALLEL` keyword controls the division of the original problem into partitions. Figure 10.2 shows a two-color scheme resulting from a 4-processor run. Note that all the LGRs have inherited the color from the base grid. The LGR coloring scheme is controlled by the `WCOL` keyword.

Figure 10.3 shows a possible set of 8 sub-grids that may arise from these colors. In this case portions of the LGR grids have been amalgamated with the base grid, and the WARP solver has no knowledge of refinement levels. This type of amalgamation is generated if the 3rd item of the `WCOL` keyword is set to 2. By amalgamation of LGR and base sub-grids, the load balancing of the parallel run is more easily facilitated. For a parallel run all colors must have equal work on all processors for a perfect load balance. The more colors that are introduced the more difficult this becomes. This amalgamation strategy ensures no extra levels of nesting are introduced by adding refinement levels. However, for cases where the nested factorization is required between refinement levels this strategy may prove to be far from ideal.

Figure 10.3 shows a different sub-grid configuration for the same coloring as in figure 10.2. Here the LGRs are not amalgamated with the base grid; instead, all LGRs of the same refinement level and the same color are amalgamated together. Hence because the base grid has 8 sub-grids, and there is one level of LGR refinement spread fairly evenly over the grid, 16 sub-grids have been generated. Here WARP has knowledge of the refinement and will utilize this by adding nesting between refinement levels. This sub-grid configuration was generated by setting the 3rd item of `WCOL` to 1.

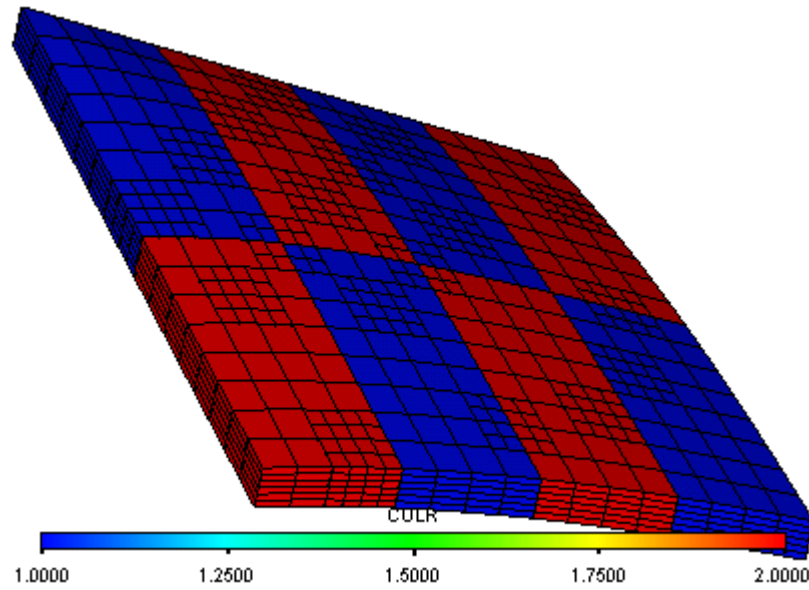


Figure 10.2. Two color (red-blue) split suitable for a 4-processors parallel run

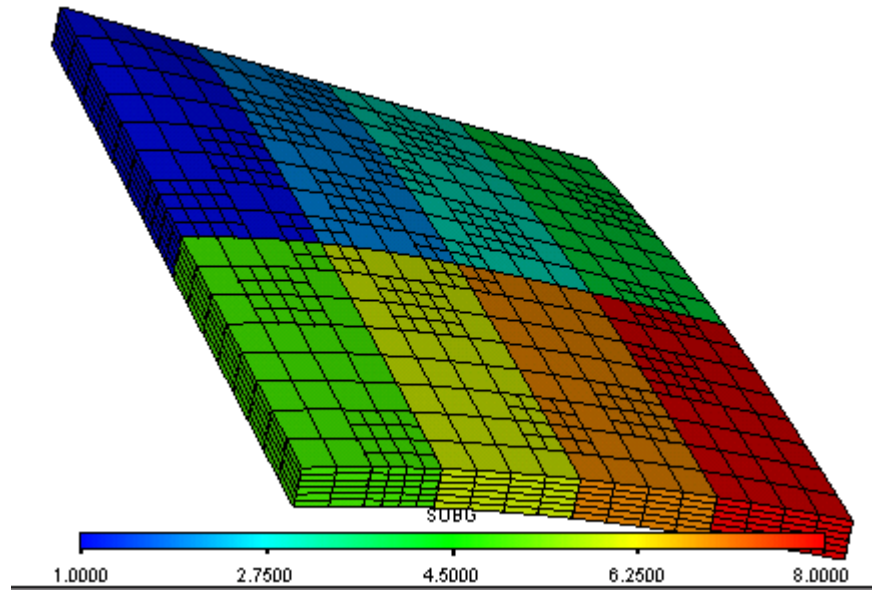


Figure 10.3. The sub-grids from the previous figure with amalgamation of LGRs and base grid

WARP solution procedure for sub-grids

Firstly consider the two color case, where S_α and S_β represents all the sub-grids in colors α and β respectively, $L_{\alpha\beta}$ and $U_{\alpha\beta}$ are the lower and upper color-to-color connective terms (these are implemented using NNC connections). Let us assume that nested factorization can be performed on each sub-grid, and hence each color, as outlined in the previous sections. The matrix system A can be decomposed into a 2×2 block color matrix thus

$$A = \begin{bmatrix} 0 & 0 \\ L_{\alpha\beta} & 0 \end{bmatrix} + \begin{bmatrix} S_\alpha & 0 \\ 0 & S_\beta \end{bmatrix} + \begin{bmatrix} 0 & U_{\alpha\beta} \\ 0 & 0 \end{bmatrix} \quad \text{Eq. 10.89}$$

The procedure is to perform nested factorization on this block system, that is the pre-conditioner is

$$\widetilde{M} = \begin{bmatrix} Q_\alpha & 0 \\ L_{\alpha\beta} & Q_\beta \end{bmatrix} \begin{bmatrix} Q_\alpha & 0 \\ 0 & Q_\beta \end{bmatrix}^{-1} \begin{bmatrix} Q_\alpha & U_{\alpha\beta} \\ 0 & Q_\beta \end{bmatrix} \quad \text{Eq. 10.90}$$

thus

$$\widetilde{M} = \begin{bmatrix} Q_\alpha & U_{\alpha\beta} \\ L_{\alpha\beta} & (L_{\alpha\beta} Q_\beta^{-1} + Q_\beta) \end{bmatrix} \quad \text{Eq. 10.91}$$

The matrix Q is chosen to preserve rowsum (of pressure only) so

$$Q_\alpha = G_\alpha \quad \text{Eq. 10.92}$$

and

$$Q_\beta = G_\beta - \text{rowsum}(L_{\alpha\beta} Q_\beta^{-1} U_{\alpha\beta}) \quad \text{Eq. 10.93}$$

where G_α and G_β are the diagonal nested factorization matrices constructed from all the sub-grids associated with colors α and β respectively.

During this procedure Q^{-1} needs to be evaluated. If individual tolerances are imposed on sub-grids (through the `LSCRITS` keyword), then this inversion is approximated by multiple iterations of GMRES, rather than just one nested factorization inversion.

For multiple colors the above procedure is applied recursively. For example if four colors $\alpha, \beta, \gamma, \delta$ are present, the matrix A could be written

$$A = \begin{pmatrix} () \\ () \\ () \\ () \end{pmatrix} \quad \text{Eq. 10.94}$$

and then nested factorization can be performed on the 2 x 2 system, where the blocks are groups of colors. Up to four levels of nesting are currently available, which corresponds to a maximum of 16 colors.

Generalized minimum residual method (GMRES)

The preconditioned GMRES method is mathematically equivalent to the Orthomin method. However, it has better numerical properties and involves less computational work, so it is generally preferred over Orthomin and is used for the WARP solver. The GMRES method for solving $Ax = b$ can be summarized as follows:

Initialization

Set initial residual

$$r_0 = b, \beta = \|r_0\|_2, v_1 = r_0/\beta \quad \text{Eq. 10.95}$$

Iteration

For $k = 1, 2, \dots$ until convergence, do

Search direction

$$u_k = M^{-1}v_k \quad \text{Eq. 10.96}$$

Matrix multiply:

$$w = Au_k \quad \text{Eq. 10.97}$$

Orthogonalize:

$$h_{i,k} = (w \cdot v_i)$$

$$w = w - \sum_{i=1}^k h_{i,k} v_i \quad \text{Eq. 10.98}$$

Normalize:

$$h_{k+1,k} = \|w\|_2$$

$$v_{k+1,k} = \frac{w}{h_{k+1,k}} \quad \text{Eq. 10.99}$$

Solution:

$$x_k = \sum_{i=1}^k \alpha_i u_i \quad \text{Eq. 10.100}$$

where the α_i are chosen to minimize the residual $\|b - Ax_k\|_2$

For complete implementation details see [Saad \(1996\)](#).

The Constrained Pressure Residual (CPR) Preconditioner

The Constrained Pressure Residual (CPR) preconditioner ([Wallis et al, 1985](#)) is employed by the [CPR](#) linear solver and the [JALS](#) linear solver.

The reservoir equations encountered in reservoir simulation are of mixed character. The pressure equations are often elliptic dominated with long range coupling, whilst the saturation equations can exhibit hyperbolic behavior, with steep local gradients. The CPR preconditioner is designed to tackle the linear equations that arise from the mixed character of the reservoir equations. This section outlines the Eclipse implementation, which is used by the CPR linear solver.

The CPR preconditioner consists of a two stage approximation to the inverse of the Jacobian matrix A . The equations can be arranged so that all of the pressure equations appear first, followed by the saturation equations (or molar density equations, for compositional). The Jacobian matrix can then be decomposed into the pressure terms A_{pp} , the saturation terms A_{ss} , and the pressure-saturation coupling terms, A_{ps} and A_{sp} . Similarly, the solution vector, x , and the residual vector, R , are also decomposed into the pressure terms and the saturation terms. This results in the following system of linear equations:

$$\begin{bmatrix} A_{pp} & A_{ps} \\ A_{sp} & A_{ss} \end{bmatrix} \begin{bmatrix} x_p \\ x_s \end{bmatrix} = \begin{bmatrix} R_p \\ R_s \end{bmatrix} \quad \text{Eq. 10.101}$$

The full system contains n_{cell} cells, with n_{eqn} equations per cell. The pressure system consists of just n_{cell} equations. An $(n_{\text{cell}} \cdot n_{\text{eqn}})$ by n_{cell} matrix C can be defined, together with its transpose C^T , to transform between the full system and the pressure system:

$$C = \begin{bmatrix} I_p \\ 0 \end{bmatrix} \quad \text{Eq. 10.102}$$

Where I_p is the n_{cell} by n_{cell} identity matrix, with the same dimensions as the pressure system. To facilitate the first stage of the preconditioner, the full system is restricted to a smaller pressure system:

$$A_p = C^T G A C \quad \text{Eq. 10.103}$$

$$r_p = C^T G R \quad \text{Eq. 10.104}$$

The additional transformation matrix, G , depends on the chosen method of solving the pressure system. In this case, G is constructed to approximately decouple the pressure system from the full system:

$$G = \begin{bmatrix} I_p - A_{ps} \hat{A}_{ss}^{-1} \\ 0 & I_s \end{bmatrix} \quad \text{Eq. 10.105}$$

The matrix $\hat{A}_{ss}$ is a simple approximation to the saturation matrix. Note that if $\hat{A}_{ss}$ was chosen as the true inverse of A_{ss} , the pressure equations would be fully decoupled, since the matrix A would have a zero block in the position of $\hat{A}_{ps}$. However, it is not efficient to do this in general as the inverse of A_{ss} is generally dense. For this reason, we replace it with a simple approximation, such as the diagonal matrix $\text{Diag}(A_{ss})$. Therefore:

$$A_p = [I_p \ 0] \begin{bmatrix} I_p - A_{ps} \hat{A}_{ss}^{-1} \\ 0 & I_s \end{bmatrix} \begin{bmatrix} A_{pp} & A_{ps} \\ A_{sp} & A_{ss} \end{bmatrix} \begin{bmatrix} I_p \\ 0 \end{bmatrix} = A_{pp} - A_{ps} \hat{A}_{ss}^{-1} A_{sp} \quad \text{Eq. 10.106}$$

$$r_p = [I_p \ 0] \begin{bmatrix} I_p - A_{ps} \hat{A}_{ss}^{-1} \\ 0 & I_s \end{bmatrix} \begin{bmatrix} R_p \\ R_s \end{bmatrix} = R_p - A_{ps} \hat{A}_{ss}^{-1} R_s \quad \text{Eq. 10.107}$$

Eclipse employs the CPR preconditioner within the Flexible GMRES (FGMRES) algorithm. The reduced pressure system is also solved using the GMRES algorithm, which results in two levels of GMRES iterations. At the outer level the full solution is obtained by performing a number of outer iterations with the two stage CPR preconditioner. For each outer iteration, a number of inner iterations are performed to obtain the pressure solution. The algorithm consists of the following steps:

1. The residual vector is restricted to the pressure system:

$$r_p = C^T G R \quad \text{Eq. 10.108}$$

2. The reduced pressure system is solved:

$$A_p x_p = r_p \quad \text{Eq. 10.109}$$

To solve the pressure system, Eclipse performs inner GMRES iterations with a nested factorization preconditioner, M_p . When the pressure system has been solved to the desired accuracy, the inner GMRES iterations are terminated.

3. The solution to the pressure system is then expanded to the full system, and provides a correction to the original residual:

$$\tilde{r} = r - A C x_p \quad \text{Eq. 10.110}$$

4. A second stage nested factorization preconditioner, $\tilde{M}$, is applied to the remaining residual to obtain a correction to the solution of the pressure system:

$$\tilde{x} = \tilde{M}^{-1} \tilde{r} \quad \text{Eq. 10.111}$$

5. The first and second stage solutions are combined, to provide the CPR preconditioned vector x to the FGMRES algorithm:

$$x = \tilde{x} + Cx_p \quad \text{Eq. 10.112}$$

Steps 1 to 5 are repeated within each outer FGMRES iteration, until the full solution has converged to within the required tolerance.

Information regarding the outer iterations can be reported with the usual performance summary quantities `NLINEARS`, `MLINEARS` and `MSUMLINS`. Information regarding the inner pressure iterations can be reported with the summary quantities `NCPRLINS`, `NLINEARP`, `MLINEARP` and `MSUMLINP`. Alternatively, the `PERFORMA` summary keyword can be used to provide all of this information.

It is more expensive to apply the two stage CPR preconditioner than the single stage nested factorization preconditioner that is used by default. Each linear iteration has a higher computational cost, and therefore takes longer to perform. However, convergence can usually be obtained in a smaller number of iterations of the full system. This makes the CPR preconditioner particularly suitable for simulations when the default linear solver is frequently failing to converge. In these cases, the more robust preconditioning could significantly reduce the number of linear iterations required and reduce the simulation time. The CPR preconditioner is also most suitable for fully implicit simulations, where the pressure system is significantly smaller than the full system.

In Eclipse 100, the CPR linear solver is compatible with the "[Reservoir Coupling](#)" option. However, inserting the `CPR` keyword in the master data file will not activate the CPR linear solver in any of the slave reservoirs. To activate the CPR linear solver in a particular slave reservoir, the `CPR` keyword must be used in the corresponding slave data file.

In Eclipse 100, there are some options that cannot be used with the CPR linear solver. These include [IMPES](#), the multiple independent reservoirs option, the Russian Doll option, the friction well option, triple porosity, nested local grid refinements, single phase simulations, item 1 of keyword [WSEGSOLV](#), [BTOBALFA](#), [BTOBALFV](#) and the rivers option. The use of [PINCHXY](#) with the [PARALLEL](#) option may result in an error, if there are horizontal pinchout connections that cross domain boundaries.

In Eclipse 300, it is currently not possible to use the CPR linear solver for the thermal option (the thermal option uses the `JALS` linear solver), the multiple (including triple) porosity option, in a dual porosity (or dual permeability) run containing local grid refinements and the `IMPES` option.

In Eclipse 300, the CPR linear solver generally performs best when the simulation is fully implicit. For simulations that use the adaptive implicit ([AIM](#)) scheme, the performance may deteriorate if the fraction of fully implicit cells is small.

Adaptive CPR

For Eclipse 100 the CPR linear solver can be used in `ORIGINAL` mode, or in `ADAPTIVE` mode. In `ORIGINAL` mode the CPR preconditioner is applied throughout the simulation. In `ADAPTIVE` mode the simulator performs tests to determine if it is efficient to apply the CPR preconditioner, so that it is only applied during difficult parts of the simulation. This can improve the overall performance, by applying the computationally more expensive CPR preconditioner only when it is required.

Note: When the CPR linear solver is used in `ADAPTIVE` mode it does not apply the default linear solver, even for cases where CPR is found to be inefficient, but instead applies the outer preconditioner mentioned in the previous section. Whilst `ADAPTIVE` CPR generally performs better than or comparable to `ORIGINAL` CPR, there will be some cases where the default linear solver performs better.

Tracer tracking

x	Eclipse 100
x	Eclipse 300

The Eclipse tracer tracking option is a general facility to follow the movement of ‘marked’ fluid elements during a simulation run.

The tracer tracking option has a wide variety of reservoir modeling applications. In the case of tracers defined to exist in the water phase, it may be used, for example, to determine the movement within the reservoir of water injected into any number of injection wells or to predict the variations in salinity or concentration of other chemical species in the water produced from the reservoir.

Eclipse 100

Tracers may also be defined to exist in a hydrocarbon phase or component, and the Eclipse 100 tracer option models the exchange of the tracer fluid elements between the free and solution states whenever mass transfer takes place in the associated hydrocarbon phase. The tracer option is therefore useful in predicting the movement of the initial free gas or solution gas during the evolution of reservoir.

Eclipse 300

The Eclipse 300 [TRACK](#) option allows a division of hydrocarbons in the original reservoir to be tracked through into recovered surface volumes.

The tracer concentrations can be initialized on a region basis using a depth table method of data input. This method of data input is especially suited to tracking fluid which initially exists within a particular region of the reservoir. In the case of gas condensate reservoirs, it is thus possible to track the oil which initially exists in the vapor phase in separate regions of the reservoir and to determine its subsequent movement through the reservoir grid. The tracer option could also be used to predict the concentration of chemical impurity species (such as the sulfur content of the oil) in the hydrocarbon production streams.

The Eclipse tracer tracking option allows up to 50 tracers to be defined in a single run. Each tracer exists in a particular stock tank phase although mass exchange of the tracer fluid may occur between the free and solution states of each phase. This ensures that if, for example, a tracer is defined to be the dry gas injected into an undersaturated oil reservoir the solution of the injected tracer gas into the oil is modeled accurately. Reinjection of tracer gas cannot, however, be modeled in the current tracer option.

The default Eclipse tracer tracking option assumes that the presence of tracers does not affect the PVT properties of the phases in which they are embedded. The tracers are thus to be regarded as passive. In Eclipse 100, the [Environmental tracers](#) option enables the modeling of contaminants and other substances as they flow within a host water, oil or gas phase. The tracer equations are a set of conservation equations for each tracer species with an exchange term for hydrocarbon phase tracers to model mass transfer effects. The tracer concentrations are updated fully-implicitly at the end of each timestep after the oil, water and gas flows have been computed.

Control of numerical diffusion (Eclipse 100)

The differential equations governing the evolution of the concentration of tracer species within each grid cell are a series of linear convection equations. Eclipse solves the tracer transport equations at each timestep using a fully implicit time discretization. Such a system of difference equations is particularly prone to exhibiting the effects of numerical diffusion errors, particularly in regions of the grid where tracer concentrations vary most rapidly. The smearing of tracer interfaces tends to be most apparent in single phase regions (for example gas injection into a gas cap, water injection into aquifers).

There are two options in Eclipse 100 to reduce the effects of numerical diffusion:

Cascade algorithm

This option is invoked by setting item 5 to 'DIFF' in keyword [TRACERS](#) in the RUNSPEC section. At present this option applies to tracers embedded in phases where no mass transfer effects can take place.

The tracer concentrations are calculated in flow order (the cascade). In cells in which the phase throughput ratio is less than one, the explicit tracer concentrations are used. However, if the throughput ratio exceeds one, the tracer concentration cascades on to the downstream cells. Hence the algorithm is stable for all timestep sizes.

In a serial run, the cascade algorithm is always used for **active** tracers whenever no mass transfer is associated with these tracers, whether or not 'DIFF' is enabled in the RUNSPEC section keyword TRACERS. It is possible to stop the cascade algorithm being used by employing the **NOCASC** keyword in the RUNSPEC section. This forces the linear solver algorithm to be used for solving all single phase tracers.

Note: The cascade algorithm is not available for parallel runs. Thus, switching to parallel in a model containing active tracers automatically forces the linear solver algorithm to be used. Consequently, small changes in results can be expected. To avoid these changes between serial and parallel runs the **NOCASC** keyword should be used in the serial run.

Note: Brine is also an active tracer and prior to the 2004A release was also solved automatically with the cascade method. Since the 2004A release, a brine tracer has been solved with the user-selected method. The cascade solver cannot be used for the Multi-component brine option.

Flux limiting scheme

The explicit flux limiting scheme is invoked by setting item 5 to 'DIFF' in keyword TRACERS in the RUNSPEC section, and in addition specifying the **TRACTVD** keyword in the PROPS section. The implicit flux limiting scheme is invoked by specifying the **TRACITVD** keyword in the PROPS section. These options only apply to tracers embedded in phases where no mass transfer effects can take place.

The scheme is second order in smoothly varying regions, but uses a limiting function to prevent unphysical oscillations near sharp fronts.

For the explicit flux limiting scheme, tracer concentrations are updated explicitly using a mini timestep to prevent instability. As the tracer equation is linear, these explicit timesteps are very cheap, and hence the method can be applied to most problems (*Rubin and Blunt, 1991*).

For the implicit flux limiting scheme, the tracer concentration is computed using the same timestepping control as the standard fully implicit solver.

Lagrangian tracer solver (Eclipse 300)

The Lagrangian tracer solver is activated with the **LAGTRAC** keyword in the RUNSPEC section and the **LAGTRUPD** keyword in the SCHEDULE section.

This solver is used in place of the standard implicit tracer solver for all tracers when activated. It uses FrontSim technology (for example the Pollock method) to trace a set of streamlines based on the flows computed by Eclipse. The tracer solver uses these streamlines to find the source of fluid in each cell and updates the tracer concentration in each cell according to the tracer concentration in the source cells.

This technique is not fundamentally conservative and does require that the streamline set accurately represents the flow of fluid in the reservoir. Assuming that this is the case, this method can give a faster, less numerically diffusive tracer solution. To ensure that the streamlines accurately describe the reservoir flow it is important that there are enough streamlines and that the streamlines are updated if the reservoir flow changes due to changes to well controls, movement of fronts and so on. The updates occur according to the data entered in the **LAGTRUPD** keyword.

The tracer concentration at the time that the streamline is traced is stored on the streamline; it is this value that is used, along with the time since this value was stored, to determine tracer concentration at the destination cell. This allows a solution with less numerical diffusion. You can update these concentrations without retracing the streamlines using `LAGTRUPD`. In any event these values are updated at each report step.

As the Lagrangian Tracer Solver is a streamline-based method, some physical processes are not solved as accurately as others. Gravity segregation can lead to the net flow of a component giving a poor representation of the movement of tracers; as the liquid phase drops and the vapor phase rises the net flow of a component will not necessarily give good results. Another area of difficulty is strongly convergent or divergent flow, such as one might find in a reservoir produced by pressure depletion. While the `LAGTRAC` algorithm contains correction terms for compression and divergence, in strong cases these first order corrections will not be sufficient.

Lagrangian tracer solver speed and accuracy

The Lagrangian tracer solver is very quick at calculating tracer distributions once a streamline set has been traced. Tracing a set of streamlines is a slow process. As a result there is a trade-off between speed and accuracy which may be controlled by judicious use of the `LAGTRAC` and `LAGTRUPD` keywords.

The third argument of the `LAGTRAC` keyword should be used to determine the number of streamlines to be traced; more streamlines will slow down both the streamline tracing and the solution of the tracer concentration. The default value has been chosen to give a reliably accurate solution, but this can be reduced in most cases to a value of 0.05.

The `LAGTRUPD` keyword may be used to control the frequency with which streamlines are retraced. The default values cause streamlines to be retraced every timestep; this is safe but slow. In the majority of cases the retrace frequency can be reduced dramatically, with a retrace roughly every report step being reasonable, and a value of at least 0.1 for argument 3 also being safe in most cases.

One final point to consider is that a set of streamlines can be used for more than one tracer. This means that multiple tracers which have the same carrier (that is tracers with the same argument 2 in `TRACER` and all `LUMPING` tracers) are all solved using one streamline set. As the majority of the cost of the Lagrangian tracer option is associated with tracing the streamlines, this means that every tracer after the first for a given carrier has a very marginal cost in memory and CPU time. This contrasts sharply with the standard tracer solver where the cost of calculation is almost proportional to the number of tracers.

Lagrangian tracer solver in parallel

The Lagrangian tracer solver is compatible with the `PARALLEL` keyword. This is implemented by gathering the grid and flow information necessary to trace a streamline onto the master processor and then copying this information to all processors. Each processor then traces some of the streamlines required, and the streamlines are finally mirrored across processors. This allows the tracer solver to run without any cross-processor communication being necessary except when tracing streamlines.

Lagrangian tracer summary

The `LAGTRAC` keyword is used to activate the Lagrangian Tracer Solver. The `LAGTRUPD` keyword is then used to define the solver method and the streamline update strategy. The `RPTSLN` keyword can be used to output the streamlines produced by the Lagrangian solver as SLN format files. `LTRACE` and `NOLTRACE` keywords are also available to allow you to fine-tune the update of the streamlines.

Using the Eclipse tracer tracking facility

Eclipse 100

The tracer tracking facility in Eclipse 100 is activated by means of three integers NOTRAC, NWTRAC, NGTRAC set in keyword **TRACERS** in the RUNSPEC section. These three integers represent the maximum number of tracers to be defined in each of the three stock tank phases: water, oil and dry gas. It is recommended that the precise number of tracers to be tracked should be specified in keyword **TRACERS** in the RUNSPEC section in order to minimize the memory requirements of the model.

Eclipse 300

The tracer tracking facility in Eclipse 300 does not require the keyword **TRACERS** in the RUNSPEC section; it is activated by entering the **TRACER** keyword in the PROPS section of the input data.

The names of each tracer and their associated stock tank phases are defined using the **TRACER** keyword in the PROPS section of the input data. In Eclipse 100 the maximum number of rows of data is (NOTRAC + NWTRAC + NGTRAC). Each tracer name is, at present, limited to be a name of up to 3 characters in length. The output of **TRACER** data is controlled by the **TRACER** argument in the **RPTPROPS** keyword.

Eclipse 100

In the case of tracers defined to exist in the gas phase when **DISGAS** is specified in the RUNSPEC section, or tracers defined in the oil phase when **VAPOIL** is specified in RUNSPEC, the full tracer solution consists of a concentration in both the free and solution states of its associated phase. Eclipse 100 automatically allocates space for tracers which may exist in the free or solution state, and issues error messages whenever insufficient data is provided to calculate the initial tracer conditions.

The initial tracer concentrations may be supplied in one of two ways. The most direct method is enumeration using the keyword **TBLK** in the SOLUTION section of the input data. An alternative method is to define a series of tracer concentration versus depth tables in the SOLUTION section, together with the associated region numbers in the REGIONS section of the input data. The data is entered by means of the keywords **TVDP** and **TNUM** respectively, and control of the depth table and region data output is achieved using the **RPTSOL** and **RPTREGS** keywords. The keywords **TBLK**, **TNUM** and **TVDP** must be used in conjunction with the character F or S (free or solution), together with the tracer name, to make a composite keyword of up to 8 characters in length. For example, to initialize a water phase tracer whose name is 'WTR', the keywords required for the region/depth table method of input would be **TVDPFWTR** and **TNUMFWTR**.

It is possible to restart a run with more tracers defined than in the original base run; this allows a tracer run to be restarted from a non-tracer base case. The extra tracers must be initialized using either **TBLK** for **TVDP** keywords as described above. If the **TBLK** or **TVDP** keywords are used for tracers that have read from the **RESTART** file, then the data is ignored.

You may specify the initial concentration of a water phase tracer in each of the analytic aquifers in a simulation run. This can be achieved using the **AQANTRC** keyword in the SOLUTION section of the input data. Output of the status of each water tracer for each analytic aquifer is obtained at each report time using the argument **AQUCT** or **AQUFET** in the **RPTSCHED** keyword.

Tracer injection control is achieved by means of the **WTRACER** keyword in the SCHEDULE section. This keyword allows the specification of the concentration of a particular tracer in the injection stream for each well. A default injection concentration value of zero is assumed for tracers not defined under the **WTRACER** keyword. The **WTRACER** keyword optionally allows the injected tracer concentration to be a function of the total cumulative fluid injected, which can be useful for modeling H₂S generating processes.

Note: Injection/production of tracers in wells having significant crossflow may lead to tracer mass balance errors due to current limitations in the tracer model. The mass balance errors can be examined in the tracer-in-place reports in the print file, enabled by argument **FIPTR** in keyword **RPTSCHED**. For non-linear tracers, for example environmental tracers with adsorption enabled, the issue with crossflow in wells may

lead to convergence problems. The problem can be resolved by disabling crossflow (see item 10 of the [WELSPEC](#) keyword).

Eclipse 100

Reporting of the tracer concentration in each grid cell is activated using the argument `TBLK` or `TRACER` in the `RPTSCHED` keyword. Output of tracer volumes within each reporting region and the inter-region tracer flows are available using the argument `FIPTR` in the `RPTSCHED` keyword. Grid block arrays of tracer concentration are reported under a mnemonic consisting of the tracer name together with the character `F` and/or `S`. For example, if `IGS` is a gas phase tracer in a run with dissolved gas ([DISGAS](#) is specified in the `RUNSPEC` section), then two grid block concentration arrays are produced under the arguments `IGSF` and `IGSS` which represent the local concentration of `IGS` in the free and dissolved gas respectively.

Eclipse 300

Each tracer sets up a solution array, with its own name, which may be reported using [RPTSOL](#) and `RPTSCHED`, or output to graphics using [RPTRST](#). For example:

```
RPTSOL
PRESSURE SWAT TR1 TR2 /
RPTRST
PRESSURE SWAT TR1 TR2 /
```

Completion, well, group and field reports for production/injection rates and cumulative production/injection of each tracer are provided with the well reports in the `PRINT` file, which may be requested by using the argument `WELLS` in the `RPTSCHED` keyword.

Eclipse 100

Tracers can be injected in rivers using the keyword [RIVTRACE](#) in the `SCHEDULE` section. Tracers can be injected in a specified reach of the river and through the upstream end of a river branch. The `RIVTRACE` keyword optionally allows the injected tracer concentration to be a function of the total cumulative water flow.

Tracking original fluids

This section describes the operation of the Eclipse 300 `TRACK` option, which enables a division of hydrocarbons in the original reservoir to be tracked through into recovered surface volumes. The option is enabled by entering the [TRACK](#) keyword with appropriate arguments in the `RUNSPEC` section. To use the enhanced tracking facility, with user-defined regions, it is also necessary to define the maximum number of regions to be used in the [REGDIMS](#) keyword in the `RUNSPEC` section.

Formulation

Suppose that we divide the hydrocarbons in a reservoir into those lying above the GOC (gas-oil contact) and those lying below it. As the reservoir is produced these components will flow through the field to the production wells. Generally, each component will move along a different path. For example, the light components such as methane will fraction into the reservoir vapor, and tend to rise to the top of the reservoir. The heavy components will tend to remain in the oil and stay at the bottom of the reservoir.

At a given producer, each component will have a fraction from below the GOC and a fraction from above it. We can thus define two streams, one of components originating from below the GOC and one from above. If these are run separately through the separator system we can obtain stock tank oil and stock tank gas volumes attributable to each of these streams.

With the enhanced Tracking option you can to divide the reservoir in any way that you choose.

Usage

The Tracking option is turned on using the `TRACK` keyword. This allows the initial fluid to be split at the GOC or at a given depth. The following text discusses splitting at the GOC; splitting at a specified depth is analogous.

In the tracking model, production volume rates originating from hydrocarbons above and below the GOC are reported separately in the `PRINT` file and when using `SUMMARY` section keywords.

Summary items are as follows:

For wells

The `SUMMARY` keyword for the oil surface volume production rate of a given well is `WOPR`. The oil surface volume production rates from above and below the GOC are `WOPRA` and `WOPRB` respectively.

These are totaled over time separately, so that the total production from the well is `WOPT` and the totals from above and below the GOC are `WOPTA` and `WOPTB`.

`WGPR`, `WGPRB`, `WGPTA` and `WGPTB` are similar items for well gas rates and totals.

For groups

The well contributions are summed over the group structure.

`GOPR` and `GOPRB` are the group rates from above and below the GOC. These are totaled over time, so that `GOPTA` and `GOPTB` are the group totals from above and below the GOC. `GGPR`, `GGPRB`, `GGPTA` and `GGPTB` are similar items for group gas rates and totals.

For the field

The field items follow those of the well and groups.

`FOPR` and `FOPRB` are the field rates from above and below the GOC; `FOPTA` and `FOPTB` are the field totals from above and below the GOC.

`FGPR`, `FGPRB`, `FGPTA` and `FGPTB` are similar items for field gas rates and totals.

Enhanced tracking

To enable enhanced tracking it is also necessary to use the `REGDIMS` keyword in the `RUNSPEC` section to define the maximum number of user-defined regions. It is strongly recommended that this maximum does not exceed the actual maximum as it can represent a considerable unnecessary overhead. An additional option in the `TRACK` keyword is the ' `REGONLY` ' keyword that can be used in place of ' `GOC` ' or ' `DEPTH` ' if region tracking is the only facility required.

To define the regions the `TRACKREG` keyword in the `REGIONS` section must be used.

Fluid in place reports

When the fluid in place in the reservoir at surface conditions is reported, the track option includes extra reports for the surface volumes obtained from the fluid in the reservoir which originated above and below the gas oil contact. These volumes may be obtained for the entire reservoir or by fluid in place reporting region.

Some reported tracer quantities can be prone to rounding errors if the model is made of cells with exceeding small pore volumes. This might happen if modeling a core when using `FIELD` or `METRIC` units. In this situation the solution is to convert to `LAB` units.

Tracer structure

Internally, the `TRACK` option sets up tracer variables. These are called A_n , for component n above the GOC and B_n for component n below the GOC. Suppose a tracer is attached to the methane. Then a tracer A_1 will be set up, initialized to 1 for cells above the GOC and 0 for cells below the GOC. The tracer B_1 will be conversely initialized to 0 for cells above the GOC and 1 for cells below the GOC. (Cells which intersect the GOC are given appropriate fractional values.)

Although the A_n and B_n tracers are set up automatically, you can still display their concentration profiles in the usual way by requesting output using `RPTRST`.

It might be thought that one tracer per component would be sufficient. However, the problem is with injection hydrocarbon, which cannot be identified with either the ‘above’ or ‘below’ original components, so that we cannot assume that components not belonging to one set must belong to the other.

For the most accurate solution we should track all of the components in the reservoir, as each will generally form a different distribution in the reservoir. However, this may be expensive, as $2N_c$ tracers will be required. An alternative is to track a smaller number and interpolate or extrapolate to find the concentrations of the missing tracers. At least one light component (generally C_1) and one heavier component (for example C_7) should be tracked. The `TRACK` keyword allows you to select which components are to be tracked, or to select `ALL` for the full set.

Enhanced tracking

The naming of the tracers is done in a similar manner. If the ‘`REGONLY`’ option is used then the tracer A_n refers to region 1, B_n to region 2, and so on. In the ‘`REGONLY`’ case it is not possible to report quantities from the regions above and below the GOC, such as `FOPRA`, `FOPRB`, `FOIPA`, and `FOIPB`, as separate tracers have not been defined in these regions. If the ‘`GOC`’ or ‘`DEPTH`’ option has been used the A_n and B_n tracer names are reserved for those, with region 1 tracers being denoted with C_n , region 2 tracers with D_n , and so on. A table is output to the print file detailing the names of the tracers and which regions they refer to.

Note that the comments about expense given above are multiplied here if a significant number of regions are defined.

The normalization option

When the fluid in place or the production from a well is obtained the tracers are used to define a stream from above the GOC and a stream from below the contact. These are then flashed separately so that the volumes from above and below can be reported separately. The sum of these two volumes will not generally be exactly equal to the total volume produced by the well, for three reasons:

- Injection gas, which is not from original reservoir components.
- The separator flash is not linear, so the volume of an oil and gas resulting from a mixture is not the same as the sum of the volumes of two sub-mixtures.
- The tracer solution will disperse like any numerical solution, so that the sum of the A and B tracer concentrations will not generally equal exactly one.

Note: The `NORM` option is provided so that you can request that the volumes be normalized to equal the total rate of the well. If normalization with the `TRACK` keyword (argument 3) is activated then `FGPRA`, `FGPRB`, `FGPTA` and `FGPTB` and similar items for well and group gas rates are calculated as:

$$FGPRA = \frac{FGPR}{FGPRA + FGPRB} FGPRA$$

Alternative production reporting

An alternative method for calculating the production from above and below the GOC is to directly split a well's volumetric flow rate according to the tracer concentrations above and below the contact. So, for example, the well oil production from fluids above the GOC, $WOPR1$, is:

$$WOPR1 = WOPR \cdot \sum_{i=1}^N WXMF_i \cdot T_i \quad \text{Eq. 10.113}$$

where:

$WOPR$ is the well volumetric oil production rate

$WXMF_i$ is the well liquid mole fraction for component i

T_i is the above GOC tracer concentration for component i

N is the number of components.

Similar equations can be written for the well oil production from fluids below the GOC and for the gas production rates. This way of splitting a well's production ensures that the sum of the volume from above and below the GOC equal the total volume produced by the well in the absence of injection gas and subject to the numerical accuracy of the simulation.

This method is known as the molar volumetric split and the resulting field, group and well rates and totals are calculated in addition to the standard `TRACK` option production quantities.

The molar volumetric split quantities may be reported using the following `SUMMARY` section keywords.

Field	Well	Group	Description
F (O, G) P (R, T) 1	W (O, G) P (R, T) 1	G (O, G) P (R, T) 1	gives the oil and gas surface volume production rates and totals from fluids above the GOC
F (O, G) P (R, T) 2	W (O, G) P (R, T) 2	G (O, G) P (R, T) 2	gives the oil and gas surface volume production rates and totals from fluids below the GOC

Following the tracers

Tracer variables are concentrations, which normally lie in the range 0 to 1. Each tracer is associated with a carrier component. The equation defining the change in the tracer solution over a timestep is then:

$$R_i = M_i^{T+\Delta T} \cdot C_i^{T+\Delta T} - M_i^T C_i^T + \Delta T \sum_n F^{i \rightarrow n} C_{in}^{T+\Delta T} + \Delta T C_i^{T+\Delta T} Q_{Mi} = 0 \quad \text{Eq. 10.114}$$

where:

M_i is the moles of carrier in cell i

C_i is the tracer concentration in cell i

$F^{i \rightarrow n}$ is molar carrier flow from cell i to neighboring cell n

Q_{Mi} is the carrier molar production from well completion in block i

C_{in} is the upstreamed tracer concentration from cell i to neighboring cell n .

Note: The tracer conservation equation is linear in C , and thus solves in one iteration of Newton's method.

SUMMARY section keywords

Refer to [Tracer and API tracking quantities](#) in the *Eclipse Reference Manual*.

The following table give additional SUMMARY keywords that are compatible with the Eclipse 300 Track option if either of the ' GOC ' or ' DEPTH ' options is used. They are not available if the ' REGONLY ' option is used.

Field	Group	Well	Description
FOPRA	GOPRA	WOPRA	Oil Production Rate above GOC
FOPRB	GOPRB	WOPRB	Oil Production Rate below GOC
FOPTA	GOPTA	WOPTA	Oil Production Total above GOC
FOPTB	GOPTB	WOPTB	Oil Production Total below GOC
FOPR1	GOPR1	WOPR1	Oil Production Rate above GOC (molar method)
FOPR2	GOPR2	WOPR2	Oil Production Rate below GOC (molar method)
FOPT1	GOPT1	WOPT1	Oil Production Total above GOC (molar method)
FOPT2	GOPT2	WOPT2	Oil Production Total below GOC (molar method)
FGPRA	GGPRA	WGPRA	Gas Production Rate above GOC
FGPRB	GGPRB	WGPRB	Gas Production Rate below GOC
FGPTA	GGPTA	WGPTA	Gas Production Total above GOC
FGPTB	GGPTB	WGPTB	Gas Production Total below GOC
FGPR1	GGPR1	WGPR1	Gas Production Rate above GOC (molar method)
FGPR2	GGPR2	WGPR2	Gas Production Rate below GOC (molar method)
FGPT1	GGPT1	WGPT1	Gas Production Total above GOC (molar method)
FGPT2	GGPT2	WGPT2	Gas Production Total below GOC (molar method)

Example

As an example, we consider the third SPE benchmark problem ([Kenyon and Behie, 1987](#)), which is a small gas condensate study. The top of the reservoir is at 7315 feet and the bottom of the reservoir is at 7475 feet. We modify the problem slightly to place a gas-oil contact at 7400 feet, and produce the field using the standard gas cycling production strategy.

The TRACK option is selected as follows:

```

TRACK
GOC 1* U C1 C7+2 /

```

This requests initialization with respect to the GOC, no normalization of the output volumes and that tracking use tracers on the C1 and C7+2 components only.

In the SUMMARY section we request output of the field oil production rate, and the contributions from hydrocarbons originally above and below the GOC:

```

FOPR
FOPRA
FOPRB

```

The resulting production profiles are:

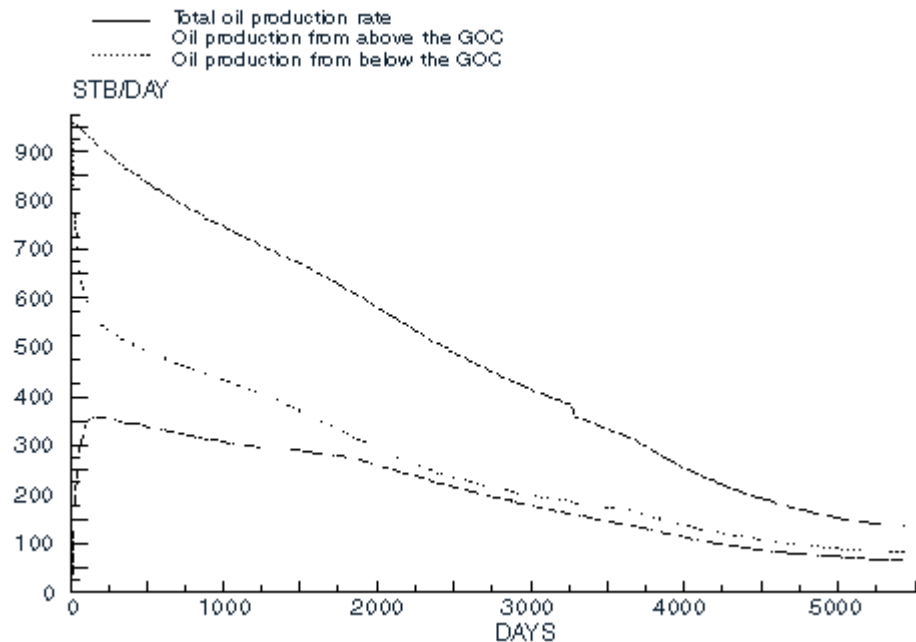


Figure 10.4. Total oil production rate and contributions from above and below GOC

Note: As the production well is completed below the GOC, initially all the production comes from here. This changes quickly as fluid movement drive by gas injection moves hydrocarbon down from cells above the GOC.

Delumping and lumping

This feature, which is available in Eclipse 300, is a straightforward implementation of paper SPE 49068 by [Leibovici et al \(1998\)](#). The basic idea is to do a full simulation with as small a number of components as possible and then to use multiple tracers carried on each of these ‘lumped’ components to represent the detailed composition of that component. The rationale and details of the approach are explained in the above paper; here we will concentrate on how to apply this procedure.

The facility is switched on with the `LUMPDIMS` keyword in the `RUNSPEC` section. This gives the number of lumped components (not necessarily as many as the number of components in the ‘lumped’ simulation) and a maximum for the number of detailed components.

The detailed composition of the fluid is described in the PROPS section. The **LUMPING** keyword assigns the number of detailed components to each lumped hydrocarbon component and also allows names to be attached to them. **The order in which the detailed components are entered is important as it will be assumed by later keywords.** The initial detailed state of the reservoir is specified with the **DETAILMF** or **DETAILVD** keyword, which determines the proportion of the detailed components making up the lumped components. This can vary with depth. A table is entered for each tracer region (if more than one), defined with keyword **TNUM**.

Various physical properties of the detailed components can also be specified in the PROPS section with the **ACFDET**, **MWDETAIL**, **TCRITDET**, **PCRITDET**, **OMEGAADE**, and **OMEGABDE** keywords. These are identical in nature to their ‘normal’ simulation counterparts, though as in a normal simulation, not all the ***CRITDET** keywords are needed in a given simulation.

The detailed composition of an injected fluid is given by using the **WINJEDET** keyword in the SCHEDULE section. This must be given for each injection well. For a given well the make-up of the detailed fluid is described in an analogous way to the **DETAILVD** keyword.

Output relies on the standard tracer output with the well, group and field tracer concentration (summary keywords **WTPC**, **GTPC** and **FTPC**) now representing the mole fraction of that detailed component within the overall fluid from that well, with the names of the tracers being those given in the **LUMPING** keyword. No other summary output is available; in particular the “Traced Mole Fraction” summary keywords; **FTMF**, **GTMF** and **WTMF** will not report useful quantities as these are the product of the tracer concentration with the carrier molar rate.

Diffusion and dispersion

Eclipse 100

Prior to 2004A diffusion was only available for environmental tracers. With the 2004A release diffusion was made available for all tracers. Also a form of velocity and concentration dependent dispersion was added, see the keywords **DISPDIMS**, **TRDIS** and **DISPERSE**.

In 2005A mechanical dispersion (see *Charbeneau (2000)*), keyword **HDISP**, was also introduced. Note that mechanical dispersion is sometimes referred to as hydraulic dispersion, or streamwise diffusion, or streamwise dispersion.

Treating passive tracers as non-linear

Eclipse 100

Sometimes problems can occur with passive tracers, which show as solutions values unacceptably below the minimum, or above the maximum, value input. Whilst the simulator should solve the passive tracer problem exactly (which in some sense it does do) minute mass balance errors in the underlying reservoir simulation get massively magnified in the tracer calculation leading to obviously incorrect results. Two-phase tracers seems particularly sensitive. The only way to overcome this problem is to solve the underlying reservoir simulation to a greater accuracy; usually well beyond where it appears to have any effect on the reservoir simulation. This unavoidably leads to large, often very large, increases in CPU times. The idea of treating passive tracers as if they were non-linear is an attempt to try to automate the process of adjusting the convergence tolerances for the simulation.

Tip: It would always be better to use **TUNING** to tighten the tolerances before the tracers notice any difficulty. It is always possible that it is too late by the time the tracers fail to converge.

Having chosen to treat a passive tracer as a non-linear tracer, with item 8 of the **TRACERS** keyword, Eclipse 100 then performs checks on the convergence of the tracer. If it notices a problem the timestep is chopped and the whole solution (including the underlying reservoir simulation) is repeated with the new

timestep. The first time this occurs `MATCORR` is switched on (if it is not already on). Also the linear and non-linear convergence tolerances are tightened by the one-off application of the factors in items 9 and 10.

Tip: We suggest the choice of a factor of 0.01-0.001 for item 9 and leaving item 10 defaulted. This is because increasing the linear iterations has a direct effect on material balance and is relatively cheap. For items 11 and 12 values of 0.1-0.3 and 3-4 seem reasonable.

Tip: It is important not to allow the tolerances to be tightened too much as the simulator will reach a point where the tolerances are below the rounding error and it becomes impossible to satisfy the conditions.

Eclipse 300

In Eclipse 300 it is not possible to treat the tracer equations as non-linear. Instead it may be necessary to ensure small residuals of the reservoir equations, for example keyword `CVCRIT` can be used to control the size of the residuals (items 41 or 42) or to increase the number of non-linear iterations (item 9). When solving the tracer equations Eclipse 300 uses the latest solution and updates the inter-block flows. However, it uses lagged NNC flows, calculated before the solution was updated. In cases where NNC flows are important, such as dual porosity, a more accurate solution may be obtained by updating the NNC flows prior to the tracer solves. This can be done using item 391 of keyword `OPTIONS3`.

Environmental tracers

x	Eclipse 100
	Eclipse 300

The environmental tracers option is an extension to the Eclipse 100 passive tracer tracking model; it is not available in Eclipse 300. The option enables the modeling of contaminants and other substances as they flow within a host water, oil or gas phase.

The passive tracer tracking models give you the ability to track up to 50 tracer substances within a single model. The environmental tracer option extends the modeling to account for adsorption of the tracer on to the bulk rock, for decay of the tracer over time, and for molecular diffusion of the tracer. It is possible to model adsorption, decay and diffusion within a single tracer.

The environmental tracer model

As for passive tracers, the flow of an environmental tracer through the porous medium is assumed to have no influence on the flow of the water and hydrocarbon phases or on the flow of other tracers.

Environmental tracers are therefore solved, in a similar fashion to passive tracers, at the end of a timestep after the oil, water and gas equations have converged.

For an environmental tracer present in a single phase the governing equation is:

$$\begin{aligned} & \frac{d}{dt} \left(\frac{VSC}{B} \right) + \frac{d}{dt} \left(V\rho_r C^a \frac{1-\Phi}{\Phi} \right) \\ &= \sum \left[\frac{Tk_r}{B\mu} (\delta P - \rho g D_z) C + DFD_c S \delta C \right] + QC - V \frac{S}{B} \lambda C \end{aligned} \quad \text{Eq. 10.115}$$

where

S denotes the host phase saturation

C denotes the flowing tracer concentration

C^a denotes the adsorped tracer concentration

ρ_r denotes the mass density of the rock formation

ϕ denotes the porosity

ρ denotes the host phase density

μ denotes host phase viscosity

D_z denotes the cell center depth.

B denotes the host phase formation volume factor

T denotes the transmissibility

k_r denotes the host phase relative permeability

V denotes the block pore volume

Q denotes the host phase production rate

P denotes the host phase pressure

g denotes the acceleration due to gravity

D_c denotes the tracer diffusion coefficient

DF denotes the diffusivity

λ denotes the tracer decay constant

$\sum$ denotes the sum over neighboring cells

Treatment of adsorption

Adsorption is treated as an instantaneous effect in the model. The effect of tracer adsorption is to strip tracer from the leading edge of a tracer front. Desorption effects may occur as the tracer concentration decreases.

The adsorption model can handle both stripping and desorption effects. You specify an adsorption isotherm, which tabulates the saturated rock adsorbed concentration versus the local tracer concentration in solution. As an alternative to tabulating, the isotherm coefficients can be supplied for an analytical isotherm which allows for dependencies of adsorption on rock permeability.

There are currently two adsorption models which can be selected. The first model ensures that each grid cell retraces the adsorption isotherm as the tracer concentration rises and falls in the cell. The second model assumes that the adsorbed tracer concentration on the rock may not decrease with time, and hence does not allow for any desorption. More complex models of the desorption process may be implemented if required.

To ensure equilibrium, the initial input tracer concentrations can either partition between the fluid and solid, thereby conserving the total input concentration in the fluid, or can be honored by adding adsorbed tracer concentration to the rock. The choice of initialization model is specified in item 3 of the keyword [TRROCK](#).

Treatment of decay

Decay of the tracer is modeled as a simple half-life process. You specify a half-life for each tracer, and the tracer starts decaying as soon it is injected into the reservoir.

Treatment of diffusion

The diffusive flow of tracer from cell i to a connected cell j is given by equation [10.115](#):

$$F_t = DF \cdot D_c S (C_{ci} - C_{cj}) \quad \text{Eq. 10.116}$$

The diffusivity (DF) is analogous to the transmissibility and has the form:

$$DF = \frac{A \cdot \phi}{d} \quad \text{Eq. 10.117}$$

where

A denotes the interface area between the two cells

d denotes the distance between the cell centers

The details of the diffusivity calculation can be found under "[Diffusion](#)".

In a system where there is significant convection of the tracer, molecular diffusion is likely to be a relatively small effect. Numerical dispersion associated with the discrete approximation of the flow

equation is likely to dominate any molecular dispersion. However, the diffusion model is useful when the convection is small. In the limiting case of zero permeability, the only mechanism for flow is by diffusion; this kind of situation may occur when modeling a contaminant initially stored in concrete.

Partitioned tracers

Partitioned tracers allow Eclipse 100 to solve a range of problems that could not be tackled otherwise. For example, if marked gas is injected into a reservoir it is possible that the marker may dissolve in the water. The partitioned tracer option can model this behavior. With a single phase tracer in, say, the gas phase, Eclipse solves a conservation equation for the total amount of tracer in a grid block taking into account the inflow and outflow of gas, equation 10.115. With a partitioned tracer one or more additional phases can be nominated (in the `TRACER` keyword) into which the tracer may also dissolve.

Note: The partitioned tracer functionality can be used independently of adsorption, decay and diffusion. Hence, it is possible to have a standard passive tracer which partitions into multiple phases.

Standard partitioned tracer

A standard partitioned tracer can exist in two phases which are specified in the `TRACER` keyword. For input and output purposes these two phases are regarded as being the ‘free’ and ‘solution’ phases, so that the conventional Eclipse 100 keywords for two-phase tracers can be used. The ‘free’ phase can be thought of as the reference phase for the tracer.

Using subscripts ‘f’ and ‘s’ to denote the free and solution phases, respectively, the governing equation for a standard environmental partitioned tracer is:

$$\begin{aligned} \frac{d}{dt} \left(V \left[\frac{S_f}{B_f} C_f + \frac{S_s}{B_s} C_s \right] \right) + \frac{d}{dt} \left(V \rho_r C^a(C_s) \frac{1-\Phi}{\Phi} \right) \\ = \sum [F_f + F_s] + Q_f C_f + Q_s C_s - V \frac{S_s}{B_s} \lambda C_s \end{aligned} \quad \text{Eq. 10.118}$$

$$\begin{aligned} F_f &= \frac{T k_{rf}}{B_f \mu_f} (\delta P - \rho_f g D_z) C_f \\ F_s &= \frac{T k_{rs}}{B_s \mu_s} (\delta P - \rho_s g D_z) C_s + D F D_c S_s \delta C_s \end{aligned} \quad \text{Eq. 10.119}$$

The concentration of the tracer in the solution phase is not independent of the concentration in the free phase. The concentrations are related through the partitioning functions:

$$K(P) = \frac{C_s}{C_f} \quad \text{Eq. 10.120}$$

where the functions $K(P)$ are entered in tabular form in the `TRACERKP` keyword and are required for each grid block. The particular table to be used is specified with the `TRKPFxxx` keyword, which is input in the `REGIONS` section. A table number is also required for use within the wellbore, and this is specified in item 5 of the `TRACER` keyword.

As shown in equations 10.118 and 10.119, when a standard partitioned tracer is used in conjunction with adsorption, decay and diffusion, it is assumed that these processes happen only in the solution phase. For example, the adsorption function, $C^a(C_s)$, in equation 10.118 is evaluated based on the tracer concentration in the solution phase.

Multi-partitioned tracers

The multi-partitioned tracer option is an extension and generalization of standard partitioned tracers. A multi-partitioned tracer, which is defined by specifying `MULT` in the item 4 of the `TRACER` keyword, can partition into any number of phases and have phase specific adsorption, decay and diffusion parameters.

The governing equation for a generalized, multi-partitioned tracer is:

$$\frac{d}{dt} \left(V \sum_j \frac{S_j}{B_j} C_j \right) + \frac{d}{dt} \left(V \rho_r C^a(C) \frac{1-\Phi}{\Phi} \right) = \sum_j F_j + \sum_j Q_j C_j - V \sum_j \frac{S_j}{B_j} \lambda_j C_j \quad \text{Eq. 10.121}$$

$$F_j = \frac{T k_{rj}}{B_j \mu_j} (\delta P - \rho_j g D_z) C_j + D F D_{cj} S_j \delta C_j \quad \text{Eq. 10.122}$$

where the summation over j indicates the phases in which the tracer is present, typically (a subset of) oil, water and gas.

As for standard partitioned tracers, the concentrations of the tracer in different phases are related through pressure dependent partitioning functions. For a multi-partitioned tracer these functions are entered in tabular form in the `TRACERKM` keyword. The reference phase is taken as the phase specified in the second item of the `TRACER` keyword. `TRACERKM` then specifies partitioning functions relating the concentration in each of the solution phases to the concentration in the reference phase. For example, for a tracer partitioning into oil, water and gas with water as the reference phase, `TRACERKM` holds two columns of data specifying the functions:

$$K_{ow}(P) = \frac{C_o}{C_w} \quad K_{gw}(P) = \frac{C_g}{C_w} \quad \text{Eq. 10.123}$$

Decay and diffusion can be modeled in all phases with phase specific decay and diffusion constants. For adsorption, the concentration to use when evaluating the adsorption function can be specified in the sixth item of the `TRACER` keyword. The options are to use either one of the concentrations in the phases that the tracer partitions into, or to use the overall, pore-volume-averaged, concentration defined by:

$$C = \sum_j \frac{S_j}{B_j} C_j \quad \text{Eq. 10.124}$$

Initialization of partitioned tracers

The initial partitioned tracer concentrations are input in tabular form using either of the keywords `TBLK` or `TVDP`. The keyword `TBLK` specifies the initial concentrations of a tracer in each grid block, whereas the keyword `TVDP` specifies tables of initial tracer concentration versus depth. Only the tracer concentration for the free (reference) phase is specified in the `SOLUTION` section. The initial free phase tracer concentrations are then updated using the formula

$$C_f = \frac{C_f^{ini} FIP_f}{\sum_j K_{jf}(P) FIP_j} \quad \text{Eq. 10.125}$$

where

FIP_j is the fluid in place for phase j and

$K = 1$ for $j = f$.

The concentration of the solution phase(s) is then calculated from the relationship $C_j/C_f = K_{jf}(P)$ which has been input using the [TRACERKP](#) or [TRACERKM](#) keyword. In this way the concentration specified in the SOLUTION section is distributed between the phases whilst honoring the partitioning functions.

Injection and production

The tracer concentration in the free and solution phase in the wellbore is calculated depending on whether the well is of production or injection type. If the well is injecting, the concentration of the free phase is taken from the keyword [WTRACER](#) and the concentration of the solution phase(s) is then calculated using the relationship $C_j/C_f = K_{jf}(P)$, where P is taken to be the pressure in the wellbore. If the well is producing, the concentration in the solution and free phases are calculated by summing the concentrations in each of the well connections. In this case the connection concentrations of the free and solution phases are taken to be the grid block values.

Note: It is possible to combine the partitioned tracer functionality with the Surfactant Model, thus allowing surfactant to partition into more than one phase. This is done by specifying a tracer called ‘ SUR ’ which Eclipse recognizes as being the surfactant. For a surfactant, adsorption is calculated based on the water phase concentration, that is the ‘free’ phase.

Using the environmental tracer option

A passive tracer is set up in the normal manner using the following Eclipse 100 keywords to specify the tracer data:

Section	Eclipse 100 keyword	Description
RUNSPEC	TRACERS	Specify the number of tracers in each phase (NOTRAC, NWTRAC, NGTRAC) and the number of environmental tracers (NETRAC).
PROPS	TRACER	Specify the host phase of each tracer.
REGION	TNUM	Specify the region numbers to be used for initializing the concentration of tracer in each grid block and specify link cells to tables of initial tracer concentration with respect to depth.
SOLUTION	TBLK or TVDP	Specify the initial tracer concentration as either the numerical value or as tables of depth versus tracer concentration.
SCHEDULE	WTRACER	Inject tracer through wells.
SCHEDULE	RIVTRACE	Inject tracer in rivers.

Table 10.1: Keywords used to specify environmental tracer data

To activate the environmental tracer option, the parameter NETRAC in keyword TRACERS in the RUNSPEC section should be set greater than zero. NETRAC should be set equal to (or greater than) the number of tracers within the model that either adsorb, decay or diffuse; it need not be set equal to the total number of tracers in the model, if not all of them adsorb, decay or diffuse.

To activate the partitioned tracer option the keyword [PARTTRAC](#) must be entered in RUNSPEC section.

The adsorption, decay and diffusion data is supplied with the keywords [TRADS](#), [TRDCY](#) and [TRDIF](#) respectively. These keywords require the tracer name to be appended to the keyword to indicate which tracer the data applies to. For adsorption, the [ADSORP](#) keyword may be used, with the tracer name specified as an argument, as an alternative to TRADS. The ADSORP keyword supplies coefficients for a

generalized Langmuir isotherm with permeability dependence. For standard partitioned tracers, the data entered in TRDIF and TRDCY apply to diffusion and decay in the solution phase. For multi-partitioned tracers, data specific to each phase can be entered in TRDIF and TRDCY.

If the adsorption model is required, the rock mass density needs to be specified by using the [TRROCK](#) keyword. The grid block masses can be output by using the 'ROCKMASS' mnemonic in the [RPTSOL](#) keyword. The [TRROCK](#) keyword is also used to specify the adsorption model and the initial distribution of adsorbed tracer. For multi-partitioned tracers the concentration to be used in adsorption calculations can be specified by the sixth item of the [TRACER](#) keyword.

A passive tracer can be thought of as a dimensionless marker of the host fluid. However, in the environmental tracer case it may be convenient to ascribe a unit to the tracer. A general unit name can be given to a tracer using the third item in the [TRACER](#) keyword. Note that the adsorption data ([TRADS](#) or [ADSORP](#) keyword) should be based on this unit.

The diffusivity values can be output by setting mnemonics 'DIFFX', 'DIFFY' and 'DIFFZ' in the [RPTGRID](#) keyword. Note that the diffusivity associated with non-neighbor connections can be found in the non-neighbor connection list generated by the [RPTGRID](#) mnemonic 'ALLNNC'.

The tables of tracer adsorption functions, tracer decay and tracer rock properties can be output to the print file by setting the mnemonics 'TRADS', 'TRDCY' and 'TRROCK' in the [RPTPROPS](#) keyword.

The quantity of the tracer that has decayed or been adsorbed can be output to the print file by setting either of the mnemonics 'TRADS' or 'TRDCY' in the [RPTSCHED](#) keyword. Two families of [SUMMARY](#) keywords are available to output the adsorption and decay: FTADS, RTADS, BTADS and FTDCY, RTDCY, BTDCY. As with other tracer [SUMMARY](#) section keywords, the tracer name should be appended on to the keyword.

Tracer in place reports can be output to the print file by specifying the 'FIPTR' mnemonic in [RPTSCHED](#). For a standard partitioned tracer the amount of the tracer in the free and solution phases will be reported. For a multi-partitioned tracer the amount reported in the 'solution' phase is the sum over all solution phases.

Restrictions

At present the adsorption, decay and diffusion options are only available for single phase and partitioned tracers. It is not possible to model adsorption, decay or diffusion for tracers existing in a phase that can dissolve in another phase. For example, it is not possible to model a decaying tracer in the gas phase of a model in which the gas can dissolve in the oil.

Single phase environmental tracers employing adsorption and decay can use either the standard tracer solver method or the “cascade” or TVD algorithms, which limit numerical diffusion (for the latter two, specify 'DIFF' in item 5 of the [RUNSPEC](#) section keyword [TRACERS](#) and for the TVD algorithm use the [TRACTVD](#) keyword). However, if a tracer uses the diffusion model it cannot use the cascade algorithm. Hence, even if 'DIFF' is set in [TRACERS](#), the default solution method is employed for any tracers with the diffusion option active. [TRACTVD](#) can be used with diffusion (and dispersion), but not when LGRs are present.

All partitioned tracers are solved using the standard tracer solver.

If the adsorption option is active, the tracer equations are non-linear. Because of this non-linearity, it is likely that the cascade method is faster than the default method, providing, of course the tracer does not employ the diffusion option as well.

Environmental tracers cannot be introduced in a restart run.

If a cell is defined with zero pore volume in the [GRID](#) section, then making it active by changing the pore volume in the [EDIT](#) section using [PORV](#), is not allowed with the environmental tracer option.

Summary of keywords

RUNSPEC section

Keyword	Description
TRACERS	Specify dimensions and options for tracers.
EQLDIMS	Specify dimensions for tracer concentration versus depth tables used for equilibration.
PARTTRAC	Enables and dimensions the partitioned tracer option.

PROPS section

Keyword	Description
RPTPROPS	Controls output from the PROPS section: TRACER outputs passive tracer names. TRADSxxx, TRROCK and TRDCYxxx output environmental tracer properties.
TRACER	Define the tracers.
TRACERKP	Define the partitioning functions K(P) for partitioned tracers (max two phases).
TRACERKM	Define the partitioning functions K(P) for partitioned tracers (any no. of phases).
TRADS	Specify adsorption data (tabulated isotherm).
ADSORP	Specify adsorption data (coefficients for analytical isotherm).
TRDCY	Specify decay data.
TRDIF	Specify diffusion data.
TRROCK	Specify rock data for adsorption.

REGIONS section

Keyword	Description
RPTREGS	Controls output from the REGIONS section: TNUM outputs tracer versus depth region numbers.
TNUM	Specify tracer concentration regions.
TRKPFxxx	Specify partitioning function regions for tracer xxx.

SOLUTION section

Keyword	Description
RPTSOL	Controls output from the SOLUTION section: FIPTR outputs tracer fluid in place reports ROCKMASS outputs rock mass TBLK outputs initial grid block tracer concentrations TVDP outputs initial tracer concentration versus depth tables.
TBLK	Specify initial tracer concentrations.
TVDP	Specify initial tracer concentration versus depth.

SUMMARY section

Refer to [Environmental tracer SUMMARY output controls](#) in the *Eclipse Reference Manual*.

SCHEDULE section

Keyword	Description
RPTSCHED	Controls output from the SCHEDULE section: FIPTR outputs tracer fluid in place reports TBLK outputs initial grid block tracer concentrations TRADS or TRDCY output tracer adsorption and decay.
WTRACER	Set tracer concentration in injection wells.
RIVTRACE	Set tracer concentration for injection into rivers.

Example problem

The tracer names are:

BBB is the water tracer with no decay or adsorption

XXX is the water tracer that adsorbs and decays

YYY is the water tracer that only decays

ZZZ is the water tracer that diffuses

RUNSPEC

```

RUNSPEC      =====
TITLE
  ENVIRONMENTAL TRACER OPTION TEST

DIMENS
  100      3      1  /

NONNC

OIL

WATER

METRIC

TRACERS
  0      4      0      2 'NODIFF' /

TABDIMS
  2      1      20      20      6      20 /

REGDIMS
  6      1      0      0      0      1 /

WELLDIMS
  2      3      1      2 /

START
  1 'MAY' 1990  /

```

GRID

GRID section

```

GRID      =====
EQUALS
  'DX'      10  /
  'DY'      100 /
  'DZ'      10  /

```

```

'PERMX' 1000 /
'PERMY' 1000 /
'PORO' 0.25 /
'TOPS' 2000 1 10 /
'TOPS' 2005 11 20 /
'TOPS' 2010 21 30 /
'TOPS' 2015 31 40 /
'TOPS' 2020 41 50 /
'TOPS' 2025 51 60 /
'TOPS' 2030 61 70 /
'TOPS' 2035 71 80 /
'TOPS' 2040 81 90 /
'TOPS' 2055 91 100 /

--
-- Zero permeability for tracer ZZZ
--
'PERMX' 0.0 41 50 2 2 1 1 /
'PERMY' 0.0 41 50 2 2 1 1 /
/

```

PROPS

```

PROPS =====
SWFN
.145 .0000 .75
.220 .0001 .05
.270 .0004 1*
.320 .009 1*
.365 .018 1*
.438 .043 1*
.510 .082 1*
.558 .118 1*
.631 .187 -.05
.703 .270 -.25
.752 .310 -.50
.795 .370 -1.50
/
0.0 0.0 0.0
1.0 1.0 0.0
/

SOF2
.205 .000
.250 .00006
.300 .0009
.370 .008
.440 .027
.490 .052
.560 .120
.635 .228
.683 .334
.730 .470
.780 .644
.855 1.00
/
0.0 0.0
1.0 1.0
/

PVTW
270 1.030 4.6E-5 0.34 0.0 /

PVDO
200 1.0 0.47
280 0.999 0.47
300 0.998 0.47
/

ROCK
270 .3E-5 /

DENSITY

```



```

850. 1000. 10. /

--
-- Define the tracers
--
TRACER
'BBB' 'WAT' /
'XXX' 'WAT' /
'YYY' 'WAT' /
'ZZZ' 'WAT' /
/

--
-- Tracer adsorption data for tracer XXX
--
TRADSXXX
0.0      0.0000
1.0      0.0005
30.0     0.0005 /
0.0      0.0000
1.0      0.0005
30.0     0.0005 /

--
-- Tracer decay data for tracer XXX
--
TRDCYXXX
10.0 /

--
-- Tracer decay data for tracer YYY
--
TRDCYYYY
15.0 /

--
-- Tracer diffusion data for tracer ZZZ
--
TRDIFZZZ
0.3 /

--
-- Rock data required for tracer adsorption
--
TRROCK
1 2650 /
2 2650 /

--
-- Output of Environmental tracer data
--
RPTPROPS
'TRROCK' /

```

REGIONS

```

REGIONS =====
FIPNUM
3*1 2 3 4 94*5
3*1 2 3 4 34*5 10*6 50*5
3*1 2 3 4 94*5 /

EQUALS
'SATNUM' 1 /
/

RPTREGS
/

```

SOLUTION

```

SOLUTION =====
EQUIL
  2600 270 2025 /

TBLKFBBB
  300*0.0 /

TBLKFXXX
  300*0.0 /

TBLKFYYY
  300*0.0 /

TBLKFZZZ
  100*0.0
    40*0.0 10*1.0 50*0.0
  100*0.0 /

RPTSOL
  'PRES' 'SOIL' 'SWAT' 'RESTART=2' 'FIP=2'
  'OILAPI' 'FIPTR=2' 'TBLK' 'FIPPLY=2'
  'SURFBLK' 'FIPSURF=2' 'TRADS' /

```

SUMMARY

```

SUMMARY =====
WBHP
/

FWIR
FOPR
FOPT

FTPRBBB
FTPTBBB
FTIRBBB
FTITBBB
FTICBBB

BTCNFB BB
  1 1 1 /
/

FTPRXXX
FTPTXXX
FTIRXXX
FTITXXX
FTIPTXXX

BTCNFXXX
  1 1 1 /
/

FTADSXXX

RTADSXXX
  2 /

BTADSXXX
  2 1 1 /
/

FTPRYYY
FTPTYYY
FTIRYYY
FTITYYY

--
-- Plot log(FTIPTYYY) :vs: time to see the pure decay.
--

```

```

FTIPTYYY
BTCNFYYY
 1 1 1 /
/

FTADSYYY
FTDCYYYY

RTADSYYY
 2 /

BTADSYYY
 2 1 1 /
/

BTDCYYYY
 2 1 1 /
/

BOSAT
 1 1 1 /
/

WTPRBBB
'OP' /

WTPRXXX
'OP' /

WTPRYYY
'OP' /

WTPRZZZ
'OP' /

RUNSUM

```

SCHEDULE

```

SCHEDULE =====
RPTSCHED
'PRES' 'SOIL' 'SWAT' 'RESTART=2' 'FIP=2'
'WELLS=2' 'SUMMARY=2' 'CPU=2' 'NEWTON=2'
'OILAPI' 'FIPTR=2' 'TBLK' 'FIPSALT=2'
'TUNING' 'SURFBLK' 'SURFADS' 'FIPSURF=2'
'TRADS' /

WELSPEDS
'OP' 'G' 100 2 2600 'OIL' /
'INJ' 'G' 1 2 2600 'WAT' /
/

COMPDAT
'OP' ' ' 2* 1 1 'OPEN' 0 .0 157E-3 /
'INJ' ' ' 2* 1 1 'OPEN' 0 .0 157E-3 /
/

WCONPROD
'OP' 'OPEN' 'RESV' 4* 250 0.0 4* /
/

WCONINJE
'INJ' 'WAT' 'OPEN' 'RESV' 1* 250 /
/

--
-- Inject tracer for 40 days.
--
WTRACER
'INJ' 'BBB' 30.0 /
/

```

```
WTRACER
  'INJ' 'XXX' 30.0 /
/

WTRACER
  'INJ' 'YYY' 30.0 /
/

TSTEP
  1 9 30

--
-- End tracer injection
--

WTRACER
  'INJ' 'BBB' 0.0 /
/

WTRACER
  'INJ' 'XXX' 0.0 /
/

WTRACER
  'INJ' 'YYY' 0.0 /
/

TSTEP
  1 9 90 9*100 /

END
```

Diffusion

x	Eclipse 100
x	Eclipse 300

Molecular diffusion can be modeled in both Eclipse 100 and Eclipse 300. This chapter describes the keywords that can be used; the diffusion coefficients; and the diffusivity calculations.

Diffusion coefficients: Eclipse 100

Eclipse 100 allows diffusion of both gas and oil components within both the oil and gas phases. For example, in a gas condensate model with no oil phase present, gas diffuses from a low R_v region to a high R_v region, and vaporized oil flows in the opposite direction from the high R_v to the low R_v region (R_v is the vaporized oil-gas ratio). Consider a gas component in the gas phase. The moles of gas and oil within the gas phase are given by:

$$M_{gas} = \frac{S_g \cdot \rho_g}{B_g \cdot MW_g} \quad \text{Eq. 10.126}$$

$$M_{oil} = \frac{R_v \cdot S_g \cdot \rho_o}{B_g \cdot MW_o} \quad \text{Eq. 10.127}$$

hence the gas total molar concentration c is

$$c_g = \frac{S_g}{B_g} \left(\frac{\rho_g}{MW_g} + \frac{R_v \cdot \rho_o}{MW_o} \right) = \frac{S_g \cdot \rho_g}{B_g \cdot MW_g} (1 + R_v F) \quad \text{Eq. 10.128}$$

The gas mole fraction is

$$x_g = \frac{\rho_g / MW_g}{\rho_g / MW_g + R_v \rho_o / MW_o} = \frac{1}{1 + R_v F} \quad \text{Eq. 10.129}$$

with

$$F = \frac{\rho_o \cdot MW_g}{\rho_g \cdot MW_o} \quad \text{Eq. 10.130}$$

where

S_g is the gas saturation

B_g is the gas formation volume factor

ρ_g is the surface density of gas

ρ_o is the surface density of oil

MW_g is the molecular weight of gas

MW_o is the molecular weight of oil

R_v is the vaporized oil-gas ratio

The diffusive flow of gas at surface conditions F_{gg} is given by:

$$F_{gg} = \frac{MW_g \cdot J_g \cdot A \cdot \varphi}{\rho_g} \quad \text{Eq. 10.131}$$

For more information see [Reid et al \(1987\)](#). The gas-in-gas diffusion coefficient D_{gg} is given by:

$$J_g = -c_g \cdot D_{gg} \cdot \frac{dx_g}{dl} \quad \text{Eq. 10.132}$$

with

$$dx_g = - \frac{F}{(1 + R_v F)^2} dR_v$$

Substituting gives the diffusive flow of gas:

$$F_{gg} = \frac{S_g}{B_g} \cdot \frac{F}{1 + R_v F} \cdot D_{gg} \cdot \frac{dR_v}{dl} \cdot A \cdot \varphi \quad \text{Eq. 10.133}$$

where

J_g is the molar flux of gas per unit area

$\frac{dx_g}{dl}$ is the mole fraction gradient of gas

A is the area of flow

φ is the porosity

In the simulator the diffusive flow of gas from cell i to j is:

$$F_{gg} = \frac{S_g}{B_g} \cdot \frac{F}{(1 + R_v F)} \times D_{gg} \times \frac{(R_{vj} - R_{vi})}{d} \times A \times \varphi$$

or

$$F_{gg} = - \frac{S_g}{B_g} \cdot \frac{F}{1 + R_v F} \cdot D_{gg} \cdot (R_{vi} - R_{vj}) \cdot T_d \quad \text{Eq. 10.134}$$

where

T_d is the diffusivity = $A\varphi/d$

d is the distance between cell centers

R_{vi} is the vapor oil-gas ratio for cell i

R_{vj} is the vapor oil-gas ratio for cell j

S_g is taken as $\text{MIN}(S_{gi}, S_{gj})$

B_g is taken as $(B_{gi} + B_{gj})/2$

R_v is taken as $(R_{vi} + R_{vj})/2$

For the oil-in-gas diffusive flow relationship F_{og} :

$$F_{og} = \frac{MW_o \cdot J_o \cdot A \cdot \varphi}{\rho_o}$$

and the oil-in-gas diffusion coefficient D_{og} is given by:

$$J_o = -c_g \cdot D_{og} \cdot \frac{dx_o}{dl}$$

$$c_g = \frac{S_g}{B_g} \cdot \frac{\rho_g}{MW_g} (1 + R_v F)$$

$$x_o = \frac{R_v F}{1 + R_v F}$$

$$dx_o = \frac{F}{(1 + R_v F)^2} dR_v$$

$$F_{og} = - \frac{S_g}{B_g} \cdot \frac{1}{(1 + R_v F)} \times D_{og} \times \frac{(R_{vj} - R_{vi})}{d} \times A \times \varphi$$

$$F_{og} = \frac{S_g}{B_g} \times \frac{1}{1 + R_v F} \times D_{og} \times (R_{vi} - R_{vj}) \times T_d$$

where

J_o is the molar flux of oil per unit area

$\frac{dx_o}{dl}$ is the mole fraction gradient of oil

For the oil-in-oil diffusive flow relationship F_{oo} :

$$F_{oo} = \frac{MW_o \cdot J_o \cdot A \cdot \varphi}{\rho_o}$$

and the oil-in-oil diffusion coefficient D_{oo} is given by:

$$J_o = -c \cdot D_{oo} \cdot \frac{dx_o}{dl}$$

$$c = \frac{S_o}{B_o} \cdot \frac{\rho_o}{MW_o} \left(\frac{R_s + F}{F} \right)$$

$$x_o = \frac{F}{F + R_s}$$

$$dx_o = - \frac{F}{(F + R_s)^2} dR_s$$

$$F_{oo} = \frac{S_o}{B_o} \cdot \frac{1}{(F + R_s)} \times D_{oo} \times \frac{(R_{sj} - R_{si})}{d} \times A \times \varphi$$

$$F_{oo} = - \frac{S_o}{B_o} \times \frac{1}{(F + R_s)} \times D_{oo} \times (R_{si} - R_{sj}) \times T_d$$

For the gas-in-oil diffusive flow relationship F_{go} :

$$F_{go} = \frac{MW_g \cdot J_g \cdot A \cdot \varphi}{\rho_g}$$

and the gas-in-oil diffusion coefficient D_{go} is given by:

$$J_g = -c_o \cdot D_{go} \cdot \frac{dx_g}{dl}$$

$$c_o = \frac{S_o}{B_o} \cdot \frac{\rho_o}{MW_o} \left(\frac{F + R_s}{F} \right)$$

$$x_g = \frac{R_s}{F + R_s}$$

$$dx_g = \frac{F}{(F + R_s)^2} dR_s$$

$$F_{go} = - \frac{S_o}{B_o} \cdot \frac{F}{(F + R_s)} \times D_{go} \times \frac{(R_{sj} - R_{si})}{d} \times A \times \varphi$$

$$F_{go} = \frac{S_o}{B_o} \times \frac{F}{(F + R_s)} \times D_{go} \times (R_{si} - R_{sj}) \times T_d$$

"Diffusivity calculations" describes the calculation of the diffusivities in more detail.

Diffusion coefficients: Eclipse 300

$$J_i = -cD_i \frac{\partial x_i}{\partial d} \quad \text{Eq. 10.135}$$

In the second model diffusion is driven by the gradient of the chemical potential:

$$J_i = -cD_i^a x_i \frac{1}{RT} \frac{\partial}{\partial d} [\mu_i - M_i g(h - h_o) + M_i D_i^T \ln(T)] \quad \text{Eq. 10.136}$$

where:

J_i is the molar flux of component i per unit area per unit time,

c is the total molar concentration given by $c = 1/v_m$,

v_m is the molar volume of the mixture,

D_i is the diffusion coefficient of component i ,

D_i^a is the activity-corrected diffusion coefficient of component i ,

D_i^T is the thermal diffusion coefficient of component i ,

x_i is the mole fraction of component i ,

$\frac{\partial}{\partial d}$ is the gradient in the direction of flow,

M_i is the molecular weight of component i ,

g is the acceleration due to gravity,

h is the height,

h_0 is the reference height,

T is the temperature,

R is the chemical potential of component i , given by as constant,

μ_i

$$\mu_i = \mu_{i0} + RT \ln(f_i) \quad \text{Eq. 10.137}$$

where:

μ_{i0} is the reference chemical potential,

f_i is the component fugacity.

Therefore, there are two possible ways of specifying diffusion coefficients:

- Use normal diffusion coefficients D_i defined by keywords **DIFFCOIL** (**DIFFCWAT** for GASWAT or CO2STORE option) and **DIFFCGAS**.
- Use activity corrected diffusion coefficients D_i^a defined by keywords **DIFFAOIL** and **DIFFAGAS**.
Thermal diffusion coefficients D_i^T can optionally be defined by keywords **DIFFTOIL** and **DIFFTGAS**.

Relationship between the two diffusion models

Equation 10.135 is written in terms of concentrations, which are not ideal for diffusion in liquids and gases at high pressure. The true equilibrium is obtained when the component chemical potential is equal to the gravity potential, which are the first two terms in equation 10.136. The third term is a non-equilibrium term and represents diffusion driven by a temperature gradient.

Relationship between the diffusion coefficients

For horizontal flow in isothermal systems, equation 10.136 can be written as:

$$J_i = -cD_i^a x_i \frac{\partial}{\partial d} \ln(f_i) \quad \text{Eq. 10.138}$$

Where $\partial \ln(f_i) / \partial d$ can be written as:

$$\frac{\partial}{\partial d} \ln(f_i) = \frac{\partial}{\partial \ln(x_i)} \ln(f_i) \cdot \frac{\partial}{\partial x_i} \ln(x_i) \cdot \frac{\partial x_i}{\partial d} = \frac{\partial}{\partial \ln(x_i)} \ln(f_i) \cdot \frac{1}{x_i} \cdot \frac{\partial x_i}{\partial d} \quad \text{Eq. 10.139}$$

ignoring the terms $\partial \ln(f_i) / \partial x_j$, equation 10.138 can then be written as:

$$J_i = - \left(c \cdot D_i^a \cdot \frac{\partial}{\partial \ln(x_i)} \ln(f_i) \cdot \frac{\partial x_i}{\partial d} \right) \quad \text{Eq. 10.140}$$

Comparing equations 10.135 and 10.140, D_i^a is given by:

$$D_i^a = \frac{D_i}{\partial \ln(f_i) / \partial \ln(x_i)} \quad \text{Eq. 10.141}$$

D_i^a is therefore the activity-corrected diffusion coefficient of Reid *et al* (1987).

Equivalence at low pressures

At low pressures, the two coefficients are equal, as $f_i \rightarrow x_i P$, and the predicted flows become the same, as $\partial \ln(f_i) / \partial d \rightarrow \partial \ln(x_i) / \partial d \rightarrow (1/x_i) \cdot (\partial x_i / \partial d)$.

Component flow

In a porous medium, with both oil and gas present, the molar concentration includes the saturation, S and porosity, ϕ , so that

$$J_i = J_{io} + J_{ig} \quad \text{Eq. 10.142}$$

with J_{io} , the oil phase molar flux of component i given in terms of activity-corrected coefficients by:

$$J_{io} = - \left(\phi \cdot S_o \cdot b_o^m \cdot D_{io}^a \cdot x_i \cdot \frac{1}{RT} \cdot \frac{\partial}{\partial d} (RT \ln f_i - M_i G h + M_i D_{io}^T \ln T) \right) \quad \text{Eq. 10.143}$$

where b_o^m is the molar density of oil, or expressed in terms of normal diffusion coefficients:

$$J_{io} = - \left(\phi \cdot S_o \cdot b_o^m \cdot D_{io} \cdot \frac{\partial x_i}{\partial d} \right) \quad \text{Eq. 10.144}$$

Similar formulae apply for J_{ig} , the gas phase molar flux of component i .

Note that separate oil and gas diffusion coefficients may be defined, D_{io} and D_{ig} .

For any two cells, the diffusive flow is proportional to the cross-sectional area between the cells, and inversely proportional to the distance between them. Integrating the flux and approximating the gradients between two grid blocks using differences, the diffusive flow between blocks is:

$$F_i^{diff} = F_{io}^{diff} + F_{ig}^{diff} \quad \text{Eq. 10.145}$$

where:

$$F_{io}^{diff} = T_D D_{io}^a \left(\frac{x_i S_o b_o^m}{RT} \right)_u \Delta (RT \ln f_i - M_i G h + M_i D_{io}^T \ln T) \quad \text{Eq. 10.146}$$

$$F_{ig}^{diff} = T_D D_{ig}^a \left(\frac{y_i S_g b_o^m}{RT} \right)_u \Delta (RT \ln f_i - M_i G h + M_i D_{io}^T \ln T) \quad \text{Eq. 10.147}$$

where (x_i, y_i) are the liquid and vapor mole fractions of component i , respectively.

In terms of normal diffusion coefficients:

$$F_{io}^{diff} = T_D D_{io} (S_o b_o^m)_u \Delta x_i \quad \text{Eq. 10.148}$$

$$F_{ig}^{diff} = T_D D_{ig} (S_g b_g^m)_u \Delta y_i \quad \text{Eq. 10.149}$$

Combinations such as $(x_i S_o b_o^m)_u$ are defined on the cell interface, and treated using upstreaming in the program, indicated by the subscript ‘u’.

In this simple case, T_D is given by:

$$T_D = \frac{\phi A}{d} \quad \text{Eq. 10.150}$$

where

A is the cross-sectional area

ϕ is the porosity

d is the distance between cell centers.

T_D is a diffusive analogue of transmissibility, called the *diffusivity*, with porosities replacing permeabilities.

In "Diffusivity calculations", the diffusivities for more complex cases are considered.

Thermal diffusion and hydrostatic equilibrium calculations

The term “thermal diffusion” is used here to describe the diffusion of components caused by a temperature gradient, and should not be confused with diffusion in a thermal simulation. The thermal diffusion term is usually small, but has been included for simulating the initial conditions in a reservoir where there is a temperature gradient. In the absence of any wells and aquifers, a simulation run reaches a steady state when there is no diffusive flow. From equation 10.136, the diffusive flow will be zero when

$$\frac{\partial}{\partial d} [\mu_i - M_i g h + M_i D_i^T \ln(T)] = 0 \quad \text{Eq. 10.151}$$

for each phase. The second term in this equation drives the heavy components to the bottom of the reservoir. The third term drives those components with a low enthalpy / high entropy to the hottest parts of the reservoir. When this last term is included the reservoir will not be in thermodynamic equilibrium (since energy is flowing) but it is in hydrostatic equilibrium. The reservoir temperature can be set with either the `TEMPVD` keyword or the `TEMPI` keyword.

The steady state solution of equation 10.151 is independent of the diffusion coefficients D_i^a . However, these coefficients control the time it takes the reservoir to reach a steady state. Large timesteps can be used if the simulation is run using the full implicit method `FULLIMP`.

The cross phase diffusion option (Eclipse 300)

As well as diffusion of oil and gas components within the oil and gas phases, diffusion of components between the phases may be important.

In Eclipse 300 two models are available:

- The activity driven diffusion. The diffusion between cells are driven by the chemical potential, thus resulting in diffusion over phase boundaries. The diffusion coefficients for the cross phase diffusion are set by combining the diffusion coefficients given by `DIFFAOIL` and `DIFFAGAS` so

$Dog_i^a = \sqrt{D_{oi}^a \times D_{gi}^a}$. The activity driven diffusion option is not compatible with the [CO2STORE](#) option

- The molecular driven diffusion. The cross phase diffusion coefficients are set by the keywords [DIFFCOG](#) and [DIFFCGO](#). The fluids are mixed on the border of each cell and the resulting composition is used to set up the potential for molecular diffusion of each component. This model can be used by EOS runs and the [CO2STORE](#) option. For [GASWAT](#) or the [CO2STORE](#) option the water-gas and gas-water diffusion coefficients are set by the keywords [DIFFCWG](#) and [DIFFCGW](#). The molecular diffusion option will construct a border composition by mixing the composition of neighbor cells. This means that the option generally will require more computational time than the activity driven diffusion option.

Note: The activity driven diffusion option can be used with the [GASWAT](#) option.

The molecular cross-phase diffusion option

As an example, consider a cell I containing oil and a cell J containing gas. An artificial border composition is constructed by $\tilde{m}_i = (bo \cdot x_i)_I + (bg \cdot y_i)_J$

where

bo is the oil molar density

x_i is the oil mole fraction of component i,

bg is the gas molar density

y_i is the gas mole fraction of component i

The border pressure and temperature is set to $\tilde{P} = \frac{P_I + P_J}{2}$, $\tilde{T} = \frac{T_I + T_J}{2}$

The corresponding oil and gas mole fractions and densities of the artificial border cell is denoted $\tilde{x}_i$, $\tilde{y}_i$, $\tilde{b}_o$ and $\tilde{b}_g$. The cross phase diffusive flow between the oil in cell I and the gas in cell J for component i is given by $F_{cross} = FX_{og} + FX_{go} + FY_{og} + FY_{go}$

$$FX_{og} = Diff \times Dog_i \times \min(So_I, Sg_J) \times bo \times \max(x_i - \tilde{x}_i, 0)$$

$$FX_{go} = Diff \times Dgo_i \times \min(So_I, Sg_J) \times \tilde{b}_o \times \min(x_i - \tilde{x}_i, 0)$$

$$FY_{og} = Diff \times Dog_i \times \min(So_I, Sg_J) \times \tilde{b}_g \times \max(\tilde{y}_i - y_i, 0)$$

$$FY_{go} = Diff \times Dgo_i \times \min(So_I, Sg_J) \times bg \times \min(\tilde{y}_i - y_i, 0) \text{ where}$$

So_I and Sg_J is the oil and gas saturation in cell I and J,

$Diff$ is the diffusivity

Dog_i , Dgo_i are the cross phase oil-gas and gas-oil diffusion coefficients.

It is possible to activate the molecular cross phase diffusion option without also specifying phase specific diffusion coefficients.

The cross phase diffusion option (Eclipse 100)

As well as diffusion of oil and gas components within the oil and gas phases, diffusion of components directly from the gas phase to the oil phase may be important.

For example, if in a dual porosity system the fracture contains gas and the matrix mainly oil, then gas is expected to diffuse from the fracture to the matrix and oil from the matrix to the fracture. This process can be modeled using the cross phase diffusion option.

The cross phase diffusion process is assumed to be limited by the slower diffusion within the oil. Hence the diffusive flow takes the form of the gas-in-oil and oil-in-oil diffusion:

$$\begin{aligned} F_{gx} &= \frac{S_c}{B_o} \cdot \frac{F}{(F + R_{sj})} \cdot D_{gx} \cdot (R'_{si} - R_{sj}) \cdot T_d \\ F_{ox} &= \frac{S_c}{B_o} \cdot \frac{1}{(F + R_{sj})} \cdot D_{ox} \cdot (R'_{si} - R_{sj}) \cdot T_d \end{aligned} \quad \text{Eq. 10.152}$$

where

cell j contains mainly oil

cell i contains mainly gas

R'_{si} is the saturated R_s corresponding to the pressure of cell i

R_{sj} is the R_s of cell j

S_c is the contact saturation $\text{MIN}(S_{oi} + S_{gi}, S_{oj} + S_{gj}) - \text{MIN}(S_{oi}, S_{oj}) - \text{MIN}(S_{gi}, S_{gj})$

D_{gx} is the gas diffusion coefficient

and D_{ox} is the oil diffusion coefficient.

Diffusivity calculations

The calculation of the diffusivity is analogous to the calculation of transmissibility, with the permeability replaced by the porosity.

Eclipse 100

The method of calculating the diffusivity varies depending on the type of model, and can be modified in line with the transmissibility calculation by using the [OLDTRAN](#) or [NEWTRAN](#) keyword.

Block center diffusivity calculations

In this case, X and Y direction diffusivities values are obtained using cell center separations and cross sectional areas obtained from D_x , D_y and D_z , with a dip correction.

Eclipse 100

This type of calculation is also referred to as [OLDTRAN](#).

The expressions for the Cartesian case are given below:

X-diffusivity

$$T_{D_{xI}} = \frac{M_{D_{xI}} A_{F_{dip}}}{B} \quad \text{Eq. 10.153}$$

where:

$T_{D_{x_I}}$ is diffusivity between cell I and cell J, its neighbor in the positive X-direction

$M_{D_{x_I}}$ represents the diffusivity multiplier for cell I

A is the interface area between cell I and J

F_{dip} represents the dip correction.

A , F_{dip} and B are given by the expressions:

$$A = \frac{(D_{x_J}D_{y_I}D_{z_I}R_{ntg_I} + D_{x_I}D_{y_J}D_{z_J}R_{ntg_J})}{(D_{x_I} + D_{x_J})} \quad \text{Eq. 10.154}$$

$$B = \frac{\left(\frac{D_{x_I}}{\phi_I} + \frac{D_{x_J}}{\phi_J}\right)}{2} \quad \text{Eq. 10.155}$$

$$F_{\text{dip}} = \frac{D_h^2}{D_h^2 + D_v^2} \quad \text{Eq. 10.156}$$

with:

$$D_h^2 = \left(\frac{D_{x_I} + D_{x_J}}{2}\right)^2 \quad \text{Eq. 10.157}$$

$$D_v^2 = (d_I - d_J)^2 \quad \text{Eq. 10.158}$$

where

d denotes depth

R_{ntg} is the net to gross ratio.

Y-diffusivity

The expression for the Y-diffusivity value is entirely analogous to the above, with the appropriate permutations of X,Y and Z.

Z-diffusivity

The expression for the Z-diffusivity value is analogous to the above, with the appropriate permutations of X, Y and Z, but differs in the following ways:

- R_{ntg} does not appear.
- There is no dip correction.

Corner point diffusivity calculations

In this case, the diffusivity values are calculated from the X-, Y- and Z-projections of the mutual interface area of the two cells. An inner product is then taken with the vector distance from the cell center to the center of the cell face, so that a dip correction is automatically incorporated.

Eclipse 100

This type of calculation is also referred to as NEWTRAN.

X-diffusivity

The X-diffusivity is given by the expression:

$$T_{D_{xI}} = \frac{M_{D_{xI}}}{\frac{1}{T_I} + \frac{1}{T_J}} \quad \text{Eq. 10.159}$$

with:

$$T_I = \frac{\phi_I R_{ntgI} A \cdot D_I}{D_I \cdot D_I} \quad \text{Eq. 10.160}$$

where

$$A \cdot D_I = A_x D_{xI} + A_y D_{yI} + A_z D_{zI} \quad \text{Eq. 10.161}$$

and

$$D_I \cdot D_I = D_{xI}^2 + D_{yI}^2 + D_{zI}^2 \quad \text{Eq. 10.162}$$

A_x , A_y and A_z are the X-, Y- and Z- projections of the mutual interface area of cell I and cell J (which need not be neighbors in the Cartesian indexing grid), and similarly D_{xI} , D_{yI} and D_{zI} are the X-, Y- and Z- components of the distance between the center of cell I and the center of the relevant face of cell I, these centers being obtained as the appropriate averages. The expression for T_J is analogous.

Y- and Z-diffusivity

The Y- and Z- diffusivity expressions are similar, the net to gross ratio being absent from the Z expression.

Radial diffusivity calculations

In this case, the expressions used are based upon true radial diffusive flow between the pressure equivalent radii, with a dip correction.

Radial diffusivity

The radial diffusivity is:

$$T_{D_{rI}} = \frac{M_{D_{rI}} F_{\text{dip}}}{\frac{1}{T_I} + \frac{1}{T_J}} \quad \text{Eq. 10.163}$$

with:

$$T_I = \frac{\phi_I R_{ntgI} D_{\theta I} D_{zI}}{D_{1p}} \quad \text{Eq. 10.164}$$

$$D_{1p} = \frac{R_1^2}{(R_2^2 - R_1^2)} \ln(R_1/R_2) + \frac{1}{2} \quad \text{Eq. 10.165}$$

and

$$T_J = \frac{\phi_J R_{ntgJ} D_{\theta J} D_{zJ}}{D_{2m}} \quad \text{Eq. 10.166}$$

$$D_{2m} = \frac{R_3^2}{(R_3^2 - R_2^2)} \ln\left(\frac{R_3}{R_2}\right) - \frac{1}{2} \quad \text{Eq. 10.167}$$

where R_1 is the inner radius of cell I, R_2 the outer radius of cell I, and R_3 the outer radius of cell J.

Azimuthal diffusivities

The theta-direction diffusivity is given by:

$$T_{D\theta I} = \frac{M_{D\theta I} F_{\text{dip}}}{\frac{1}{T_I} + \frac{1}{T_J}}$$

with

$$T_I = \frac{2\phi_I R_{ntgI} D_{zI} \ln\left(\frac{R_2}{R_1}\right)}{D_{\theta I}}$$

where R_1 is the inner radius, and R_2 is the outer radius. The expression for T_J is defined similarly.

$\ln()$ implies the natural logarithm throughout. This expression is also used in completing the circle in radial geometry.

Vertical diffusivities

The Z-direction diffusivity is given by:

$$T_{DzI} = \frac{M_{DzI}}{\frac{1}{T_I} + \frac{1}{T_J}}$$

with

$$T_I = \frac{\phi_I D_{\theta I} (R_2^2 - R_1^2)}{D_{zI}}$$

where R_1 is the inner radius, and R_2 is the outer radius. The expression for T_J is defined similarly.

Matrix-fracture diffusivity calculations

The matrix-fracture coupling diffusivities are proportional to the overall cell volume and to the matrix cell bulk volume:

$$T_D = \text{DIFFMMF} \cdot \varphi_m \cdot V \cdot \sigma \quad \text{Eq. 10.168}$$

where

φ_m is the matrix cell porosity

V is the matrix cell bulk volume

σ is a factor to account for the matrix-fracture interface area per unit bulk volume.

Kazemi ([Kazemi et al, 1976](#)) has proposed the following form for σ :

$$\sigma = 4 \left(\frac{1}{l_x^2} + \frac{1}{l_y^2} + \frac{1}{l_z^2} \right) \quad \text{Eq. 10.169}$$

where l_x , l_y and l_z are typical X,Y and Z dimensions of the blocks of material making up the matrix volume.

In the above equation, l_x , l_y and l_z are thus not related to the simulation grid dimensions. Alternatively, as σ acts as a multiplier on the matrix-fracture coupling, it may simply be treated as a history matching parameter.

σ can be specified as a single value for the whole field, or on a cell by cell basis using the keyword [SIGMAV](#). If σ is defined on a cell by cell basis, the values corresponding to the first $\text{NDIVIZ} / 2$ layers are used.

Using the molecular diffusion option

The Molecular Diffusion option is initiated by the keyword [DIFFUSE](#) in the RUNSPEC section.

The diffusivity will be calculated automatically as described above. The default diffusivities can be modified in two ways:

- by multiplying the calculated value by an input number, or
- by inputting the diffusivities explicitly.

The multipliers are input using the [DIFFMX](#), [DIFFMY](#), [DIFFMZ](#) keywords in much the same way as [MULTX](#), [MULTY](#), [MULTZ](#) in the GRID section; and the diffusivities can be overwritten in the EDIT section using [DIFFX](#), [DIFFY](#), [DIFFZ](#) analogously to the [TRANX](#), [TRANX](#), [TRANZ](#) keywords. The diffusivities associated with the NNCs representing the matrix-to-fracture flows in dual porosity runs can be modified using the [DIFFMMF](#) keyword.

Eclipse 100

In addition, inter-region diffusivity multipliers may be specified using [MULTREGD](#) and [MULTNUM](#) in the GRID section.

Eclipse 100

The diffusion coefficients and average phase molecular weights have to be supplied with the [DIFFC](#) keyword in the PROPS section. Separate gas-in-oil and oil-in-oil diffusion coefficients need to be specified for the cross phase diffusion model. If these are not supplied, no cross phase diffuse will occur.

Eclipse 300

Diffusion coefficients can be entered using [DIFFCOIL](#), [DIFFCGAS](#), [DIFFCWAT](#), [DIFFCOG](#), [DIFFCGO](#), [DIFFCWG](#), [DIFFCGW](#) or [DIFFAOIL](#) and [DIFFAGAS](#).

Eclipse 100

In dual porosity runs, the fracture-to-fracture diffusion calculation can be turned off using the [DIFFDP](#) keyword.

Summary of keywords

RUNSPEC

- [DIFFUSE](#) Enables molecular diffusion

GRID

Restriction	Keyword	Description
	DIFFMMF	Matrix-fracture diffusivity multipliers
	DIFFMR	Diffusivity multipliers in the radial direction
Eclipse 100	DIFFMR-	Diffusivity multipliers in the negative radial direction.
	DIFFMTHT	Diffusivity multipliers in the theta direction
Eclipse 100	DIFFMTH-	Diffusivity multipliers in the negative theta direction.
	DIFFMX	Diffusivity multipliers in the X-direction
Eclipse 100	DIFFMX-	Diffusivity multipliers in the negative X-direction.
	DIFFMY	Diffusivity multipliers in the Y-direction
Eclipse 100	DIFFMY-	Diffusivity multipliers in the negative Y-direction.
	DIFFMZ	Diffusivity multipliers in the Z-direction
Eclipse 100	DIFFMZ-	Diffusivity multipliers in the negative Z-direction.
Eclipse 100	MULTREGD	Multiplies the diffusivity between flux or MULTNUM regions.

EDIT

Restriction	Keyword	Description
	DIFFR	Diffusivities in the radial direction
	DIFFTHT	Diffusivities in the theta direction
	DIFFX	Diffusivities in the X-direction
	DIFFY	Diffusivities in the Y-direction
	DIFFZ	Diffusivities in the Z-direction
Eclipse 100	MULTREGD	Multiplies the diffusivity between flux or MULTNUM regions.

PROPS

Restriction	Keyword	Description
Eclipse 300	DIFFAGAS	Activity corrected gas diffusion coefficients.
Eclipse 300	DIFFAOIL	Activity corrected oil diffusion coefficients
Eclipse 100	DIFFC	Molecular diffusion data for each PVT region.
Eclipse 300	DIFFCGAS	Gas diffusion coefficients.
Eclipse 300	DIFFCOIL	Oil diffusion coefficients.

Restriction	Keyword	Description
Eclipse 300	DIFFCWAT	Water diffusion coefficients (GASWAT or CO2STORE option).
Eclipse 300	DIFFCWG	Water-Gas cross phase diffusion coefficients (GASWAT or CO2STORE option.
Eclipse 300	DIFFCGW	Water-Gas cross phase diffusion coefficients (GASWAT or CO2STORE option.
Eclipse 300	DIFFCOG	Oil-Gas cross phase diffusion coefficients.
Eclipse 300	DIFFCGO	Gas-Oil cross phase diffusion coefficients.
Eclipse 100	DIFFDP	Restricts molecular diffusion in dual porosity runs.
Eclipse 300	DIFFTGAS	Thermal diffusion coefficients for gas.
Eclipse 300	DIFFTOIL	Thermal diffusion coefficients for oil.

SCHEDULE

- [DIFFMMF](#) Matrix-fracture diffusivity multipliers

Non-Darcy flow

x	Eclipse 100
x	Eclipse 300

The non-Darcy feature described here is the Forchheimer correction which takes into account the inertia effects due to high velocity that may occur in high permeability regions, such as fractures. The Forchheimer parameter can be input by using either the [VDFLOW](#) or [VDFLOWR](#) keywords in the PROPS section. For a homogeneous reservoir the model writes (cgs units):

$$\frac{\partial P}{\partial x} = \left(\frac{\mu}{Kk_r A} \right) q + \beta \rho \left(\frac{q}{A} \right)^2 \quad \text{Eq. 10.170}$$

where:

q is the volumetric flow rate

K is the rock permeability

k_r is the relative permeability

A is the area through which flow occurs

μ is the dynamic fluid viscosity

ρ is the fluid density

β is the Forchheimer parameter

$\frac{\partial P}{\partial x}$ is the pressure gradient normal to the area

The Forchheimer parameter β is the user input in Forchheimer units: 1 Forch = 1 atm.s².gm⁻¹. An alternative unit for β can be obtained by noting that 1 atm = 1.01325 bars and 1 bar = 10⁶ gm.cm⁻¹.s⁻², therefore 1 Forch = 1 atm.s².gm⁻¹ = 1.01325.10⁶ cm⁻¹.

Typical values in cm⁻¹ are rather large for engineering purposes and the Forchheimer (Forch) is used in all unit sets. [Dake \(1978\)](#) shows a typical value of $\beta = 10^7 \text{ cm}^{-1} = 9.86 \text{ Forch}$ at $K = 60 \text{ mD}$.

For other unit sets (FIELD, METRIC, LAB and PVT) two constants C_1 and C_2 are introduced as

$$\frac{\partial P}{\partial x} = \frac{1}{C_1} \left(\frac{\mu}{Kk_r A} \right) q + \frac{C_2}{C_1} \beta \rho \left(\frac{q}{A} \right)^2 \quad \text{Eq. 10.171}$$

When considering the flow between two grid blocks i and n the simulator use transmissibility. Applying block-centered transmissibility for simplicity (OLDTRAN) gives:

$$q = C_1 F_{ND} T_{in} \frac{k_r}{\mu} \Delta P_{in} \quad \text{Eq. 10.172}$$

where:

$$T_{in} = \frac{A}{\frac{\Delta X_i}{2K_i} + \frac{\Delta X_n}{2K_n}} \quad \text{Eq. 10.173}$$

and where the **non-Darcy flow** factor is given by:

$$F_{ND} = \frac{1}{1 + C_2 \rho \frac{T_{in} k_r}{A^2 \mu} \frac{1}{2} (\beta_i \Delta X_i + \beta_n \Delta X_n) q} \quad \text{Eq. 10.174}$$

Note that the F_{ND} factor is dimensionless and gives a quantitative indication of the deviation of the flow behavior from Darcy's law for which $F_{ND} = 1$.

Unit conversion factors

Denoting by a star index other unit sets and taking advantage of some variables being expressed in the same units whatever the unit system (such as the viscosity and the β factor), the constants C_1 and C_2 are obtained as:

$$C_1 = \frac{\Delta P_{cgs}}{\Delta P^*} \frac{\Delta X^*}{\Delta X_{cgs}} \frac{K_{cgs}}{K^*} \frac{A_{cgs}}{A^*} \frac{q^*}{q_{cgs}} \quad \text{Eq. 10.175}$$

$$C_2 = \frac{\rho_{cgs}}{\rho^*} \frac{q_{cgs}}{q^*} \left(\frac{\Delta X_{cgs}}{\Delta X^*} \right)^2 \frac{K_{cgs}}{K^*} \left(\frac{A^*}{A_{cgs}} \right)^2 \quad \text{Eq. 10.176}$$

The following table gives the numerical values of these unit conversion factors.

	Metric	Field	Lab	PVT-Metric
C_1	0.00852702	0.00112712	3.6	0.00864
C_2	1.1574074E-9	3.17277463E-8	2.77777778E-7	1.1574074E-9

Table 10.2: Unit conversion factors C_1 and C_2

Construction of flows

From the equation for F_{ND} there are two coupled equations to solve for each phase p :

$$F_{ND} = \frac{1}{1 + C_2 \rho_p \frac{T_{in} k_{rp}}{A^2 \mu_p} \frac{1}{2} (\beta_i \Delta X_i + \beta_n \Delta X_n) q} \quad \text{Eq. 10.177}$$

and

$$q = C_1 F_{ND} T_{in} \left(\frac{k_{rp}}{\mu_p} \right) \Delta P_{pin} \quad \text{Eq. 10.178}$$

Substituting q into F_{ND} a quadratic equation is obtained for F_{ND}

$$B F_{ND}^2 + F_{ND} - 1 = 0 \quad \text{Eq. 10.179}$$

that can be solved to give (the negative root being ignored)

$$F_{ND} = \frac{-1 + \sqrt{1 + 4B}}{2B} \quad \text{Eq. 10.180}$$

where for simplicity:

$$B = C_2 C_1 \rho_p \left(\frac{T_{ni}}{A} \right)^2 \left(\frac{k_{rp}}{\mu_p} \right)^2 \frac{1}{2} (\beta_i \Delta X_i + \beta_n \Delta X_n) \Delta P_{pni} \quad \text{Eq. 10.181}$$

Assumption and simplifications

As the Forchheimer correction is expected to be significant only in regions of high velocity, it is assumed that those regions have a constant rock permeability K and constant porosity ϕ . Under these assumptions the B -factor can be simplified to:

$$B = C_2 C_1 \beta \rho_p \left(\frac{T_{ni} K}{A} \right) \left(\frac{k_{rp}}{\mu_p} \right)^2 \Delta P_{pni} \quad \text{Eq. 10.182}$$

Using the fact that in Eclipse the factor unit C_1 is included in the transmissibility ($T_{ni} = C_1 T_{ni}$), the final form for the B -factor used in Eclipse 300 is obtained:

$$B = C_2 \beta \rho_p \left(\frac{T_{ni} K}{A} \right) \left(\frac{k_{rp}}{\mu_p} \right)^2 \Delta P_{pni} \quad \text{Eq. 10.183}$$

The values of F_{ND} and $B/(\Delta P_{pni})$ for gas and oil may be examined using the mnemonics FFOR (G, O) and BFOR (G, O) respectively in the [RPTSOL](#) or [RPTSCHEG](#) keywords and also as summary vectors using the mnemonics BFFOR (G, O) and BBFOR (G, O) respectively.

Note: In the current version, Eclipse 100 reports FFORG and BFORG, that is, for the gas phase only. Summary vectors are not available.

Forchheimer models

In the above discussion the Forchheimer parameter (β) is a constant supplied by the user using the keywords [VDFLOW](#) or [VDFLOWR](#). This parameter may alternatively be calculated from one of two models implemented in Eclipse 300. The choice of model is made using the [VELDEP](#) keyword in the RUNSPEC section, with the model parameters supplied using the [VDKRG](#) and [VDKRO](#) keywords in the PROPS section.

Eclipse 300 For each phase p , the models are:

Model 1:

$$\beta_p = \frac{a_p}{\phi^{b_p} S_p^{c_p} (K k_{rvp})^{d_p}} \quad \text{Eq. 10.184}$$

Model 2:

$$\beta_p = \beta_{dp} S_p^{c_p} (K k_{rvp})^{d_p} \quad \text{Eq. 10.185}$$

where S_p is the normalized saturation of the p^{th} phase, which is related to the true saturation $S_p^{(3P)}$ by:

$$S_p = \frac{S_p^{(3P)}}{1 - S_w} \quad \text{Eq. 10.186}$$

where S_w is the water saturation.

The set of parameters (a_p, b_p, c_p, d_p) or (β_{dp}, c_p, d_p) are, **and must be**, experimentally determined. There appears to be no real physical basis for the porosity and saturation dependency in equation 10.184 and saturation dependency in equation 10.185 other than in the original work done on the Forchheimer effect. These dependencies may be ignored by setting the values of (b_p, c_p) in the [VDKRO](#) and/or [VDKRG](#) keywords in the PROPS section to zero.

The values of β for gas and oil may be examined using the mnemonics BTFOR (G, O) in the [RPTSOL](#) or [RPTSCHED](#) keywords.

Forchheimer parameter based well D-factors

If a Forchheimer parameter for the gas phase is calculated using one of the models specified via the [VELDEP](#) keyword, a D-factor for each production well connection is also calculated.

Note: For Eclipse 100, a D-factor is **not** calculated, by default, if a constant Forchheimer parameter is supplied using the keywords [VDFLOW](#) or [VDFLOWR](#). However, if [item 218](#) of the [OPTIONS](#) keyword is set to 2, the D-factor calculated via either the [VDFLOW](#) or [VDFLOWR](#) keywords will be applied to wells except those for which the [WDFAC](#) or [WDFACCOR](#) keywords have been specified or for well completions for which item 12 of the [COMPDAT](#) keyword (or item 13 of the [COMPDATL](#) keyword) has been specified.

Note: For Eclipse 300, a D-factor is **not** calculated if a constant Forchheimer parameter is supplied using the keywords [VDFLOW](#) or [VDFLOWR](#).

For Eclipse 300, D-factors calculated via the [VELDEP](#) keyword override D-factors supplied using the keywords [COMPDAT](#) or [WDFAC](#). If the [WDFACCOR](#) keyword is specified, however, then the value calculated from its correlation overrides the value calculated from equation 10.187. Calculated D-factors for well connections may be written to the summary file using the mnemonic [CDFAC](#).

The equation for the Forchheimer parameter dependent D-factor is as follows:

$$D = \alpha \cdot \beta_g \cdot \frac{K_e}{h} \cdot \frac{1}{r_w} \cdot \frac{\gamma_g}{\mu_g} \quad \text{Eq. 10.187}$$

where:

α is a constant, the value of which depends on the units system (see [table 10.3](#))

β_g is the Forchheimer parameter for gas (in Forchheimer units, Forch)

K_e is the effective permeability of the connected grid block (generally the geometric mean of permeabilities involved)

h is the length of the connection

r_w is the wellbore radius

γ_g is the relative density of produced or injected gas at surface conditions with respect to air at standard temperature and pressure

μ_g is the gas viscosity in the connection grid block

	Metric	Field	Lab	PVT-Metric
Unit	day/sm ³	day/Mscf	hr/scc	day/sm ³
Value	2.24460 x 10 ⁻¹⁰	6.83352 x 10 ⁻⁸	5.41375 x 10 ⁻¹¹	2.25533 x 10 ⁻¹⁰

	Metric	Field	Lab	PVT-Metric
Pre-2006.1 value	2.70863 x 10 ⁻¹⁸	7.67000 x 10 ⁻¹⁷	6.50072 x 10 ⁻²³	2.70863 x 10 ⁻¹⁸

Table 10.3: Unit-dependent constants for the Forchheimer parameter-dependent skin-factor

The values of the unit-dependent constant were changed in 2006.1. The pre-2006.1 values are still available for back-compatibility. To revert to the pre-2006.1 values, set [item 96](#) of `OPTIONS3` keyword to 2 or 3.

Note that equation 10.187 uses an approximation based on the assumption that the pressure equivalent radius, r_0 , of the well is much greater than the wellbore radius. This may not be a good approximation with smaller grid sizes. Equation 10.188, which does not use this approximation, can be used by setting item 96 of the `OPTIONS3` keyword to 1 or 3.

$$D = \alpha \cdot \beta_g \cdot \frac{K_e}{h} \cdot \left(\frac{1}{r_w} - \frac{1}{r_0} \right) \cdot \frac{\gamma_g}{\mu_g} \quad \text{Eq. 10.188}$$

Velocity-dependent relative permeabilities

At high gas velocities, in addition to the Forchheimer correction, it is also possible to take into account more complex effects by introducing a velocity dependence of the relative permeabilities ([Henderson et al, 2000](#)). The model may make the oil and gas relative permeabilities functions of Capillary Number (CN) according to the user-specified switches set using the `VELDEP` keyword in the `RUNSPEC` section. The properties of the velocity-dependent relative permeabilities for the oil and gas are set with the keywords `VDKRO`, `VDKRG` and `VDKRGCC`.

These models are intended for gas condensate systems where the expected reduction in Productivity Index (PI) when a well bottom hole pressure drops below the fluid dew point pressure has not been observed in practice. Several theories have been suggested as to why the gas relative permeability in particular remains relatively high but the current consensus seems to imply that a combination of low Interfacial Tensions (IFTs) at reservoir conditions and high velocities as flow converges towards a producer are the main driving forces.

The normal engineering practice is to combine IFT and velocity to form the Capillary Number; several alternative definitions are available for the Capillary Number and three of these are described below. Where flow velocities are high, turbulence effects as predicted by the Forchheimer model become significant and a model for this effect has been developed as part of this overall scheme. These models do not work with black oil runs since they are intended for gas condensate systems.

Capillary number model

The CN model has two effects on the gas and oil relative permeabilities, namely as the CN increases:

- It reduces the residual saturations
- It changes the relative permeability from the user-specified (immiscible) saturation curves towards an internally-generated miscible curve.

The CN is calculated from one of three alternate models. For phase $p = (\text{oil, gas})$

Capillary number model 1

$$N_{cp}^{(1)} = \frac{v_g \mu_g}{\sigma} \quad \text{Eq. 10.189}$$

Capillary number model 2

$$N_{cp}^{(2)} = \frac{Kk_{rvp}\Delta P_p}{\sigma L} \quad \text{Eq. 10.190}$$

Capillary number model 3

$$N_{cp}^{(3)} = (2\phi S_p Kk_{rvp})^{1/2} \frac{\Delta P_p}{\sigma} \quad \text{Eq. 10.191}$$

where:

S_p is the normalized or HCPV saturation of hydrocarbon phase p, which is related to the grid block phase saturation $S_p^{(3P)}$ according to: $S_p = \frac{S_p^{(3P)}}{1-S_w}$

ΔP_p is the pressure drop of hydrocarbon phase p in the direction of flow

μ_g is the gas viscosity

v_g is the gas velocity in the direction of flow

σ is the gas-oil surface tension

k_{rvp} is the CN-modified relative permeability of hydrocarbon phase p (more details below)

The CN-modified relative permeabilities k_{rvp} are calculated at the previous timestep. This is discussed in more detail below.

Note: Capillary number model 1 for the oil and gas phases depends only on the gas velocity and gas viscosity. This apparent discrepancy has been verified by experiment.

Given a phase the CN calculated from equations 10.189, 10.190 or 10.191, and Normalized Capillary Number (NCN) are defined by:

$$N_{cnp} = \frac{N_{cbp}}{N_{cp}} \quad \text{Eq. 10.192}$$

where N_{cbp} is the phase Base Capillary Number (BCN). The phase BCN is a lower threshold value below which the CN has no effect on the phase relative permeabilities. As with all the parameters in this and the Forchheimer models, the phase BCN should be determined experimentally for the reservoir and fluid of interest. If this data is not available, it may be estimated using the following procedure.

Base capillary number estimation

Generally, the rock or base relative permeability curves used to describe the flow of gas and oil within the reservoir will have been generated at or near ambient conditions. Ideally, the phase CNs will have been measured from these laboratory analyses and are reported to you; in practice, this is rarely the case. At ambient conditions, the reservoir gas and condensate are as different in composition as it is possible to get; therefore, the gas-oil surface tension is likely to be its maximum. In addition, the low pressure gas viscosity is likely to be at its minimum.

Using PVT i , it should be possible to perform a Constant Volume Depletion (CVD) experiment on the reservoir fluid, starting from the dew point and ending at the likely abandonment pressure. The values of the gas-oil surface tension and gas viscosity can then be taken from this final stage pressure.

Away from the wells, fluid velocities rarely exceed 10 ft./day [$\sim 3.5 \times 10^{-5}$ m/s]. This value for gas velocity along with the low pressure gas-oil surface tension and gas viscosity can be used in equation 10.189 to estimate the oil and gas BCN in the absence of experimental data.

The effect of CN on residual saturation

Experimental evidence suggests that as the CN rises above the BCN, the residual saturation is reduced. In the CN model implemented here, this effect is modeled through the use of the saturation scaling parameter, X_p , where:

$$X_p = 1 - e^{(-m_p N_{cnp})} \quad \text{Eq. 10.193}$$

where m_p is, **and must be**, an experimentally determined parameter. Therefore, given a phase residual saturation S_{rbp} determined from routine Special Core AnaLysis (SCAL), the effect of the CN dependency is to change the residual saturation to:

$$S_{rbp} \rightarrow X_p S_{rbp} \quad \text{Eq. 10.194}$$

Note if the m_p -parameter is set to zero, it has the effect of setting that phase residual saturation to zero regardless of the value of the phase CN.

The effect of CN on relative permeability curves

The miscible or straight-line relative permeability curve is constructed from:

$$k_{rmp} = \frac{\left(\frac{S_p}{(1-S_{wco})} - \frac{X_p S_{rbp}}{(1-S_{wco})} \right)}{\left(1 - \frac{X_p S_{rbp}}{(1-S_{wco})} \right)} \quad \text{Eq. 10.195}$$

where S_{wco} is the connate water saturation and S_p is the ordinary (not HCPV) phase saturation. This can be simplified to the form:

$$k_{rmp} = \frac{(S_p - X_p S_{rbp})}{((1-S_{wco}) - X_p S_{rbp})} \quad \text{Eq. 10.196}$$

The user-input base relative permeability curves, k_{rbp} , are modified because of the change in residual saturation through equation 10.196. The CN-modified relative permeability curve for phase p is now calculated from:

$$k_{rvp} = N_{cnp}^{1/n_p} k_{rbp} + \left(1 - N_{cnp}^{1/n_p} \right) k_{rmp} \quad \text{Eq. 10.197}$$

where n_p is given by:

$$n_p = n_{1p} S_p^{n_{2p}} \quad \text{Eq. 10.198}$$

and (n_{1p}, n_{2p}) are, **and must be**, experimentally determined parameters.

Explicit calculation of capillary number dependency

Equations 10.189, 10.190 and 10.191 show that the CN is a function of the phase relative permeability k_{rvp} : note equations 10.189 and 10.190 are effectively equivalent assuming the second

expression is only applied to the gas phase. However, equations 10.193, 10.195 and 10.197 show that k_{rvp} is a function of CN.

It is possible to write this dependency as a non-linear equation that can be solved in a fully implicit scheme and it is planned to do this in a future release. For now, however, the simulator calculates the phase CN and NCN at the end of the last timestep. This may have some stability implications for otherwise implicit completion cells (in Fully Implicit or AIM modes of operation). This is still an area of research.

Alternative model for capillary number effects

An alternative model for CN effects, based on the work of [Whitson et al \(1999\)](#), is also available. This model is designed for near wellbore flows exhibiting condensate ‘blockage’ type behavior in a gas condensate reservoir. It may be activated via item 5 of the [VELDEP](#) keyword in the RUNSPEC section.

This model is more straightforward than the above in that:

- There is no effect on residual saturations
- There is no lower threshold or base capillary number value. (Below which the CN has no effect on the gas relative permeabilities.)
- It depends on only two parameters, both of which have default values.

In the implementation in Eclipse 300, the CN-modified gas relative permeability, k_{rg} , is an admixture of a straight-line miscible relative permeability, k_{rgM} , and the user-input immiscible, rock relative permeability, k_{rgI} ; this admixture is controlled by a CN dependent transition function, f_I :

$$k_{rg} = f_I k_{rgI} + (1 - f_I) k_{rgM} \quad \text{Eq. 10.199}$$

The calculation of the miscible relative permeability is discussed later. The transition function depends on the gas capillary number, N_{cg} , and is given by:

$$f_I = \frac{1}{(\alpha \cdot N_{cg})^n + 1} \quad \text{Eq. 10.200}$$

where

$$\alpha = \alpha^0 / \overline{k_{rg}} \quad \text{with } \overline{k_{rg}} = \frac{k_{rgM} + k_{rgI}}{2} \quad \text{Eq. 10.201}$$

and α^0 is a constant depending only on rock properties:

$$\alpha^0 = \frac{\alpha_c^0}{\sqrt{K \cdot \phi}} \quad \text{Eq. 10.202}$$

where K is the rock permeability and ϕ is the porosity.

The gas capillary number is calculated from model 1, equation 10.189, using a pore gas velocity:

$$v_{pg} = \frac{v_g}{\phi \cdot (1 - S_w)} \quad \text{Eq. 10.203}$$

This model depends on two parameters: the exponent n in equation 10.200 and the coefficient α_c^0 in equation 10.202. These parameters are defaulted to 0.65 and 1.0E4 respectively, with the keyword [VDKRG](#) permitting you to override these values.

Calculation of the miscible gas relative permeability

In the well inflow generalized pseudo-pressure option, at each pseudo-pressure below the saturation pressure, the model iterates to find a gas saturation such that the ratio of the **immiscible** mobilities (that is, using immiscible relative permeability) of oil and gas is equal to the ratio of the total liquid moles (L) to total vapor moles (V). Hence:

$$\frac{k_{roI}b_o}{\mu_o} \frac{\mu_g}{k_{rgI}b_g} = \frac{L}{V} \quad \text{Eq. 10.204}$$

The ratio of the immiscible relative permeabilities is calculated at this gas saturation,

$$R = k_{rgI}/k_{roI} \quad \text{Eq. 10.205}$$

Note that, in block-to-block flow calculations, the gas saturation is available so k_{rgI} and k_{roI} are calculated directly at that saturation in order to calculate the ratio R .

The miscible gas relative permeability is calculated from

$$k_{rgM} = k_{rg}^0 \left\{ \frac{1}{1 + (1/R)} \right\} \quad \text{Eq. 10.206}$$

where k_{rg}^0 is the gas relative permeability at $S_g = 1 - S_{wco}$ (and S_{wco} is the connate water saturation).

Pre-2007.1 behavior

Pre-2007.1, the miscible gas relative permeability was calculated using a straight line curve of the form, equation 10.195 but with no change in the residual saturation (that is, X_g has a value of unity).

This behavior can be restored (instead of using equation 10.206) by setting item 99 of OPTIONS3 to a non-zero value.

Calculation of the oil relative permeability

The calculation of the oil relative permeability depends on the value of item 5 in the VELDEP keyword.

- If VELDEP item 5 has a value of 1, then

$$k_{ro} = f_{oI}k_{roI} + (1 - f_{oI})k_{roM}$$

where the miscible oil relative permeability is of the form given by equation 10.195 but with no change in the residual saturation (that is, X_o has a value of unity), and the immiscible-miscible transition function, f_{oI} , is the same as that used in the original model for capillary number effects in oil flows (see equation 10.197) so

$$f_{oI} = N_{cnp}^{1/n_p}$$

The calculation of f_{oI} requires n_{1o} , n_{2o} and N_{cbo} (and the selection of the capillary number model using VELDEP item 1). Hence it increases the number of parameters required by the alternative model from 2 to 5.

- If VELDEP item 5 has a value of 2, then

$$k_{ro} = k_{rg}/R$$

with R calculated using equation 10.205. This is the preferred method for achieving the desirable property that the alternative model only depends on two parameters.

Pre-2007.1 behavior of the alternative model

Pre-2007.1, the alternative model did not use equation 10.204, but instead found a gas saturation such that the ratio of the **modified** mobilities of oil and gas is equal to the ratio of the total liquid moles (L) to total vapor moles (V). That is

$$\frac{k_{ro}b_o}{\mu_o} \frac{\mu_g}{k_{rg}b_g} = \frac{L}{V}$$

where

b_p is the phase molar density and

μ_p is the phase viscosity.

This pre-2007.1 behavior can be recovered by setting [item 99](#) of `OPTIONS3` to a non-zero value.

Notes

- This alternative model for CN effects in gas flows cannot be used at the same time as the original CN model for **gas** flows. If `VELDEP` keyword item 5 is set to 1, then the alternative model can be used in conjunction with the original model for CN effects in **oil** flows. If `VELDEP` keyword item 5 is set to 2, then it cannot be used with the original model. The selection of models is controlled by the `VELDEP` keyword.
- Both CN and Forchheimer effects may be included in the generalized pseudo pressure option that calculates a blocking factor due to near well condensate dropout (see "[Special inflow equations](#)"). This is controlled by the `PICOND` keyword.

Non-Newtonian fluid flows

x	Eclipse 100
	Eclipse 300

Simulation of non-Newtonian fluid flows requires a significant change to the traditional Darcy flow equation. Indeed the rheology of the fluid, which can be described by an apparent or effective viscosity, may be a non-linear function of the flow rate and may as well depend on the rock properties such as the permeability and the porosity. In order to remain as close to the Darcy form as we can, we introduce a multiplier B that modifies the mobility of the fluid depending the cell properties and local flow rate. The solution of the non-linear equation is obtained by local Newton iteration backed up by a secant method.

The multiphase Darcy equation is therefore modified as follows:

$$Q = AK(k_r/\mu)B\Delta P \quad \text{Eq. 10.207}$$

where B is **positive** and may be a complex function involving grid properties, flow rate or pressure drop, and the rheology of the fluid (models are provided below):

$$B = B(d, K, \phi, Q, |\Delta P|, n, \tau, \delta, \beta) \quad \text{Eq. 10.208}$$

The rheology (n, τ, δ, β) is set to depend on the polymer concentration (used as a tracer) that allows having a distributed rheology in the reservoir.

As B is dependent on the flow rate, we need to solve the flow between two adjacent cells and the flow at the connections between the wells and the reservoir.

Simulation of Non-Newtonian fluid flows

Flow between two cells in the reservoir

The flow between two adjacent cells i and j in the reservoir in a given direction is the solution of the following flow equation:

$$\Delta P_{ij} = \frac{\mu_{ij}}{k_{rij}} \left[\frac{d_i}{AK_i B_i(\dots, Q, \dots)} \frac{Q}{\mu_{ij}} + \frac{d_j}{AK_j B_j(\dots, Q, \dots)} \frac{Q}{\mu_{ij}} \right] \quad \text{Eq. 10.209}$$

where k_{rij}/μ_{ij} is the upstream mobility. This equation is solved numerically using a Newtonian algorithm backed up with a secant method. The solution of this equation leads to:

$$Q = \begin{cases} \frac{k_{rij}}{\mu_{ij}} T^* \Delta P_{ij} & \text{if } (B_i > 0) \text{ and } (B_j > 0) \\ 0 & \text{otherwise} \end{cases} \quad \text{Eq. 10.210}$$

where the new transmissibility T^* is

$$T^* = \frac{1}{\frac{d_i}{AK_i B_i(\dots, Q, \dots)} \frac{Q}{\mu_{ij}} + \frac{d_j}{AK_j B_j(\dots, Q, \dots)} \frac{Q}{\mu_{ij}}} \quad \text{Eq. 10.211}$$

Flow at the well connection

For the flow between the well and the reservoir grid block, we assume that there is no preferred direction for the non-Newtonian fluid. Under that simplification we keep the connection factor unchanged, so the well flow rate writes:

$$Q = T_w MB \Delta P \quad \text{Eq. 10.212}$$

where

T_w is the well connection factor

M is the mobility

B is the non-Newtonian modifier, function of the flow rate Q

ΔP is the pressure drop (draw)

Equation 10.212 is also solved by local Newtonian iteration backed up with a secant method to find the flow rate and therefore the right modifier value.

Note: The assumption that there is no preferred flow path at the connection between the well and the grid block containing the well may become questionable if the degree of anisotropy (ratio between the permeabilities in the plane containing the completion) becomes significant. Because of the shear thinning effect the injectivity can become direction dependent.

Unit conversion

Equations 10.207 and 10.212 are only valid when expressed in cgs units. In addition to the unit factor conversion used for Newtonian fluids, we need to introduce another coefficient to take into account the conversion for the B factor. For other units systems used in Eclipse we therefore introduce a coefficient C_u such as:

$$C_u = \frac{B_{cgs}}{B^*} \quad \text{Eq. 10.213}$$

where

B_{cgs} is the non-Newtonian modifier expressed in cgs units and

B^* is expressed in other units.

Non-Newtonian model: Herschel-Bulkley fluids

Herschel-Bulkley fluids are described by the following non-Newtonian modifier

$$B = \begin{cases} \frac{\left(1 - \frac{d\alpha\tau}{|\Delta P|}\right)}{\frac{1}{12}\left(9 + \frac{3}{n}\right)^n (72\delta K)^{(1-n)/2} \left(\frac{|Q|}{A}\right)^{1-n}} & \text{if } \left(\frac{d\alpha\tau}{|\Delta P|}\right) < 1 \\ 0 & \text{otherwise} \end{cases}$$

where:

Q is the volumetric flow rate

K is the rock permeability

n is the power index $0 < n \leq 1$

A is the area through which flow occurs

d is the distance between cells

τ is the yield stress

α is a weighing factor for the yield. $\alpha = \sqrt{\frac{\phi}{2}} \sqrt{\frac{\delta}{K}}$, with the porosity ϕ

δ is the tortuosity

ΔP is the pressure difference between cells

As mentioned earlier the rheological parameters n , τ , δ are function of the polymer concentration to allow spatial distribution of the fluid properties. The unit conversion C_u is now also dependent on n

$$C_u = \left(\frac{Q_{cgs}}{Q^*} \right)^{1-n} \left(\frac{A^*}{A_{cgs}} \right)^{1-n} \left(\frac{K_{cgs}}{K^*} \right)^{(1-n)/2} \quad \text{Eq. 10.214}$$

	Metric	Field	Lab
C_u (HB.)	$\left(\frac{\sqrt{10}}{86.4} \right)^{1-n}$	$\left(\frac{541.16844955}{8640} \right)^{1-n}$	$\left(\frac{\sqrt{10}}{6} \right)^{1-n}$

Table 10.4: Units of conversion, Herschel-Bulkley fluids

Usage

This model applies only to aqueous phase (water and polymer). To activate the model the following keywords are required:

Section	Keyword	Description
RUNSPEC	POLYMER	activate the polymer flood option
RUNSPEC	NNEWTF	activate the Herschel-Bulkley model and provides some table sizes
REGIONS	HBNUM	specifies which table in FHERCHBL is to be used for each cell in the simulation.
PROPS	FHERCHBL	provides the look up tables for the rheological parameters as a function of the polymer concentration.

Velocity calculations

x	Eclipse 100
x	Eclipse 300

In a typical reservoir simulation, inter-cell flows are calculated but the fluid velocities are not explicitly calculated. Velocities are only calculated for specific models. These include:

- polymer shear-thinning effects ("[Treatment of the non-Newtonian rheology](#)")
- velocity-dependent relative permeabilities and the Forchheimer effect, ("[Non-Darcy flow](#)")
- "[Asphaltene deposition](#)"
- tracer dispersion
- miscibility effects due to surfactant ("[Surfactant model](#)")
- gas mobility reduction due to foam injection ("[Gas mobility reduction](#)").

Fluid velocities inside the well are also calculated and are discussed in:

- "[Multisegment wells](#)"
- the "[Wellbore friction option](#)".

This chapter describes the way reservoir velocities are calculated.

Velocity concepts

Velocities can be determined from flows, which are calculated across the cell boundaries. This presents two problems. Firstly flows are usually calculated after the fluid properties are calculated (since they themselves depend on the fluid properties), and secondly, flows depend on the pressure and fluid properties in both the upstream and downstream cells. Therefore, unlike other fluid properties, the velocity is a function of the solution in a cell and in the neighboring cells. For these two reasons, velocity terms are often used explicitly, that is they are calculated at the previous timestep, so they can be determined before they are used and the derivative terms can be ignored.

The velocity is a vector, with x , y and z components. The cell boundary velocities represent the components of the velocity in the direction normal to the boundaries. To calculate a single-cell velocity it is convenient to assume the cell boundaries are orthogonal and to use the resultant velocity (with the local cell direction components rather than global Cartesian components):

$$\|v\| = \sqrt{v_x^2 + v_y^2 + v_z^2} \quad \text{Eq. 10.215}$$

where the velocity components are calculated by averaging the flows in and out on opposite faces:

$$v_x = (v_x^+ + v_x^-)/2 \quad \text{Eq. 10.216}$$

This method of averaging fails to deal with the case when a cell may have more than two neighbors in each direction, for example if it is connected to a local grid refinement (LGR) or if there are other non-neighbor connections. In this case, an area-weighted average can be applied more consistently:

$$v_x = (\Sigma A_x v_x)/(\Sigma A_x) \quad \text{Eq. 10.217}$$

where the sum is over all faces (positive and negative) in the same direction. Here the sense of the velocities is in the $+x$ direction, but see below.

If a cell contains a well, then it would be expected that the velocities near the well would be much larger than the background flow velocities. Well flows are not included in these calculations, and the only way at present to capture the well velocity would be to have sufficiently small grid cells around the well, so that

the velocity into the cell represents the velocity close to the well. In these cases equation 10.217 can be reinterpreted to represent the average inflow velocity into the cell (the sense of v_x is into the cell rather than $+x$), as, with the original interpretation of equation 10.217, then if the flow is into the well in all directions, the net velocity is zero.

Velocity definitions

Different models use different definitions of velocity. For example, the Darcy velocity u_{Dp} , interstitial velocity u_{Ip} or phase velocity u_p may be used, which are calculated in slightly different ways:

$$u_{Dp} = \frac{Q_p}{A} \quad \text{Eq. 10.218}$$

$$u_{Ip} = \frac{Q_p}{A\varphi} \quad \text{Eq. 10.219}$$

$$u_p = \frac{Q_p}{A\varphi S_p} \quad \text{Eq. 10.220}$$

where

Q_p is the phase volume flow,

A is the cell boundary area,

φ is the porosity,

S_p is the mobile phase saturation.

The Darcy velocity does not take into account the porosity, which limits the flow area. Neither the Darcy velocity nor the interstitial velocity take account of the phase saturations, and so they are similar to superficial velocities used in well flow modeling. If the phase velocity, equation 10.220 is used, but is averaged over the entire fluid volume, by multiplying by the saturation, the resulting average velocity is the interstitial velocity, equation 10.219. All these velocities are cell-scale velocities and are often used to approximate pore-scale phenomena. As such, the choice of which one to use is somewhat arbitrary, and model parameters can be adjusted accordingly, as long as it is clear which velocity is being used.

Eclipse 100 velocity calculations

The primary variable calculated by Eclipse 100 is the mass flow for each phase through a cell face. The volumetric flow is obtained from this by dividing by the density (equivalently, multiplying by the formation volume factor). The area this flows through is the part of the cell face area occupied by those pores which are occupied by the mobile fraction of this phase. The velocity is obtained from the volumetric flow by dividing by this flow area. This is equivalent to the phase velocity:

$$u_p = \frac{B_p F_p}{A\varphi(S_p - S_c)} \quad \text{Eq. 10.221}$$

where:

u_p is the velocity of phase p ,

B_p is the formation volume factor of phase p ,

F_p is the volumetric flow of phase p ,

A is the area of the cell face through which it flows

φ is the porosity at the cell face,

S_p is the saturation of phase p , and

S_c is the equivalent critical saturation.

Before the 2009.1 version, the correction for the mobile saturation was not used. The older formulation can be restored by setting [item 137](#) of the `OPTIONS` keyword to 1.

The polymer shear thinning option still uses the pre-2009.1 calculation to be consistent with its original validation. To output these velocities for visualization, again use `OPTIONS 137`.

Capillary number

The capillary number is used in both the "[Foam model](#)" and the "[Surfactant model](#)". The capillary number represents the ratio of shear forces to capillary (surface tension) forces and is often defined as:

$$N_c = \frac{u_p \mu_p}{\sigma_{pq}} \quad \text{Eq. 10.222}$$

where:

u_p is the velocity of phase p ,

μ_p is the viscosity of phase p ,

σ_{pq} is the surface tension between phases p and q .

The velocity across a cell face can be expressed in terms of the volume flow Q_p which can in turn be expressed in terms of the transmissibility T and phase potential P_p :

$$u_p = \frac{Q_p}{A_m} = \frac{C_N}{C_D} \frac{T k_{rp} \Delta P_p}{A_m \mu_p} \quad \text{Eq. 10.223}$$

where:

C_N is a unit conversion constant,

C_D is the Darcy constant,

A_m is the flow area, which could be A , $A\varphi$ or $A\varphi S_p$ depending on the velocity model, and

k_{rp} is the relative permeability of phase p .

When equation [10.223](#) is substituted into equation [10.222](#), the capillary number (for a boundary face) is:

$$N_{cn} = \frac{C_N}{C_D} \frac{T k_{rp} \Delta P_p}{A_m \sigma_{pq}} = \left(k_{rp} \frac{A}{A_m} \right) \cdot \left(C_N \frac{K}{\sigma_{pq}} \frac{\partial P_p}{\partial n} \right) \quad \text{Eq. 10.224}$$

where:

A is the cell boundary area,

n is a direction, x , y or z .

The $k_{rp} \frac{A}{A_m}$ term in equation 10.224 is often ignored:

$$N_{cn} = \frac{C_N}{C_D} \frac{T \Delta P_p}{A \sigma_{pq}} = \left(C_N \frac{K}{\sigma_{pq}} \frac{\partial P_p}{\partial n} \right) \quad \text{Eq. 10.225}$$

This is equivalent to using the Darcy velocity ($A_m = A$), and assuming the flow is for a single phase ($k_{rp} = 1$). The cell capillary number is then determined from the boundary capillary numbers:

$$N_c = \frac{C_N}{C_D} \left\| \frac{T \Delta P_p}{A} \right\| \frac{1}{\sigma_{pq}} = C_N \frac{\|K \cdot \nabla P_p\|}{\sigma_{pq}} \quad \text{Eq. 10.226}$$

The value of the unit conversion constant C_N depends on the units of the terms used in equation 10.226 as shown in the following table.

	Metric	Field	Lab	PVT-M
K	mD	mD	mD	mD
ΔP_p	bar/m	psi/ft	atm/cm	atm/m
σ_{pq}	N/m	lbf/in	dyne/cm	N/m
C_N	$10^{-10}(\text{bar/atm}) = 0.9869234 \cdot 10^{-10}$	$12 \cdot 10^{-15} (\text{m/ft})^2 (\text{bar/atm}) = 1.274784 \cdot 10^{-13}$	10^{-5}	10^{-10}
C_D	0.00852702 mD.day.bar/m ² /cP	0.001127 mD.day.psi.(ft/ bbl)/cP	3.6 mD.hr.atm/cm ² /cP	0.00864 mD.day.atm/m ² /cP

Table 10.5: Capillary number constant and Darcy Constant

For two phases p and q there are two “natural” capillary numbers that could be used:

$$N_{cpq} = \frac{v_p \mu_p}{\sigma_{pq}}$$

or:

$$N_{cqp} = \frac{v_q \mu_q}{\sigma_{pq}}$$

Eclipse 100 capillary numbers

Eclipse 100 uses the second formulation, equation 10.226 for capillary numbers.

In the surfactant model, the oil phase potential is used together with the oil-water surface tension to determine the capillary number:

$$N_{cow} = C_N \frac{\|K \cdot \nabla P_o\|}{\sigma_{ow}} \quad \text{Eq. 10.227}$$

In the foam model, the oil phase potential is again used together with the water-gas surface tension:

$$N_{cwg} = C_N \frac{\|K \cdot \nabla P_o\|}{\sigma_{wg}} \quad \text{Eq. 10.228}$$

In both these models an RMS value of $\|K \cdot \nabla P_o\|$ is used, after the components have been averaged in a manner analogous to equation 10.215:

$$\|K \cdot \Delta P\| = \sqrt{\left(K_x \cdot \frac{\partial P}{\partial x}\right)^2 + \left(K_y \cdot \frac{\partial P}{\partial y}\right)^2 + \left(K_z \cdot \frac{\partial P}{\partial z}\right)^2} \quad \text{Eq. 10.229}$$

which for a given cell i with neighbor j is calculated in the x direction according to:

$$K_x \cdot \frac{\partial P}{\partial x} = 0.5 \left[\left(\frac{K_x}{D_x} \right)_{i-1, i} \cdot (P_i - P_{i-1}) + \left(\frac{K_x}{D_x} \right)_{i, i+1} \cdot (P_{i+1} - P_i) \right] \quad \text{Eq. 10.230}$$

and similarly for the y and z directions. The K/D value is calculated in an analogous manner to the transmissibility and depends upon how the geometry has been specified.

Eclipse 300 capillary numbers

Eclipse 300 uses the first formulation in equation 10.226 for capillary numbers.

In the surfactant model, the oil phase potential is used together with the oil-water surface tension:

$$N_{cow} = \frac{C_N}{C_D} \left\| \frac{T \Delta P_o}{A} \right\| \frac{1}{\sigma_{ow}} \quad \text{Eq. 10.231}$$

In the foam model, the gas phase potential is used together with the water-gas surface tension:

$$N_{cwg} = \frac{C_N}{C_D} \left\| \frac{T \Delta P_g}{A} \right\| \frac{1}{\sigma_{wg}} \quad \text{Eq. 10.232}$$

The boundary terms are averaged in a manner analogous to equation 10.216 before the RMS value is calculated.

Matrix-fracture flows

In dual porosity simulations, the matrix-fracture flow contribution can be calculated in a similar manner to equations 10.231 and 10.232 but with the area term calculated by

$$A = V \sqrt{\sigma} \quad \text{Eq. 10.233}$$

where V is the bulk volume of the cell and σ is the sigma factor representing the matrix block size. For fracture cells this contribution to the capillary number is likely to be small and will be ignored.

Asphaltene deposition and entrainment

In the Eclipse 300 three-component asphaltene model (see "Deposition - entrainment") the oil phase velocity is calculated using an RMS of the directional velocities:

$$\overline{U_o} = C \left\| \frac{k_{ro} T \Delta P_o}{\mu_o A} \right\| \frac{1}{\phi(1-S_g)S_o} \quad \text{Eq. 10.234}$$

Here the relative permeability is included in the flow term, and the effective area of flow includes the porosity, oil saturation and solid saturation (representing deposited asphaltene).

The unit conversion constant is given in the following table.

	Metric	Field	Lab	PVT-M
T	cP.m ³ /day/bar	cP.bbl /day/psi	cP.cm ³ /hr/Atm	cP.m ³ /day/Atm
ΔP_o	bar	psi	Atm	Atm
μ_o	cP	cP	cP	cP
A	m ²	ft ²	cm ²	m ²
C	1	5.61458 ft ³ /bbl	1	1
$\overline{U}_o$	m/day	ft/day	cm/hr	m/day

Table 10.6: Velocity constant